

Shorter, Tighter, FAESTer: Optimizations and Improved (QROM) Analysis for VOLE-in-the-Head Signatures^{*}

Carsten Baum¹, Ward Beullens², Lennart Braun³, Cyprien Delpech de Saint Guilhem⁴, Michael Klooß^{5**}, Christian Majenz¹, Shibam Mukherjee^{6,7}, Emmanuela Orsini⁸, Sebastian Ramacher⁹, Christian Rechberger⁶, Lawrence Roy¹⁰, and Peter Scholl¹⁰

¹ Technical University of Denmark

² IBM Research Europe

³ Université Paris Cité, CNRS, IRIF, F-75013, Paris, France

⁴ 3MI Labs

⁵ Karlsruhe Institute of Technology

⁶ Graz University of Technology

⁷ Know Center

⁸ Bocconi University

⁹ Austrian Institute of Technology

¹⁰ Aarhus University

team@faest.info

Abstract. In the past decade and largely in response to the NIST standardization effort for post-quantum cryptography, many new designs for digital signatures have been proposed. Among those, the FAEST digital signature scheme (Baum et al., CRYPTO 2023) stands out due to its interesting security-performance trade-off. It only relies on well-tested symmetric-key cryptographic primitives, as it constructs a digital signature from a zero-knowledge (ZK) proof of knowledge of an AES key. To achieve this, it uses the VOLE-in-the-Head ZK proof system which relies only on pseudorandom generator (PRG) and hash function calls. FAEST simultaneously has relatively small signature size and competitive sign and verify times.

In this work, we improve both the security and practical efficiency of FAEST. We improve the main computational bottleneck of the original construction by replacing hash function calls in the underlying vector commitment scheme with calls to an AES-based PRG. At the same time, we also improve the signature size by revisiting the evaluation of the AES block cipher in ZK. We use observations from Galois Theory to compress the size of the witness (and thus signature), due to the algebraic nature of the AES S-Box. We implemented our new construction, and our benchmarks show that its sign and verify times reduce up to 50% over the state-of-the-art while achieving the same security and smaller signatures.

Finally, we analyze our resulting signature scheme both in the Quantum Random Oracle Model (QROM) and its classical analogue. To achieve concretely good security bounds, we devise a new classical proof for FAEST based on Renyi divergence techniques. We construct a QROM analogue and present a new Fiat-Shamir transform which is applicable to VOLE-in-the-Head-based signature schemes.

^{*} This article is the full version of an article with the same name [BBB⁺25], © IACR 2025.

^{**} Most research conducted while at ETH Zurich.

Table of Contents

Shorter, Tighter, FAESTer: Optimizations and Improved (QROM) Analysis for VOLE-in-the-Head Signatures	1
<i>Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpéch de Saint Guilhem, Michael Kloß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl</i>	
1 Introduction	4
Our Contributions.	4
Related Work.	5
2 Preliminaries	5
2.1 FAEST Signature Scheme	6
2.2 EUF-CMA and EUF-KO Security if Signature Schemes	7
2.3 Batch All-but-One Vector Commitments	8
2.4 Universal Hash Functions	9
Standard Constructions.	10
Composition and Truncation of Hashes.	10
Constructions.	10
VOLE Universal Hash.	10
ZK Universal Hash.	11
2.5 Interactive Proof Systems	12
2.6 Hardness Assumptions for Symmetric Primitives	13
IV-based Tweakable PRGs.	13
Divergence Analysis for PRF-type Security.	14
2.7 The Compressed Oracle Technique	16
3 VOLE Commitments	17
PRG-Based Leaf Commitments.	17
Analysis of Simple Leaf Commitments	17
3.1 Batch Vector Commitments	19
BAVC Correctness.	19
3.2 Converting BAVC to VOLEs	22
From Seeds to VOLEs.	22
VOLE Commitments and Reconstructions.	22
VOLE Commitment.	23
VOLE Reconstruction.	23
3.3 VOLECommit is Binding	24
Formal Proof	25
3.4 Additional Properties: Hiding and Unpredictability	28
Hiding.	29
Unpredictability.	31
4 The Optimized FAEST Signature Scheme	32
4.1 Optimized Constraints in FAEST	32
4.2 The FAEST Algorithms	33
5 EUF-CMA Security of FAEST and FAEST-EM	36
5.1 Switching to False Statements.	39
5.2 EUF-KO	40
Interactive Proof Induced by FAEST.	40
From EUF-KO to SR Soundness.	41
From SRS IP to SRS IOP.	44
FAEST IOP is RBR-sound.	47
5.3 EUF-CMA	49
6 QROM Proofs	53
6.1 Challenge 1: No Tight Generic Reduction for Fiat-Shamir	53

6.2	Challenge 2: Proving Tight Soundness of Fiat-Shamir from RBR Soundness.....	54
6.3	Challenge 3: Reprogramming a QRO at a Computationally Unpredictable Input.....	56
6.4	QROM Security of FAEST	57
6.5	Proof Overview	58
6.6	FAEST as Non-Standard Fiat-Shamir Transformation of 7-Challenge IP (6-Challenge for FAEST-EM).....	58
6.7	RBR Soundness of FAEST-IP ⁺	59
6.8	FAEST _r ^{elc} Soundness to FAEST-IP ⁺ RBR Soundness.....	59
6.9	FAEST EUF-KO $\Leftarrow$ FAEST _r ^{elc} Soundness.....	62
6.10	Quantum-Guess Unpredictability of VOLECommit.....	63
6.11	FAEST EUF-CMA $\Leftarrow$ FAEST EUF-KO	64
7	Implementation and Benchmarks.....	66

1 Introduction

Digital signatures are among the most fundamental building blocks in cryptography. Due to the well-known insecurity of currently deployed RSA- and discrete logarithm-based signature algorithms against quantum computers, the US National Institute of Standards and Technology (NIST) has already begun the standardization of three post-quantum signature schemes. However, in light of shortcomings of the recently standardized signature algorithms, NIST has called for the submission of additional candidate post-quantum signature schemes.¹¹ One of the algorithms currently under consideration in this call is the FAEST signature scheme [BBD⁺23, FAE23]. Similar to the already standardized SPHINCS+ signature scheme [BHK⁺19], FAEST only relies on symmetric cryptographic primitives. While having a larger verification time than SPHINCS+, FAEST outperforms it in every other performance metric. To achieve this, it uses the recently introduced VOLE-in-the-Head (VOLEitH) zero-knowledge (ZK) proof system to prove knowledge of an AES key. This makes FAEST an interesting *conservative* choice for a digital signature, as it avoids (potentially less-analyzed) public-key assumptions.

When compared with standardized schemes based on lattices such as the well-known Dilithium [Dil17], the performance of FAEST is, however, not competitive. In particular when it comes to computation time, this appears inherent due to VOLEitH: it shares some core building blocks with signature schemes based on the popular MPC-in-the-Head (MPCitH) paradigm [IKOS07, CDG⁺17, KKW18, BDK⁺21]. These have a major performance bottleneck, which is the generation and reconstruction of vector commitments (or, “PRG trees” based on the GGM paradigm). Essentially all recent signature schemes based on MPCitH, but also FAEST, use a commitment due to Katz et al. [KKW18]. Their construction has a very small commitment and opening size (recently improved upon in [BBM⁺24]). It however calls a cryptographic hash function such as SHA-3 many times. Due to the slow performance of hashing compared with AES, which has hardware support on modern processors, the inherent slowdown from committing to vectors (or opening commitments) can make up a substantial percentage of the overall sign/verify time. At the same time, this slowdown appears inherent: the collision-resistance of these hash function calls guarantees the binding property.

Another disadvantage over Dilithium and other signatures is the size. While outperforming SPHINCS+ and previous MPCitH-based signatures such as Picnic [CDG⁺17, KZ20] or Banquet [BDK⁺21] substantially, recent work [BBM⁺24] has shown that the limiting factor to VOLEitH signature size is not necessarily the proof system, but the structure of the proven circuit. FAEST evaluates AES inside the ZK proof, but uses a rather direct conversion of AES into a circuit. Moreover, although FAEST is claimed to be post-quantum secure by the authors, this was only heuristically proven in [FAE23]. Their QROM proof relies on an additional conjecture and the proof reduction has a large loss (compared to the classical proof).

Our Contributions. In this work, we improve the FAEST signature scheme and the VOLE-in-the-Head paradigm in three core aspects.

Improved Batch All-but-One Vector Commitments. We revisit the vector commitment construction by Katz et al. and replace the majority of the hash-based commitments with a statistically binding commitment scheme based on a PRG and a universal hash function. This is reminiscent of a well-known commitment scheme by Naor [Nao91] and improves computational efficiency substantially. The resulting vector commitment is statistically secure in the Random Oracle Model, but only has a computationally inefficient extractor. We show that a twist to the FAEST construction allows us to accommodate this in an updated security proof. Here, we use an approach due to Katz and Wang [KW03] to reduce EUF-KO security to soundness instead of knowledge soundness. Compared with [KW03], we employ an additional lossiness argument to be able to replace the public key by one which has no valid matching secret keys.

We additionally present a variant of the PRG-based commitment that uses 2λ bits of AES output as a commitment; its analysis requires a novel assumption that is a strengthening of collision-resistance of multiple blocks of AES output.

Shorter VOLE-Proofs for AES Evaluation. In FAEST and prior MPCitH-based signatures using AES [DDOS19, BDK⁺21], the dominant cost of proving AES in ZK is the need to commit to the input of every S-box as

¹¹ <https://csrc.nist.gov/Projects/pqc-dig-sig/standardization>

part of the witness. These works rely on linearity of the commitments to evaluate the linear layers in AES, and prove consistency of every S-box input/output pair using the algebraic structure of the S-box. Recently, [BBM⁺24] observed that, by relying on techniques from the QuickSilver proof system [YSWW21] that allow for proving constraints of degree > 2 , it's possible to prove an AES evaluation by only committing to the S-Box output of every second round. However, this requires degree-7 constraints, which is relatively complex and has not been implemented. In this work, we show a trade-off: by using elementary tools of Galois Theory, we include S-box output decompositions for every second AES round, and information that is 50% smaller for the intermediate rounds. Our new circuit only needs degree-3 constraints, and can be computed very efficiently in VOLEitH.

QROM Proof. We give a proof of EUF-CMA security in the quantum-accessible random oracle model that reduces the EUF-CMA-security of the VOLEitH signature algorithm FAEST to the PRG and PRF security of the standard symmetric key primitives AES and SHAKE. The multiplicative reduction loss is a small constant (around 8 for FAEST parameters) and all additive loss terms are at most a Grover's search advantage bigger than the corresponding terms in the ROM security theorem. To achieve this, we develop three technical and conceptual innovations and present them in a general setting to ease reuse.

1. We re-conceptualize Fiat-Shamir signature constructions based on interactive proof systems (IPs) in the (Q)ROM as non-standard Fiat-Shamir transformation of a different IP in the plain model.
2. We provide a tight security theorem of a class of FS transformations (that includes the standard variant and our non-standard one) based on round-by-round soundness of the underlying IP.
3. We define quantum list unpredictability as a security property of cryptographic algorithms, to be able to generalize the efficient Rényi-divergence-based ROM technique used in the security proof of FAEST to the QROM.

We have fully implemented our improved signature scheme. Our modifications reduce sign/verify time up to 60% compared to [BBM⁺24], and signature sizes by up to 10%. At the 128-bit security level, the smallest version of our signature is, e.g., 3.9kB, while sign/verify take just 2–3ms on a fast laptop.

Related Work. There is a vast body of work on MPCitH-based ZK proofs and signatures. Here, we mention a few works not discussed so far, which have techniques or results related to ours. Bui et al. [BCD24] recently proposed to use AES-based commitments in VOLEitH vector commitments. Their analysis uses the ideal cipher model, while our first variant based on universal hashing only requires a PRG, while our second variant (which is essentially the same as theirs) uses a new collision-resistance style assumption, but does not require any ideal models. Another recent work [KLS24] further improves on this by relaxing the binding property of vector commitments to allow for several possible valid preimages, which reduces signature size for their MPCitH-based signature. We believe this approach can also be applied to VOLEitH, but it would require a significantly more complex analysis, since it means the VOLE consistency check can no longer be analysed with the modular approach we take. [BCC⁺24] shows how to properly salt GGM trees to improve concrete security. We apply similar salts throughout our construction, using a per-signature IV and additional tweaks to domain-separate different uses of AES.

Several MPCitH protocols have been analysed in the QROM including, e.g., Picnic [DFMS19, DFMS22b, DFMS22a] and the Syndrome-Decoding-in-the-Head signature scheme [AHJ⁺23]. Additionally, there is a more general straightline extractability result for multi-round protocols with a certain structure [HJMN24]. Unfortunately, none of these techniques yield satisfying results for FAEST and VOLEitH: the generic reductions lack tightness, while the straightline extraction results either only apply to signatures built from interactive proofs with up to 5 rounds, or require some form of special soundness.

2 Preliminaries

Notation. Let λ be the security parameter. For a function $F: \mathcal{X} \rightarrow \mathcal{Y}$, we write $\text{dom}(F) = \mathcal{X}$ for the domain of F , $\text{cod}(F) = \mathcal{Y}$ for the codomain of F , and $\text{im}(F) = \{F(x) \mid x \in \mathcal{X}\} \subseteq \mathcal{Y}$ for the image of F .

AddToUpperWord ($iv = (iv_0, \dots, iv_3), \alpha$)	AddToLowerWord (iv, α)
INPUT: $iv_i \in \{0, 1\}^{32}, \alpha \in [0..2^{32})$ OUTPUT: $iv' \in \{0, 1\}^{128}$	INPUT: $iv_i \in \{0, 1\}^{32}, \alpha \in [0..2^{32})$ OUTPUT: $iv' \in \{0, 1\}^{128}$
1 : $\quad // \quad iv_3$ is “upper” 32 bits in little-endian order 2 : $iv'_3 \leftarrow \text{NumRec}(32, iv_3) + \alpha \bmod 2^{32}$ 3 : return ($iv_0, iv_1, iv_2, \text{BitDec}(iv'_3)$)	1 : $\quad // \quad iv_0$ is “lower” 32 bits in little-endian order 2 : $iv'_0 \leftarrow \text{NumRec}(32, iv_0) + \alpha \bmod 2^{32}$ 3 : return ($\text{BitDec}(iv'_0), iv_1, iv_2, iv_3$)

PRG ($sd, iv, twk; m$)
INPUT: $sd \in \{0, 1\}^\lambda, iv \in \{0, 1\}^{128}, twk \in \{0, 1\}^{32}, m \in \mathbb{N}$ OUTPUT: $s \in \{0, 1\}^m$
1 : $h := \lceil m/128 \rceil$ 2 : $r := m - (h - 1) \cdot 128$ 3 : $\bar{k} \leftarrow \text{AES-}\lambda.\text{KeyExpansion}(sd)$ 4 : $iv' \leftarrow \text{AddToUpperWord}(iv, twk)$ 5 : for $i \in [0..h)$ do 6 : $s_i \leftarrow \text{AES-}\lambda.\text{Encrypt}(\bar{k}, \text{AddToLowerWord}(iv', i))$ 7 : return $s_0 \parallel \dots \parallel s_{h-2} \parallel s_{h-1}[0..r-1]$

Fig. 1. Tweakable PRG based on AES-CTR

PRG and Hash Functions. Our construction uses a tweakable **PRG**: $\{0, 1\}^\lambda \times \{0, 1\}^{128} \times \{0, 1\}^{32} \rightarrow \{0, 1\}^*$ that we instantiate with AES- λ in CTR mode. The call **PRG**($sd, iv, twk; m$) uses sd as AES key and produces m bits of output by encrypting the IV iv with the tweak twk added to the upper 32-bit word and the counter added to the lower 32-bit word (see Fig. 1).

Moreover, we use $H_0: \{0, 1\}^{128} \rightarrow \{0, 1\}^{3\lambda}$ as hash function for commitment keys; $H_1: \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ as collision-resistant hash for vector commitments; $H_2^j: \{0, 1\}^* \rightarrow \{0, 1\}^*$, for $j \in \{1, 2, 3\}$, as hash function for the j -th Fiat-Shamir challenge (modeled as a random oracle); $H_3: \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda+128}$ as hash function for secret randomness derivation; and $H_4: \{0, 1\}^* \rightarrow \{0, 1\}^{128}$ as hash function for IV derivation. We instantiate them with SHAKE-128 or SHAKE-256 depending on λ and apply domain separation.

2.1 FAEST Signature Scheme

FAEST is a post-quantum digital signature scheme in which the public key is defined via a block-cipher-based encryption function E and a secret key k , so that $\text{pk} = (x, y)$ with $y = E_k(x)$. In spirit, this resembles prior “MPC-in-the-Head” (MPCitH) constructions such as Picnic [CDG⁺17, ZCD⁺20] and Banquet [BDK⁺21], where the signature is derived from a non-interactive zero-knowledge argument of knowledge of k . However, FAEST uses the novel VOLE-in-the-Head (VOLEitH) paradigm introduced in [BBD⁺23] instead, which relies on efficient vector oblivious linear evaluation (VOLE).

The scheme begins by committing via a binary GGM tree to a collection of seeds, each encoding a piece of the simulated VOLE-based protocol. These seeds are arranged in a single batch “all-but-one” vector commitment [BBM⁺24], drastically reducing opening costs by letting the signer reveal just enough sibling nodes to reconstruct all but a few hidden seeds. After setting up the VOLE correlation $(\mathbf{u}, \mathbf{v}, \mathbf{q})$, the next step in FAEST is for the signer to run a variant of the QuickSilver proof system, to prove knowledge of the one-way function key used to generate the public key. At a high level, QuickSilver treats each wire value in the circuit as being authenticated by a VOLE MAC, where the prover holds $(\mathbf{w}, \mathbf{v})$ and the verifier holds $\mathbf{q}$ with a global key Δ , such that $\mathbf{q} = \mathbf{v} + \mathbf{w} \cdot \Delta$. Any attempt to change $\mathbf{w}$ or $\mathbf{v}$ requires guessing Δ , which happens with negligible probability if Δ is in a large enough field, e.g. $\mathbb{F}_{2^\lambda}$.

ZK Phase. More concretely, after the VOLE step that produces random correlations $(\mathbf{u}, \mathbf{v})$ and $(\mathbf{q}, \Delta)$, the protocol proceeds with the prover using its secret key $\mathbf{sk} = k$ and the public input $\mathbf{pk} = (x, y)$ to compute $\text{AES}_k(x)$. During this computation, specific bits related to the S-box inputs and outputs are recorded to form an extended witness $\mathbf{w} \in \mathbb{F}_2^\ell$. To commit to this witness, the prover sends $\mathbf{d} = \mathbf{u} + \mathbf{w}$. This commitment enables the verifier to update its VOLE outputs by calculating

$$q'_i := q_i + d_i \cdot \Delta = v_i + w_i \cdot \Delta.$$

Next, to prove the correctness of the AES evaluation, the protocol requires that the extended witness satisfies a set of C constraints, expressed by functions $f_1, \dots, f_C$ with the property that $f_i(w_0, \dots, w_{\ell-1}) = 0$. These constraints are evaluated in a gate-by-gate manner corresponding to their circuit representations. Linear operations are evaluated directly using the linearity of VOLE commitments, while multiplication operations are handled by viewing each VOLE output (w_i, v_i) as the coefficients of a degree-1 polynomial $\rho_i(X) = w_i \cdot X + v_i$. In this way, with a single multiplication, the signer obtains a degree-2 polynomial $\rho_i(X) \cdot \rho_j(X)$ and the verifier its evaluation $\rho_i(\Delta)\rho_j(\Delta)$. While this process can continue up to higher degree d , in FAEST the maximum degree is 2.

After the constraints are computed, the verifier challenges the prover. Rather than sending C independent random coefficients, the verifier sends a challenge vector $\mathbf{r}$ used to form a single, combined constraint polynomial

$$\hat{f}(X) = \sum_{i=1}^C r_i \cdot p_i(X),$$

which must be zero in the leading coefficient if all constraints have 0 as leading coefficient. The prover then reveals the coefficients $(a_0, \dots, a_d)$ of this polynomial so that the verifier can confirm that the highest-degree coefficient a_d is zero and that the polynomial evaluation matches the value computed from the VOLE outputs. If the constraint were violated, a dishonest prover would succeed with probability at most $d \cdot 2^{-\lambda}$. To maintain zero-knowledge, the protocol masks the coefficients $(a_0, \dots, a_d)$ before they are revealed. The entire protocol is made non-interactive using the Fiat-Shamir transform.

The success probability of a cheating prover can be bounded as follows:

Lemma 1. *Let $\sum_{i=0}^d a_i X^i = f(X) \in \mathbb{F}_{2^\lambda}[X]$ with $a_d \neq 0$. The prover sends $b_0, \dots, b_{d-1}$ to the verifier, which holds $f(\Delta)$ for a uniformly random $\Delta \in \mathbb{F}_{2^\lambda}$. Then $f(\Delta) = \sum_{i=0}^{d-1} \Delta^i b_i$ with probability at most $d/2^\lambda$.*

Proof. Define the polynomial $g(X) = \sum_{i=0}^{d-1} b_i X^i$. Since $a_d \neq 0$, we have that $f(X) \neq g(X)$ and both polynomials coincide for at most d choices of $\Delta \in \mathbb{F}_{2^\lambda}$ (as they are otherwise identical due to the fundamental theorem of algebra). Since Δ is chosen uniformly at random and independent of g , the claim follows. $\square$

Security Levels. The original FAEST algorithm works for three different security levels, achieving 128-, 192- or 256-bit security. For 128 bit security, x is encrypted using AES-128 with key k (just as in FAEST). We encrypt x and $x \oplus 1$ for 192-bit security using the same key under AES-192 and accordingly for 256-bit security.

FAEST with Even Mansour (FAEST-EM). [FAE23] introduced a FAEST variant that uses AES as Even-Mansour cipher called FAEST-EM, where the function we compute for 128-bit security is $\text{AES-128}_x(k) \oplus k$, i.e., we use the publicly known x as AES key. This means that the key schedule does not need be performed as part of the zero-knowledge proof, reducing signature size [DKR⁺22]. For 192 and 256-bit security, FAEST-EM will use Rijndael encryption with 192- or 256-bit block size instead of AES. When we need to adjust our proofs for FAEST-EM (compared to FAEST), we highlight the difference in magenta.

Parameters. Tables 1 and 2 give an overview of the parameters used in FAEST.

2.2 EUF-CMA and EUF-KO Security if Signature Schemes

Definition 2 (EUF-CMA and EUF-KO Security). *A signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is existentially unforgeable against chosen message attacks (EUF-CMA) (resp. existentially unforgeable against key-only attacks (EUF-KO)) in the random oracle model if any PPT adversary has at*

Table 1. Main parameters for FAEST and FAEST-EM

Parameter	Description
λ	Security parameter in $\{128, 192, 256\}$
τ	Number of small-VOLE instances
w_{grind}	Grinding parameter: w upper bits of Δ must be zero
T_{open}	Threshold for the maximum opening size of the GGM tree
n_{leafcom}	Number of λ -bit blocks in each leaf commitment
B	Extra length padding for the VOLE check
$E_k(x)$	One-way function used to generate the public key
ℓ	Witness length for ZK proof (in bits)

Table 2. Extra parameters used in the VOLE-in-the-Head components of FAEST

Parameter	Formula	Description
n_{leafcom}	3 for FAEST, 2 for FAEST-EM	Number of λ -bit blocks in leaf commitments
τ_1	$(\lambda - w_{\text{grind}}) \bmod \tau$	Number of larger small-VOLE instances
τ_0	$\tau - \tau_1$	Number of smaller small-VOLE instances
k	$\lfloor (\lambda - w_{\text{grind}}) / \tau \rfloor + 1$	Bit-length of the larger small-VOLE instances
$k - 1$	$k - 1$	Bit-length of the smaller small-VOLE instances
N_i	2^k if $i < \tau_1$, 2^{k-1} otherwise	Field size in the i -th small-VOLE instance
L	$\sum_{i=0}^{\tau-1} N_i = \tau_1 2^k + \tau_0 2^{k-1}$	Number of leaves in the GGM tree
B	16	Extra length padding for the VOLE check
$\hat{\ell}$	$\ell + 3\lambda + B$	Witness length + extra randomness for VOLE/ZK checks

most negligible advantage of winning the EUF-CMA (resp. EUF-KO) security game (Fig. 2). We denote by $\text{AdvEUF-CMA}_{\mathcal{A}}^{\text{Sig}} = \Pr[\text{ExpEUF-KO}_{\mathcal{A}}^{\text{Sig}} = 1]$ (resp. $\text{AdvEUF-KO}_{\mathcal{A}}^{\text{Sig}} = \Pr[\text{ExpEUF-KO}_{\mathcal{A}}^{\text{Sig}} = 1]$) the advantage of an adversary $\mathcal{A}$. We define existential unforgeable against key-only attacks with random public key, short EUF-RK, except that $\text{pk} \leftarrow \mathcal{PK}$ is chosen from the ambient space of public keys (without knowledge of the secret key); we write $\text{ExpEUF-RK}(\text{pk})$ to make pk explicit.

For FAEST, EUF-KO with random public key samples pk as $\text{pk} = (\mathbf{x}, \mathbf{y}) \leftarrow \{0, 1\}^{2\lambda}$.

2.3 Batch All-but-One Vector Commitments

Definition 3 (VC). Let H be a random oracle. A (non-interactive) batch all-but-one vector commitment scheme VC (with message space $\mathcal{M}$) in the RO model is defined by the following polynomial-time algorithms, which have access to an RO, and take the security parameter 1^λ as well as $\tau, N_0, \dots, N_{\tau-1}$ as inputs:

- Commit**(r, iv) $\rightarrow (\text{com}, \text{decom}, (m_0^{(i)}, \dots, m_{N_i-1}^{(i)})_{i \in [0..\tau)})$: on input some randomness r and IV iv outputs a commitment com with opening information decom for messages $(m_0^{(i)}, \dots, m_{N_i-1}^{(i)})_{i \in [0..\tau)} \in \mathcal{M}^{N_0 + \dots + N_{\tau-1}}$.
- Open**(decom, I) $\rightarrow \text{decom}_I \vee \perp$: On input an opening decom and the index vector $I \subset [0..N_0) \times \dots \times [0..N_{\tau-1})$, output $\perp$ or an opening decom_I for I .
- Reconstruct**($\text{decom}_I, I = (\Delta_0, \dots, \Delta_{\tau-1}), \text{iv}$) $\rightarrow ((m_j^{(i)})_{j \in [0..N_i) \setminus \{\Delta_i\}})_{i \in [0..\tau)} \vee \perp$: Given an opening decom_I , an index vector I as well as the IV iv either output the commitment com and all messages $(m_j^{(i)})_{j \in [0..N_i) \setminus \{\Delta_i\}}$ (accept the opening) or $\perp$ (reject the opening).

ExpEUF-CMA	ExpEUF-KO	ExpEUF-RK	OSign(msg)
1 : $\mathcal{M} := \emptyset$			1 : $\mathcal{M} := \mathcal{M} \cup \{\text{msg}\}$
2 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}^H(\text{param}, \text{param}_{\text{OWF}})$		$\text{pk} \leftarrow \mathcal{PK}$	2 : return SIG.Sign(sk, msg)
3 : $(\text{msg}, \sigma) \leftarrow \mathcal{A}^{H, \text{OSign}}(\text{param}, \text{param}_{\text{OWF}}, \text{pk})$			
4 : return $\text{Verify}^H(\text{msg}, \text{pk}, \sigma; \text{param}, \text{param}_{\text{OWF}}) \wedge \text{msg} \notin \mathcal{M}$			

Fig. 2. The $\boxed{\text{EUF-CMA}}$, resp. EUF-KO, resp. $\boxed{\text{EUF-RK}}$ security experiments in the random oracle model; H is a short-hand for any number of random oracles. Only for EUF-CMA does $\mathcal{A}$ get access to OSign.

We now define correctness for the commitment scheme and allow the sender to potentially abort for certain choices of I during **Open**.

Definition 4 (Correctness with aborts). VC is correct with aborts if for all r, iv and $I = (\Delta_0, \dots, \Delta_{\tau-1})$, where $\Delta_i \in [0..N_i]$, the following outputs *True*:

$$\begin{aligned}
&(\text{com}, \text{decom}, M) \leftarrow \text{Commit}(r, \text{iv}) \\
&\forall \text{decom}_I \leftarrow \text{Open}(\text{decom}, I) \\
&\text{output } \text{decom}_I = \perp \vee \text{Reconstruct}(\text{decom}_I, I, \text{iv}) = (\overline{M}, \text{com})
\end{aligned}$$

where $M = (m_0^{(i)}, \dots, m_{N_i-1}^{(i)})_{i \in [0..\tau]}$ and $\overline{M} = (\overline{m}_0^{(i)}, \dots, \overline{m}_{N_i-1}^{(i)})_{i \in [0..\tau]}$ with $\overline{m}_j^{(i)} = m_j^{(i)}$ for $j \neq \Delta_i$ and $\overline{m}_{\Delta_i}^{(i)} = \perp$ if $j = \Delta_i$.

2.4 Universal Hash Functions

In this work, we use universal hash functions to perform consistency checks.

Definition 5. A family of linear hash functions is a family of matrices $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$. The family is ε -almost universal if for any non-zero $\mathbf{x} \in \mathbb{F}_q^n$, we have

$$\Pr_{\mathbf{H} \leftarrow \mathcal{H}} [\mathbf{H}\mathbf{x} = 0] \leq \varepsilon.$$

The family is ε -almost uniform, if for any non-zero $\mathbf{x} \in \mathbb{F}_q^n$ and for any $\mathbf{v} \in \mathbb{F}_q^r$, we have

$$\Pr_{\mathbf{H} \leftarrow \mathcal{H}} [\mathbf{H}\mathbf{x} = \mathbf{v}] \leq \varepsilon.$$

The hashes must also satisfy the following hiding property:

Definition 6. A matrix $\mathbf{H} \in \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$ -hiding if the distribution of $\mathbf{H}\mathbf{v}$ is independent from $\mathbf{v}_{[0..n]}$ when $\mathbf{v}_{[n..n+h]} \leftarrow \mathbb{F}_q^h$. A hash family $\mathcal{H} \subseteq \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$ -hiding if every $\mathbf{H} \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding.

We will use the following, straightforward method of transforming a uniform hash family into a universal family that is hiding.

Proposition 7. Let $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$ be an ε -almost uniform hash family. Let $\mathcal{H}' \subseteq \mathbb{F}_q^{r \times (n+r)}$ be the family $\{[\mathbf{H} \ I_r] : \mathbf{H} \in \mathcal{H}\}$, where I_r is the $r \times r$ identity matrix. Then, it holds that (1) $\mathcal{H}'$ is ε -almost universal, and (2) $\mathcal{H}'$ is $\mathbb{F}_q^n$ -hiding.

Proof. Let $\mathbf{x} = \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{x}_1 \end{bmatrix}$ be non-zero, for $\mathbf{x}_0 \in \mathbb{F}_q^n$ and $\mathbf{x}_1 \in \mathbb{F}_q^r$. If $\mathbf{H}' \leftarrow \mathcal{H}'$ then, $\mathbf{H}\mathbf{x} = 0$ implies $\mathbf{H}\mathbf{x}_0 = \mathbf{x}_1$, and since at most one of $\mathbf{x}_0, \mathbf{x}_1$ are zero, we cannot have $\mathbf{x}_0 = 0$, so this holds with probability at most ε . For the second part, the hiding property holds because I_r ensures that if the last r elements of the input to the hash are uniform then they perfectly mask the rest. $\square$

Standard Constructions. As building blocks, we use two well-known constructions of linear universal hash families [CW79, BJKS94]. The first is a simple matrix hash family, where $\mathcal{H} = \mathbb{F}_q^{r \times n}$, which is q^{-r} -uniform. The second is a polynomial-based hash, where the input $\mathbf{v} \in \mathbb{F}_q^n$ is parsed as the coefficients of a polynomial of degree up to $n - 1$, and sampling a hash function involves evaluating the polynomial at a randomly chosen point in $\mathbb{F}_q$. Since the polynomial has at most $n - 1$ roots over $\mathbb{F}_q$, this hash family is $(n - 1)/q$ -almost universal. We also use a variant of this where the random point is restricted to a subset $S \subset \mathbb{F}_q$, which is $(n - 1)/|S|$ -universal.

Composition and Truncation of Hashes. We rely on the following composition results. Similar properties have been shown in e.g. [Sti92, Roy22].

Proposition 8. *Let $\mathcal{H}, \mathcal{H}'$ be ε and ε' -almost universal families. Then, the concatenation $\left\{ \begin{bmatrix} \mathbf{H} \\ \mathbf{H}' \end{bmatrix} : \mathbf{H} \in \mathcal{H}, \mathbf{H}' \in \mathcal{H}' \right\}$ is $\varepsilon\varepsilon'$ -almost universal.*

Proof. Follows from independence of $\mathbf{H}$ and $\mathbf{H}'$. $\square$

Proposition 9. *Let $\mathcal{H} \subseteq \mathbb{F}_q^{r' \times n}$ be ε -almost universal and $\mathcal{H}' \subseteq \mathbb{F}_q^{r \times r'}$ be ε' -almost uniform. Then, the product $\{\mathbf{H}'\mathbf{H} : \mathbf{H} \in \mathcal{H}, \mathbf{H}' \in \mathcal{H}'\}$ is $(\varepsilon + \varepsilon')$ -almost uniform.*

Proof. Let $\mathbf{x} \in \mathbb{F}_q^n$ be non-zero. Then, $\mathbf{x}' = \mathbf{H}\mathbf{x}$ is non-zero with probability at least $1 - \varepsilon$. If $\mathbf{x}'$ is non-zero, then for any $\mathbf{v} \in \mathbb{F}_q^r$, $\mathbf{H}'\mathbf{x}' \neq \mathbf{v}$ with probability at least $1 - \varepsilon'$, by the uniformity of $\mathcal{H}'$. It follows that $\Pr[\mathbf{H}'\mathbf{H}\mathbf{x} \neq \mathbf{v}] \geq (1 - \varepsilon)(1 - \varepsilon') \geq 1 - \varepsilon - \varepsilon'$, and so the product is an $(\varepsilon + \varepsilon')$ -almost uniform family. $\square$

Proposition 10. *Let $\delta \in \mathbb{N}$ and $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$ be an ε -almost uniform family. Then, the truncated family $\{\mathbf{H}_{[0..r-\delta]} : \mathbf{H} \in \mathcal{H}\}$ is εq^δ -uniform.*

Proof. For each $\mathbf{H} \in \mathcal{H}$, write $\mathbf{H} = \begin{bmatrix} \mathbf{H}_0 \\ \mathbf{H}_1 \end{bmatrix}$, where $\mathbf{H}_0 \in \mathbb{F}_q^{(r-\delta) \times n}$ and $\mathbf{H}_1 \in \mathbb{F}_q^{\delta \times n}$. Let $\mathbf{y} = \begin{bmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \end{bmatrix} \in \mathbb{F}_q^r$ and $\mathbf{x} \in \mathbb{F}_q^n \setminus \{0\}$. If $\mathbf{H} \leftarrow \mathcal{H}$ we have $\Pr[\mathbf{H}\mathbf{x} = \mathbf{y}] \leq \varepsilon$. Applying conditional probability, we get

$$\begin{aligned} \Pr[\mathbf{H}_0\mathbf{x} = \mathbf{y}_0 \wedge \mathbf{H}_1\mathbf{x} = \mathbf{y}_1] &\leq \varepsilon \\ \Pr[\mathbf{H}_0\mathbf{x} = \mathbf{y}_0] \cdot \Pr[\mathbf{H}_1\mathbf{x} = \mathbf{y}_1 \mid \mathbf{H}_0\mathbf{x} = \mathbf{y}_0] &\leq \varepsilon \\ \Pr[\mathbf{H}_0\mathbf{x} = \mathbf{y}_0] &\leq \varepsilon \cdot (\Pr[\mathbf{H}_1\mathbf{x} = \mathbf{y}_1 \mid \mathbf{H}_0\mathbf{x} = \mathbf{y}_0])^{-1} \\ &\leq \varepsilon \cdot q^\delta \end{aligned}$$

where the final inequality comes from fixing a $\mathbf{y}_1 \in \mathbb{F}_q^\delta$ that maximizes $p = \Pr[\mathbf{H}_1\mathbf{x} = \mathbf{y}_1 \mid \mathbf{H}_0\mathbf{x} = \mathbf{y}_0]$, which implies p is at least $q^{-\delta}$. $\square$

Constructions. To define the hash functions that we use in this work, which can be found in Figure 3, we use the following two algorithms:

- **ToField**($\mathbf{x}; k$): maps $\mathbf{x} \in \{0, 1\}^{nk}$ into a (vector of) field element(s) $\mathbf{x} \in \mathbb{F}_{2^k}^n$.
- **ToBits**($\mathbf{x}; k, n$): maps $\mathbf{x} \in \mathbb{F}_{2^k}^n$, for $n \geq 1$, into a bit string $\mathbf{x} \in \{0, 1\}^{nk}$.

Note that the algorithms in Figure 3 specify how a random bit string $\mathbf{sd}$ of the appropriate length is used to sample a function from the family and evaluate it on a given input $\mathbf{x}$.

VOLE Universal Hash. The first check, to verify consistency of the VOLE correlations, requires a family that is linear over $\mathbb{F}_2$. It's evaluated on inputs in $\mathbb{F}_2^{\hat{\ell}}$, where $\hat{\ell} = \ell + 3\lambda + B$, ℓ is the witness length and $B = 16$ is a parameter chosen for security.¹²

To build the hash, we map the seed $\mathbf{sd}$ into $(r_0, r_1, r_2, r_3, s, t) \in \mathbb{F}_{2^\lambda}^5 \times \mathbb{F}_{2^{64}}$. The input $\mathbf{x} \in \mathbb{F}_2^{\hat{\ell}}$ is first split into $(\mathbf{x}_0, \mathbf{x}_1)$, where $\mathbf{x}_0 \in \{0, 1\}^{\ell+\lambda}$, and then $\mathbf{x}_0$ is parsed twice, first as a vector $\hat{\mathbf{y}}$ of $\mathbb{F}_{2^\lambda}$ elements, and then as a vector $\bar{\mathbf{y}}$ of $\mathbb{F}_{2^{64}}$ elements. Then, compute

¹² Our signature scheme actually calls **VOLEHash** on input an $\hat{\ell} \times \lambda$ matrix, which is translated into computing the hash on each column separately, with the same seed.

VOLEHash (sd, $(\mathbf{x}_0, \mathbf{x}_1) \in \{0, 1\}^{\ell+2\lambda} \times \{0, 1\}^{\lambda+B}$)	ZKHash (sd, $(\mathbf{x}_0, x_1) \in \mathbb{F}_{2^\lambda}^\ell \times \mathbb{F}_{2^\lambda}$)
1 : $\parallel$ λ -bit $\mathbf{r}_i, \mathbf{s}$; 64-bit $\mathbf{t}$ 2 : $\parallel$ λ -bit $\mathbf{r}_i, \mathbf{s}$; 64-bit $\mathbf{t}$ 3 : Parse $\mathbf{sd} = (\mathbf{r}_0 \parallel \mathbf{r}_1 \parallel \mathbf{r}_2 \parallel \mathbf{r}_3 \parallel \mathbf{s} \parallel \mathbf{t}) \in \{0, 1\}^{5\lambda+64}$ 4 : $r_i := \text{ToField}(\mathbf{r}_i, \lambda)$, for $i \in [0..3]$ 5 : $s := \text{ToField}(\mathbf{s}, \lambda)$ and $t := \text{ToField}(\mathbf{t}, 64)$ 6 : $\ell' := \lambda \cdot \lceil (\ell + \lambda) / \lambda \rceil$ 7 : $\mathbf{x}_0 := \mathbf{x}_0 \parallel 0^{\ell' - (\ell + \lambda)}$ $\parallel$ pad to multiple of λ 8 : $\hat{\mathbf{y}} := \text{ToField}(\mathbf{x}_0, \lambda)$, $\bar{\mathbf{y}} := \text{ToField}(\mathbf{x}_0, 64)$ 9 : $h_0 := \sum_{i=0}^{\ell'/\lambda-1} s^{\ell'/\lambda-1-i} \cdot \hat{\mathbf{y}}[i]$ (in $\mathbb{F}_{2^\lambda}$) 10 : $h_1 := \sum_{i=0}^{\ell'/64-1} t^{\ell'/64-1-i} \cdot \bar{\mathbf{y}}[i]$ (in $\mathbb{F}_{2^{64}}$) 11 : $h'_1 := \text{ToField}(\text{ToBits}(h_1) \parallel 0^{\lambda-64}, \lambda)$ 12 : $(h_2, h_3) := (r_0 h_0 + r_1 h'_1, r_2 h_0 + r_3 h'_1)$ 13 : $\mathbf{h} := (\text{ToBits}(h_2) \parallel \text{ToBits}(h_3)[0..B]) \oplus \mathbf{x}_1$ 14 : return $\mathbf{h}$	1 : $\parallel$ λ -bit $\mathbf{r}_i, \mathbf{s}$; 64-bit $\mathbf{t}$ 2 : Parse $\mathbf{sd} = (\mathbf{r}_0 \parallel \mathbf{r}_1 \parallel \mathbf{s} \parallel \mathbf{t}) \in \{0, 1\}^{3\lambda+64}$ 3 : $r_i := \text{ToField}(\mathbf{r}_i, \lambda)$, for $i \in \{0, 1\}$ 4 : $s := \text{ToField}(\mathbf{s}, \lambda)$ 5 : $t := \text{ToField}(\mathbf{t} \parallel 0^{\lambda-64}, \lambda)$ 6 : $h_0 := \sum_{i=0}^{\ell-1} s^{\ell-1-i} \cdot \mathbf{x}_0[i]$ 7 : $h_1 := \sum_{i=0}^{\ell-1} t^{\ell-1-i} \cdot \mathbf{x}_0[i]$ 8 : $\mathbf{h} := \text{ToBits}(r_0 h_0 + r_1 \cdot h_1 + x_1)$ 9 : return $\mathbf{h}$

Fig. 3. Universal hashing algorithms

$$\begin{aligned}
h_0 &= \hat{y}_0 s^{\hat{\ell}/\lambda-1} + \hat{y}_1 s^{\hat{\ell}/\lambda-2} + \dots + \hat{y}_{\hat{\ell}/\lambda-2} s + \hat{y}_{\hat{\ell}/\lambda-1} \quad \text{in } \mathbb{F}_{2^\lambda}, \\
h_1 &= \bar{y}_0 t^{\hat{\ell}/64-1} + \bar{y}_1 t^{\hat{\ell}/64-2} + \dots + \bar{y}_{\hat{\ell}/64-2} t + \bar{y}_{\hat{\ell}/64-1} \quad \text{in } \mathbb{F}_{2^{64}}
\end{aligned}$$

Viewing h_1 as an element of $\mathbb{F}_{2^\lambda}$ (by zero-padding), the hash is then defined by computing, in $\mathbb{F}_{2^\lambda}$

$$\begin{bmatrix} h_2 \\ h_3 \end{bmatrix} = \begin{bmatrix} r_0 & r_1 \\ r_2 & r_3 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \end{bmatrix}$$

Finally, take the first $\lambda + B$ bits of the concatenation of the field elements h_2 and h_3 , and XOR this with $\mathbf{x}_1$ to obtain the output.

We argue security of the construction below. Note that instead of aiming for $\varepsilon = 2^{-\lambda}$, we aim for $2^{-\lambda-B}$, where $B = 16$. The extra few bits of security compensate for the $\binom{\tau}{2}$ security loss in the proof of the SoftSpokenVOLE protocol from [BBD⁺23].

Lemma 11. *VOLEHash is an ε_v -almost universal hash family in $\mathbb{F}_2^{(\lambda+B) \times \hat{\ell}}$, for $\varepsilon_v = 2^{-\lambda-B}(1 + 2^{B-50})$, if $\hat{\ell} \leq 2^{13}$. Furthermore, VOLEHash is $\mathbb{F}_2^{\ell+\lambda}$ -hiding.*

Proof. We show ε_v -uniformity of the hash that outputs the first $\lambda + B$ bits of (h_2, h_3) , i.e., without adding $\mathbf{x}_1$. By Proposition 7, this implies ε_v -universality of the final hash, as well as the hiding property.

The first part of the hash — computing h_0, h_1 — is a concatenation of two polynomial hashes, over either $\mathbb{F}_{2^\lambda}$ or $\mathbb{F}_{2^{64}}$. These are ε -universal with $\varepsilon = d/|\mathbb{F}|$, where d is the polynomial degree and $\mathbb{F}$ is the field, and we have $d \leq \hat{\ell}/64$. Since binary field multiplication is bilinear over $\mathbb{F}_2$, both of these hashes are also $\mathbb{F}_2$ -linear. Applying Proposition 8, their concatenation is then ε_0 -universal with $\varepsilon_0 \leq \hat{\ell}^2/2^{\lambda+76}$. Note that for $\hat{\ell} \leq 2^{13}$, we have $\varepsilon_0 \leq 2^{-\lambda-50}$.

The second part of the hash starts with a 2×2 matrix hash, which is $2^{-2\lambda}$ -uniform. After truncation, the resulting hash is ε_1 -uniform for $\varepsilon_1 = 2^{-\lambda-B}$, by Proposition 10. The final combined hash is the product of these two parts, so applying Proposition 9 and summing the probabilities, we get that for all $\hat{\ell} \leq 2^{13}$, the hash is ε_v -uniform for $\varepsilon_v = \varepsilon_0 + \varepsilon_1 \leq 2^{-\lambda-B}(1 + 2^{B-50})$. $\square$

ZK Universal Hash. Our second hash, used for verifying AES constraints in ZK, must be linear over $\mathbb{F}_{2^\lambda}$. It works on inputs of $\ell' = \ell + 1$ field elements. We map the seed $\mathbf{sd}$ into $r_0, r_1, s, t \in \mathbb{F}_{2^\lambda}$, where

r_0, r_1 and s are uniform, while t (viewed as an $\mathbb{F}_2$ polynomial) is zero in all its degree ≥ 64 coefficients (and uniform otherwise). Let

$$\mathbf{r}^\top := \begin{bmatrix} r_0 & r_1 \end{bmatrix} \begin{bmatrix} s^{\ell-1} & s^{\ell-2} & \dots & s & 1 \\ t^{\ell-1} & t^{\ell-2} & \dots & t & 1 \end{bmatrix}$$

The hash of an input $\mathbf{x} = (x_0, x_1) \in \mathbb{F}_{2^\lambda}^\ell \times \mathbb{F}_{2^\lambda}$ is simply $h = \mathbf{r}^\top \mathbf{x}_0 + x_1$.

Lemma 12. *ZKHash is an ε_{zk} -almost universal hash family in $\mathbb{F}_{2^\lambda}^{1 \times \ell'}$, for $\varepsilon_{\text{zk}} = 2^{-\lambda}(1+2^{-38})$, if $\ell' \leq 2^{13}$. Furthermore, ZKHash is $\mathbb{F}_{2^\lambda}^\ell$ -hiding.*

Proof. As with **VOLEHash**, by Proposition 7 it suffices to show ε_{zk} -uniformity of the hash defined by $\mathbf{r}^\top \mathbf{x}_0$.

This hash is a product of two separate hashes. The outer hash — multiplication by $(r_0 \ r_1)$ — is a standard matrix hash, which is $2^{-\lambda}$ -almost uniform. The inner hash is a concatenation of two polynomial evaluations, the first of which defines an almost-universal hash for $\varepsilon_0 = (\ell' - 1)/2^\lambda$, and the second for $\varepsilon_1 = (\ell' - 1)/2^{64}$; together, this gives an $\varepsilon_0 \varepsilon_1$ -almost universal hash (via Proposition 8), where $\varepsilon_0 \varepsilon_1 \leq (\ell')^2 / 2^{\lambda+64} \leq 2^{-\lambda-38}$, for all $\ell' \leq 2^{13}$. Combining the inner and outer parts, from Proposition 9 we get that **ZKHash** is ε_{zk} -almost uniform for $\varepsilon_{\text{zk}} = 2^{-\lambda}(1 + 2^{-38})$. $\square$

2.5 Interactive Proof Systems

We need some definitions relating to (non-)interactive proof systems.

Definition 13 (Interactive proof system (Extracted from [CY24, Section 13.1])). *An interactive proof (IP) for relation $\mathcal{R}$ in (the random oracle model) is a tuple of algorithms $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ where the prover and the verifier interact over $k \in \mathbb{N}$ rounds as follows (with access to one or more random oracles).*

- The IP prover $\mathcal{P}_{\text{IP}}$ receives as input the instance $\mathbb{x}$ and a witness $\mathbb{w}$ where $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ and the IP verifier $\mathcal{V}_{\text{IP}}$ receives as input only the instance $\mathbb{x}$.
- In each round $i \in [k]$, the prover $\mathcal{P}_{\text{IP}}$, (with access to one or more random oracles,) sends a message α_i and then the verifier $\mathcal{V}_{\text{IP}}$ responds with a (possibly empty) message ρ_i .
- After the interaction, $\mathcal{V}_{\text{IP}}$ outputs a bit, where 1 denotes acceptance and 0 denotes rejection, computed based on the instance $\mathbb{x}$, (random oracle queries,) the prover messages $(\alpha_i)_{i \in [k]}$ and the verifier messages $(\rho_i)_{i \in [k]}$.

We define basic soundness and round-by-round soundness.

Definition 14 (Soundness). *A (possibly non-interactive) IP $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ for a language $\mathcal{L}$ is called sound if for all adversarial provers $\mathcal{A}$, the soundness advantage $\text{AdvSnd}_{\mathcal{A}}^{\text{IP}}$ is negligible in the security parameter, where*

$$\text{AdvSnd}_{\mathcal{A}}^{\text{IP}} = \Pr[\mathbb{x} \notin \mathcal{L} = 1 \wedge b = 1 \mid \mathbb{x} \leftarrow \mathcal{A}(1^\lambda); b \leftarrow \langle \mathcal{A}(\mathbb{x}), \mathcal{V}_{\text{IP}}(\mathbb{x}) \rangle = 1].$$

Definition 15 (Round-by-Round (RBR) Soundness (based on [CCH⁺18])). *For a IP Π for a language $\mathcal{L}$, let RBRState be a state function mapping an instance and a transcript prefix to $\{0, 1\}$, with the following properties. For $\mathbb{x} \notin \mathcal{L}$,*

1. $\text{RBRState}(\mathbb{x}, \emptyset) = 0$, where $\emptyset$ denotes the empty transcript prefix.
2. If for some instance $\mathbb{x}$ and a transcript prefix trc that is empty ($i=0$) or ends in the i -th challenge with $\text{RBRState}(\mathbb{x}, \text{trc}) = 0$, then for any prover message pmsg_{i+1} it holds that

$$\Pr_{\text{chall}_{i+1}} [\text{RBRState}(\mathbb{x}, \text{trc} | \text{pmsg}_{i+1} | \text{chall}_{i+1}) = 1] \leq \epsilon,$$

where chall_{i+1} is sampled uniformly from the appropriate challenge space, and the RBR soundness error ϵ depends on the security parameter only.

3. For any complete transcript trc , if $\text{RBRState}(\mathbb{x}, \text{trc}) = 0$ then the verifier rejects.

We say Π has RBR soundness if for all no-instances $x \notin \mathcal{L}$ and all transcripts trc , ϵ is upper bounded by a negligible function of the security parameter.

There are many flavors of the Fiat-Shamir transformation in the literature. We use a version where the hash for deriving the i -th challenge includes the $(i-1)$ -st challenge (if any) and the i -th prover message pmsg_i .

Definition 16 (Fiat-Shamir Transformation). Let $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ be an interactive proof system with ℓ challenges chall_i for $i \in [\ell]$ and $\ell+1$ prover messages pmsg_i , $i \in [\ell+1]$. Given challenge derivation functions f_i , $i \in [\ell]$, we define the Fiat-Shamir transformation $\text{FS}[\text{IP}] = (\mathcal{P}_{\text{FS}[\text{IP}]}, \mathcal{V}_{\text{FS}[\text{IP}]})$ of IP . Setting $\text{chall}_0 = \iota$, the empty string, $\mathcal{P}_{\text{FS}[\text{IP}]}$ executes $\mathcal{P}_{\text{IP}}$, computing chall_i as $f_i(\text{chall}_{i-1} \parallel \text{pmsg}_i)$ and outputs $(\text{pmsg}_i)_{i=1}^{\ell+1}$. Given a proof $(\text{pmsg}_i)_{i=1}^{\ell+1}$, as input $\mathcal{V}_{\text{FS}[\text{IP}]}$ recomputes $(\text{chall}_i)_{i=1}^{\ell}$ and runs $\mathcal{V}_{\text{IP}}$ on the expanded proof $(\text{pmsg}_1, \text{chall}_1, \dots, \text{pmsg}_{\ell+1})$.

2.6 Hardness Assumptions for Symmetric Primitives

Most of our hardness assumptions are related to AES, in particular indistinguishability of AES from a pseudorandom permutation. We do not make the hardness assumptions on the hashes $H_0, \dots, H_4$ explicit; these are idealized as random oracles in the security proofs. We use the standard notions of computational indistinguishability, pseudorandom functions and pseudorandom permutations.

Definition 17 (Indistinguishability). Let $\mathcal{D}_0, \mathcal{D}_1$ be two distributions. Let $\mathcal{A}$ be an adversary in the following experiment $\text{ExpDist}_{\mathcal{A}}$: 1. $x_0 \leftarrow \mathcal{D}_0$, $x_1 \leftarrow \mathcal{D}_1$, $b \leftarrow \{0, 1\}$, 2. $b' = \mathcal{A}(1^\lambda, x_b)$ 3. Return $b' = b$. The distinguishing advantage of $\mathcal{A}$ is

$$\text{AdvDist}_{\mathcal{A}}^{\mathcal{D}_0, \mathcal{D}_1} = 2 \cdot \left| \Pr[\text{ExpDist}_{\mathcal{A}}] - \frac{1}{2} \right| = |\Pr[b' = 1 \mid b = 0] - \Pr[b' = 1 \mid b = 1]|$$

If $\mathcal{A}$ is given access to Q independent samples of $\mathcal{D}_0$ or $\mathcal{D}_1$ instead, we write $\text{AdvDist}_{\mathcal{A}}^{\mathcal{D}_0, \mathcal{D}_1}[Q]$ for the advantage in the respective game. The definition for indistinguishability of oracles (instead of distributions) is analogous.

An efficiently computable function family $\text{PRF}: \mathcal{K}_\lambda \times \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda$ where $\mathcal{K}_\lambda$ is the key space is a *pseudorandom function (PRF)* if for a uniform secret key $\mathbf{k} \leftarrow \mathcal{K}_\lambda$, oracle-access to $\text{PRF}(\mathbf{k}, \cdot)$ is indistinguishable from access to truly random function $\text{H}: \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda$ for any efficient adversary. A *pseudorandom permutation (PRP)* $\text{PRP}: \mathcal{K}_\lambda \times \mathcal{X}_\lambda \rightarrow \mathcal{X}_\lambda$ is a PRF such that $\text{PRP}(\mathbf{k}, \cdot)$ is a permutation for any $\mathbf{k} \in \mathcal{K}_\lambda$.

IV-based Tweakable PRGs. For analysis reasons, we use a notion of IV-based tweakable PRG tailored to our construction **PRG** in Fig. 1, with block sizes of 128. Then we define and analyze the security of **PRG** in two aspects: a pseudorandomness notion which is used to prove that our commitment scheme is hiding, and an injectivity-type notion which is needed only in **FAEST-EM** and used to prove that the commitment scheme is statistically binding.

Definition 18 (IV-based tweakable PRG). An IV-based tweakable PRG is an efficient map $\text{PRG}_m: \{0, 1\}^\lambda \times \{0, 1\}^{128} \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{\alpha \cdot 128}$, where $\alpha \in \mathbb{N}$ and output length $m = \alpha \cdot 128$.

Pseudorandomness Notions. Since our notion of IV-based tweakable PRG is so closely tied to the **PRG** construction from Fig. 1, we define two types of security, namely PRP-type and PRF-type security in Definition 19, which capture a multi-instance setting where an adversary receives multiple PRG outputs for the same key, under different tweaks (the tweak is viewed as the input to the PRP or PRF). The PRP-type security notion is exactly what is achieved by modeling AES as a pseudorandom permutation. While the PRP/PRF switching lemma could be used to show PRF-type security, this would lead to a poor concrete bound, due to birthday attacks and the fixed block size of 128 in AES. However, by using a divergence-based switching lemma (Corollary 22), we can still show PRF-type security in our setting, where signature queries will be limited to $Q_{\text{sig}} \leq 2^{64}$.

Definition 19 (PRP-type and PRF-type security of PRG). Let $\alpha \in \mathbb{N}$ and let $\text{PRG}_m: \{0, 1\}^\lambda \times \{0, 1\}^{128} \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{\alpha \cdot 128}$ be a deterministic polynomial-time algorithm with output length $m = \alpha \cdot 128$ (not necessarily **PRG** from Section 2). Let $\mathcal{A}$ be a T -query adversary in the following PRP-type game:

- For $k \leftarrow \{0, 1\}^\lambda$, $\text{iv} \leftarrow \{0, 1\}^{128}$.
- Sample $b \leftarrow \{0, 1\}$, set $\text{RP} = \{\}$ as an empty associative array and let $B = \emptyset$.
- Run $b' \leftarrow \mathcal{A}^{\text{PRG}^b}(\text{iv})$ where $\mathcal{A}$ is limited to T oracle queries and
 - $\text{PRG}^0(k, \text{iv}, \text{twk})$: Output $\text{PRG}_m(k, \text{iv}, \text{twk})$
 - $\text{PRG}^1(k, \text{iv}, \text{twk})$: If $\text{RP}[\text{twk}] = \perp$: For $i = [1, \alpha]$ sample $y_i \leftarrow \{0, 1\}^{128} \setminus B$, and set $B := B \cup \{y_i\}$. Set $\text{RP}[\text{twk}] = (y_1 \parallel \dots \parallel y_\alpha)$ and output $\text{RP}[\text{twk}]$.
- Output 1 (win) if $b = b'$, else 0 (lose).

Let p be the probability that $\mathcal{A}$ wins the game. The advantage $\text{AdvPRP}_A^{\text{PRG}^m}[T]$ of T -query adversary against the PRP-type security of the (IV-based tweakable) PRG PRG is defined as

$$\text{AdvPRP}_A^{\text{PRG}^m}[T] = |2p - 1|.$$

Define the PRF-type game which is identical to the above, except that we implement a (lazy sampled) random function RF instead of the random permutation RF . We write $\text{AdvPRG}_A^{\text{PRG}^m}[T] = |2p - 1|$ to denote the respective advantage for PRF-type security.

Definition 19 imposes a stronger requirement on PRG than strictly necessary for our proofs. Namely, we let the adversary choose the tweaks freely, while in our construction, the tweaks are dependent on the GGM layer and fixed.

Remark 20 (PRP-type security). It is easy to see that our PRG constructed in Section 2 satisfies PRP-type security of Definition 19 by a straightforward reduction to PRP security of AES with αT queries. (Observe that (by definition of [AddToUpperWord](#) and AES-CTR) we never query AES on the same input, hence we get αT distinct queries and thus random blocks.)

To deal with PRF-type security, we argue with ∞ -divergence in the following.

Divergence Analysis for PRF-type Security. By construction, AES is a pseudorandom *permutation* (PRP), yet we want to treat it as a pseudorandom *function* (PRF). Naively, switching from a random permutation to a random function is distinguishable with advantage about Q^2/D for domain size D . Since our domain would be $\{0, 1\}^{128}$, i.e., 128bit block (as specified by AES), we could not prove higher than 128bit security, even when limited to 2^{64} signatures overall. Fortunately, this is just a proof artefact and easily resolved by using suitable proof technique, such as the “H-coefficient” method or “Rényi divergence”. Namely, we have the following facts.

Lemma 21 (∞ -divergence). Let X and Y be random variables with values in a set Ω and countable support. Let $\rho := \sup_{\omega \in \Omega} \frac{\Pr[X=\omega]}{\Pr[Y=\omega]}$, where $0/0 := 0$ and $t/0 := \infty$ for any $t > 0$. For any function $f: \Omega \rightarrow \mathbb{R}_{\geq 0}$ we have

$$\mathbb{E}[f(X)] \leq \rho \cdot \mathbb{E}[f(Y)]$$

In particular, for any event $E \subseteq \Omega$, we have

$$\Pr[X \in E] \leq \rho \cdot \Pr[Y \in E]$$

Note that $D_\infty(X \| Y) := \log(\rho)$ is called ∞ -divergence or Rényi divergence of order ∞ .

Proof. We have by definition of ρ and since $f \geq 0$ that

$$\begin{aligned} \mathbb{E}[f(X)] &= \sum_{\omega \in \Omega} \Pr[X = \omega] \cdot f(\omega) \\ &= \sum_{\omega \in \Omega} \frac{\Pr[X = \omega]}{\Pr[Y = \omega]} \cdot \Pr[Y = \omega] \cdot f(\omega) \\ &\leq \sum_{\omega \in \Omega} \rho \cdot \Pr[Y = \omega] \cdot f(\omega) \\ &= \rho \cdot \mathbb{E}[f(Y)] \end{aligned}$$

which proves the first claim. The second follows since $\Pr[X \in E] = \mathbb{E}[f(X)]$ for the predicate $f(\omega) = \mathbb{1}_{[\omega \in E]}$. $\square$

Corollary 22. Let $\mathcal{X}$ be a set of size N and let $n \in \mathbb{N}$ with $n \leq N$. Let $\mathcal{A}: \mathcal{X}^n \rightarrow \{0, 1\}$ be any function. Let R be the uniform distribution on $\mathcal{X}^n$ and let P be the distribution of n uniform samples without replacement from $\mathcal{X}$. Then we have

$$\Pr[\mathcal{A}(P) = 1] \leq e^{\frac{n(n-1)}{N}} \Pr[\mathcal{A}(R) = 1]$$

Proof. By Lemma 21, it suffices to show $\frac{\Pr[P=x]}{\Pr[R=x]} \leq e^{\frac{n(n-1)}{N}}$. Since $\Pr[P = x] \leq \frac{1}{N \cdot (N-1) \cdot \dots \cdot (N-n+1)}$ and $\Pr[R = x] = \frac{1}{N^n}$, we have to show

$$\frac{N^n}{(N \cdot (N-1) \cdot (N-n+1))} = \left(\prod_{i=1}^{n-1} \left(1 - \frac{i}{N}\right) \right)^{-1} \stackrel{(?)}{\leq} e^{\frac{n(n-1)}{N}}.$$

When $n-1 \leq 0.79N$, this follows directly from the bound $e^{-x} \leq 1 - x/2$ for $\frac{x}{2} \in [0, 0.79]$. To handle general $n \leq N$, we need a more technical argument. First, we take logarithms to express the claim to prove as

$$\sum_{i=1}^{n-1} \ln \left(1 - \frac{i}{N}\right) \stackrel{(?)}{\geq} -\frac{n(n-1)}{N}. \quad (1)$$

Observe that $f(x) = \ln(1-x)$ is concave and monotone decreasing over $[0, 1)$ (since $f'(x) = -\frac{1}{1-x} < 1$, and $f''(x) = -(1-x)^{-2} < 0$). Dividing the sum by N , we can interpret it as a right-sided Riemann sum which approximates an integral over f with $n-1$ rectangles of width $1/N$. Let $g(x) := -f(x)$ so that $g(x)$ is a monotone increasing function over $[0, 1)$. Let $S_L := \frac{1}{N} \cdot \sum_{i=0}^{n-2} g(\frac{i}{N})$ and $S_R := \frac{1}{N} \cdot \sum_{i=1}^{n-1} g(\frac{i}{N})$ denote the left-sided and right-sided Riemann sums bounding the integral: $S_L \leq \int_0^{\frac{n-1}{N}} g(x) dx \leq S_R$. Note that the difference between the Riemann sums is exactly $S_R - S_L = \frac{1}{N} \cdot g(\frac{n-1}{N}) - \frac{1}{N} \cdot g(\frac{0}{N}) = \frac{1}{N} \cdot g(\frac{n-1}{N})$ by construction as $g(0) = 0$. Thus we arrive at the inequality

$$\frac{1}{N} \cdot \sum_{i=1}^{n-1} g\left(\frac{i}{N}\right) = S_R = S_L + \frac{1}{N} \cdot g\left(\frac{n-1}{N}\right) \leq \int_0^{\frac{n-1}{N}} g(x) dx + \frac{1}{N} \cdot g\left(\frac{n-1}{N}\right).$$

Plugging in $g(x) = -\ln(1-x)$ and simplifying the resulting lower bound, we find

$$\begin{aligned} \frac{1}{N} \cdot \sum_{i=1}^{n-1} \ln \left(1 - \frac{i}{N}\right) &\geq \int_0^{\frac{n-1}{N}} \ln(1-x) dx + \frac{1}{N} \cdot \ln \left(1 - \frac{n-1}{N}\right) \\ &= \left[-(1-x) \cdot \ln(1-x) - x \right]_0^{\frac{n-1}{N}} + \frac{1}{N} \cdot \ln \left(1 - \frac{n-1}{N}\right) \\ &= -\left(1 - \frac{n-1}{N}\right) \cdot \ln \left(1 - \frac{n-1}{N}\right) - \frac{n-1}{N} + \frac{1}{N} \cdot \ln \left(1 - \frac{n-1}{N}\right) \\ &= -\left(1 - \frac{n}{N}\right) \cdot \ln \left(1 - \frac{n-1}{N}\right) - \frac{n-1}{N}. \end{aligned}$$

Multiplying Eq. (1) by $\frac{1}{N}$ and using the above bound, it suffices to show that

$$-\left(1 - \frac{n}{N}\right) \cdot \ln \left(1 - \frac{n-1}{N}\right) - \frac{n-1}{N} \stackrel{(?)}{\geq} -\frac{n(n-1)}{N^2}. \quad (2)$$

First, suppose that $n = N$. Then the claim holds because $(1 - \frac{n}{N}) = 0$ and $\frac{n-1}{N} = \frac{n(n-1)}{N^2}$. Now assume $n < N$. Then we can rewrite Eq. (2) as

$$-\ln \left(1 - \frac{n-1}{N}\right) \stackrel{(?)}{\geq} \frac{1}{(1 - \frac{n}{N})} \left(-\frac{n(n-1)}{N^2} + \frac{n-1}{N} \right) = \frac{n-1}{N}.$$

From the inequality $-\ln(1-x) \geq x$ for $x < 1$, the above claim follows for all $n < N$. This proves Eq. (1) and completes the proof. $\square$

In particular, we can use Corollary 22 to switch outputs of PRGs in the GGM tree to truly random outputs. Then, they can serve as random seeds for the next GGM application.

Remark 23 (Hybrids with ∞ -divergence). For a sequence of hybrids $H_0, H_1, \dots, H_n$, where $\Pr[H_{i-1} = 1] \leq \rho_i \cdot \Pr[H_i = 1] + \varepsilon_i$, with $\rho_i \geq 1$ and $\varepsilon_i \geq 0$, we find

$$\begin{aligned} \Pr[H_0 = 1] &\leq \left(\prod_{j=1}^n \rho_j \right) \cdot \Pr[H_n = 1] + \sum_{i=1}^n \left(\prod_{j=1}^{i-1} \rho_j \right) \cdot \varepsilon_i \\ &\leq \left(\prod_{j=1}^n \rho_j \right) \cdot \left(\Pr[H_n = 1] + \sum_{i=1}^n \varepsilon_i \right) \end{aligned}$$

This is the analogue to the usual hybrid argument.

We are now ready to prove PRF-type security of the PRG.

Lemma 24. *Let $\beta \in \mathbb{N}$ and let $\text{PRG}_m: \{0, 1\}^\lambda \times \{0, 1\}^{128} \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{\beta \cdot 128}$ be a deterministic polynomial-time algorithm with $m = \beta \cdot 128$. Let $\mathcal{A}$ be a T -query adversary for PRF-type security of PRG (Definition 19). Let $\mathbf{G}_{\text{PRP}}$ denote the output of $\mathcal{A}$ in the real PRP-type game and $\mathbf{G}_{\text{RP}}$ (resp. $\mathbf{G}_{\text{RF}}$) be the output in the ideal PRP-type game (resp. PRF-type). Then it holds that*

$$\Pr[\mathbf{G}_{\text{PRP}} = 1] \leq e^{\beta^2 T^2 / 2^{128}} \cdot \Pr[\mathbf{G}_{\text{RF}} = 1] + \text{AdvPRP}_{\mathcal{A}}^{\text{PRG}_m}[T].$$

Proof. Let $\mathbf{G}_{\text{RP}}$ denote the ideal PRP-type game. Then

$$\Pr[\mathbf{G}_{\text{PRP}} = 1] \leq \Pr[\mathbf{G}_{\text{RP}} = 1] + \text{AdvPRP}_{\mathcal{A}}^{\text{PRG}_m}[T]$$

follows by definition. By an application of Corollary 22, switching the sampling of blocks without replacement to sampling of blocks with replacement (i.e., truly at random) and therefore to $\mathbf{G}_1$, leads to

$$\Pr[\mathbf{G}_{\text{RP}} = 1] \leq e^{\beta^2 T^2 / 2^{128}} \cdot \Pr[\mathbf{G}_{\text{RF}} = 1]. \quad \square$$

2.7 The Compressed Oracle Technique

Here, we recall a minimal amount of notation relating to the compressed oracle technique [Zha19, CFHL21] which we use in our QROM proof (Section 6). We identify $D \in (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}}$ with a partial function from $\mathcal{X}$ to $\mathcal{Y}$, where $D(x) = \perp$ indicates that $D(x)$ is undefined. We write $D[x \mapsto u]$ for the partial function that maps x to u and x' to $D(x')$ for all $x' \neq x$. We say that a predicate P on $(\mathcal{Y} \cup \{\perp\})^{\mathcal{X}}$ has q_P -local witnesses if for all $D \in (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}}$ with $P(D) = 1$ there exists a tuple $z = (x_1, \dots, x_i) \in \mathcal{X}^i$ for $i \leq q_P$ such that for all $D' \in (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}}$ that agree with D on z , i.e., $D'(x_j) = D(x_j)$ for $j = 1, \dots, i$, it holds that $P(D') = 1$. We denote the validity of a witness z for a predicate P and a (partial) function D as $\langle z, P, D \rangle$. Furthermore we define $|D| = |\{x \in \mathcal{X} \mid D(x) \neq \perp\}|$.

We recall a lemma for QROM query bounds from [AHJ⁺23].

Lemma 25 (Special case of Lemma 1 in [AHJ⁺23]). *Let $H: \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle and let $P: (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}} \rightarrow \{0, 1\}$ be a predicate with q_P -local witnesses. Let further $\mathcal{A}^H$ be a QROM algorithm making at most q quantum queries to H and outputs a candidate witness z for P . Then*

$$\sqrt{\Pr_{z \leftarrow \mathcal{A}^H}[\langle z, P, H \rangle]} \leq \sum_{k=1}^{q+q_P} \sqrt{10 \max_{\substack{x, D: \\ |D| \leq k, \neg P(D)}} \Pr_{u \leftarrow \mathcal{Y}}[P(D[x \mapsto u])]}. \quad (3)$$

Here the maximum is taken over $x \in \mathcal{X}$ and $D \in (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}}$.

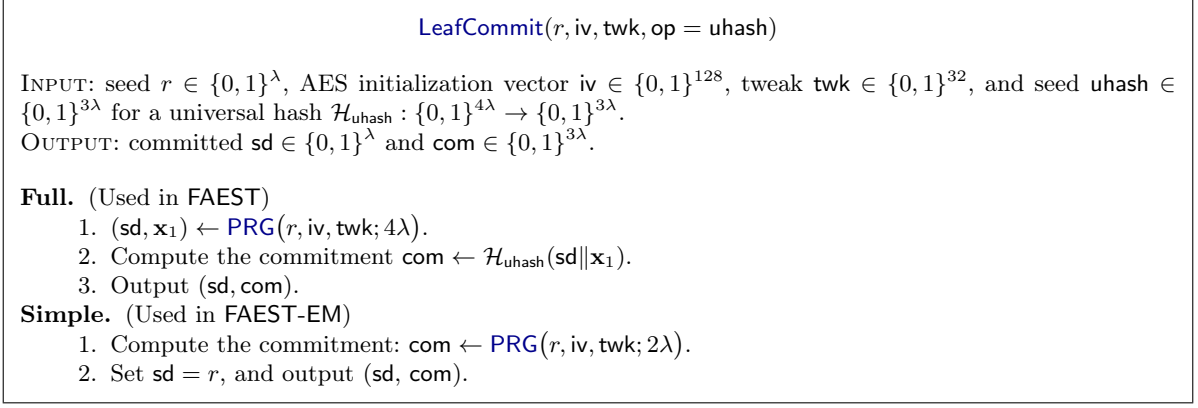


Figure 4. Pseudorandom commitment algorithms based on a PRG

3 VOLE Commitments

In this section, we describe our optimized all-but-one vector commitment BAVC and how it is converted into VOLE commitments. Our construction is based on the classic GGM tree technique for puncturable PRF, which gives an all-but-one commitment. We extend it to a batch of τ such commitments, so that τ distinct leaves can remain hidden while the remaining are opened, and use a rejection sampling optimization to reduce the size of openings, from [BBM⁺24]. Our main novelty is use of PRG-based leaf commitments instead of hash-based ones, which allows for a faster implementation, though it complicates the security analysis.

PRG-Based Leaf Commitments. We describe two leaf-commitment functions in Figure 4: the **Full** variant, which we apply to FAEST, is more conservative but slightly larger, while **Simple** is smaller but makes a stronger assumption on AES, so we use it for FAEST-EM. **Full** uses a universal hash function family $\mathcal{H}$, which must be ε -universal for $\varepsilon = 2^{-3\lambda}$ and $\mathbb{F}_2^\lambda$ -hiding.. In practice, we can instantiate this in a standard way using finite field multiplication over $\mathbb{F}_{2^{3\lambda}}$, via the function $\mathcal{H}_s(x_0 || x_1) = x_0 \cdot s + x_1$, where $x_0 \in \{0, 1\}^\lambda, x_1 \in \{0, 1\}^{3\lambda}$ are each mapped into elements of $\mathbb{F}_{2^{3\lambda}}$.

We now show that the **Full** variant is statistically binding, if $uhash$ is sampled after iv and twk are fixed.¹³

Lemma 26. *Let $iv \in \{0, 1\}^{128}, twk \in \{0, 1\}^{32}$. If $uhash \leftarrow \{0, 1\}^{3\lambda}$ and $\mathcal{H}$ is $2^{-3\lambda}$ -universal, then **LeafCommit** is perfectly binding, except with probability $2^{-\lambda}$.*

Proof. Suppose there exist two distinct openings r, r' that are valid for the same commitment com . This implies

$$\mathcal{H}_{uhash}(\text{PRG}(r, iv, twk)) = \mathcal{H}_{uhash}(\text{PRG}(r', iv, twk)).$$

Since iv, twk are fixed, and $r, r' \in \{0, 1\}^{2\lambda}$, there are only $2^{2\lambda}$ possible pairs $(\text{PRG}(r), \text{PRG}(r'))$. Since $\mathcal{H}$ is $2^{-3\lambda}$ -universal, any given pair of inputs collides with probability at most $2^{-3\lambda}$. We conclude that there exist no such collisions, and so **LeafCommit** is perfectly binding, except with probability $2^{-\lambda}$. $\square$

Analysis of Simple Leaf Commitments For the **Simple** variant, it would be straightforward to argue that the construction is *computationally* binding, assuming the 2λ -bit PRG is collision-resistant. Unfortunately, this is not enough to use in FAEST, since it does not provide us with a way to *extract* a committed value; indeed, there may be multiple preimages r for any given commitment. Another workaround could be to rely on an idealized model such as an ideal cipher for the PRG, allowing extraction by observing PRG queries. However, we prefer to avoid this, especially given that the zero-knowledge proof in FAEST already makes non-black-box use of AES, which strongly contradicts modeling AES as an ideal primitive.

¹³ Later, we guarantee this by deriving $uhash$ from a random oracle that depends on iv .

Instead, we prove security by relying on a non-standard (and inefficiently falsifiable) strengthening of collision resistance, which asserts that it is hard to even *find an image* for which a *collision exists*, i.e. for which two or more preimages exist. Note that the adversary does not need to output the *preimages*, their existence suffices.

Definition 27 (Almost-Injective PRG). Let $\text{PRG}(k, \text{iv}, \text{twk})$ be a (IV-based tweakable) PRG, where $k \in \mathcal{K}_\lambda = \{0, 1\}^\lambda$, $\text{iv} \in \mathcal{I}_\lambda$, and $\text{twk} \in \mathcal{T}_\lambda$. Consider the following game, parameterized by Q_{out} :

1. The adversary outputs $\text{iv} \in \mathcal{I}_\lambda$ together with Q_{out} tuples $(\text{twk}_j, y_j)_{j=1}^{Q_{\text{out}}}$.
2. If for any j , there exists $\mathbf{k} \neq \mathbf{k}'$ in $\mathcal{K}_\lambda$ such that $\text{PRG}(\mathbf{k}, \text{iv}, \text{twk}_j) = y_j = \text{PRG}(\mathbf{k}', \text{iv}, \text{twk}_j)$, output 1 (success). Else output 0.

We define the advantage of $\mathcal{A}$ in the game as

$$\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{out}}] = \Pr[\text{The game outputs 1}].$$

We say that PRG is almost injective if for any polynomial-time adversary and polynomial Q_{out} , the advantage $\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{out}}]$ is negligible.

Observe that the adversary must produce an element y_j for which a collision exists to win in Definition 27. Since it seems hard to tell if an image y_j has more than one preimage, without *knowing* more than one preimage (i.e. a collision), the assumption seems plausible. Indeed, if an adversary knows just one preimage r for its output y_j , then since there are only $2^\lambda - 1$ other possible preimages, yet y comes from $\{0, 1\}^{2\lambda}$, heuristically, a second preimage only exists with probability $\approx 2^{-\lambda}$. Therefore, it seems that the only strategy for an attacker to win is to first find a collision.

As a sanity check, we consider simple attacks on Definition 27 for the PRG constructed in Section 2 under heuristic simplifications.

Remark 28 (Heuristic). We model AES as an ideal cipher¹⁴ and assume a bound Q_{AES} on ideal cipher queries. The map $\text{PRG}: \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ which is defined as $\text{PRG}(\mathbf{k}, \text{iv}) = (\text{AES}.\text{Enc}(\mathbf{k}, \text{iv}), \dots, \text{AES}.\text{Enc}(\mathbf{k}, \text{AddToLowerWord}(\text{iv}, h)))$ is a random oracle in $\mathbf{k}$ for fixed iv (where $h = 2\lambda/128$, see Section 2). We heuristically ignore the correlations caused by tweak twk and counter i acting on the same IV space via AddToUpperWord and AddToLowerWord respectively. Moreover, we heuristically treat ideal cipher queries and PRG queries as the same. This heuristic is partly justified by the PRG construction; ideal cipher encryption queries can be recovered by running PRG. We note however, that it is not clear if *decryption queries* could somehow be exploited for improved attacks; we ignore them.

Next, we focus on naive strategies an adversary could follow: Let $m = 2^\lambda$, $n = 2^{2\lambda}$. Sticking to single IV chosen in advance, and mounting a mix of birthday attack and guessing via Q_{out} images $\mathbf{y}_j$. The birthday attack part is straightforward. By focusing on a single iv , the probability of collision with Q_{AES} PRG queries is less than $\frac{Q_{\text{AES}}^2}{n}$. Note that, importantly, in this case the adversary *knows* for certain that it found a $\mathbf{y}$ with multiple preimages. On the other hand, as long as no collisions occurred, all images are the “same”, namely, random distinct elements. Thus, it may as well output a random subset of Q_{out} of these $\mathbf{y}$ s. The probability of success in that case is the same for each subset, and is given by $1 - (1 - \frac{Q_{\text{out}}}{n})^{m - Q_{\text{AES}}} \leq \frac{Q_{\text{out}}(m - Q_{\text{AES}})}{n}$. In summary, the success probability of this strategy is upper-bounded by $\frac{Q_{\text{AES}}^2}{n} + \frac{Q_{\text{out}}(m - Q_{\text{AES}})}{n} = \frac{Q_{\text{AES}}(Q_{\text{AES}} - Q_{\text{out}})}{n} + \frac{Q_{\text{out}} \cdot m}{n}$.

Using the Assumption. Under this assumption, we argue that LeafCommit has a (weak) form of statistical binding, where the following holds: there exists an (inefficient) extractor, Ext , such that for any efficient adversary $\mathcal{A}$ who outputs a tuple $(\text{iv}, \text{twk}, \text{com})$, one of two properties hold: (1) Ext successfully outputs a unique preimage r for com (if one exists), or (2) $\mathcal{A}$ wins the almost-injectivity game, with non-negligible advantage.

This strategy ensures that, when proving binding of the commitment, we can define an inefficient extractor to prove that either the adversary’s committed values are unique and well-defined, or, the adversary violated almost-injectivity. For details, we refer to the proof of Lemma 32 in Section 3.3.

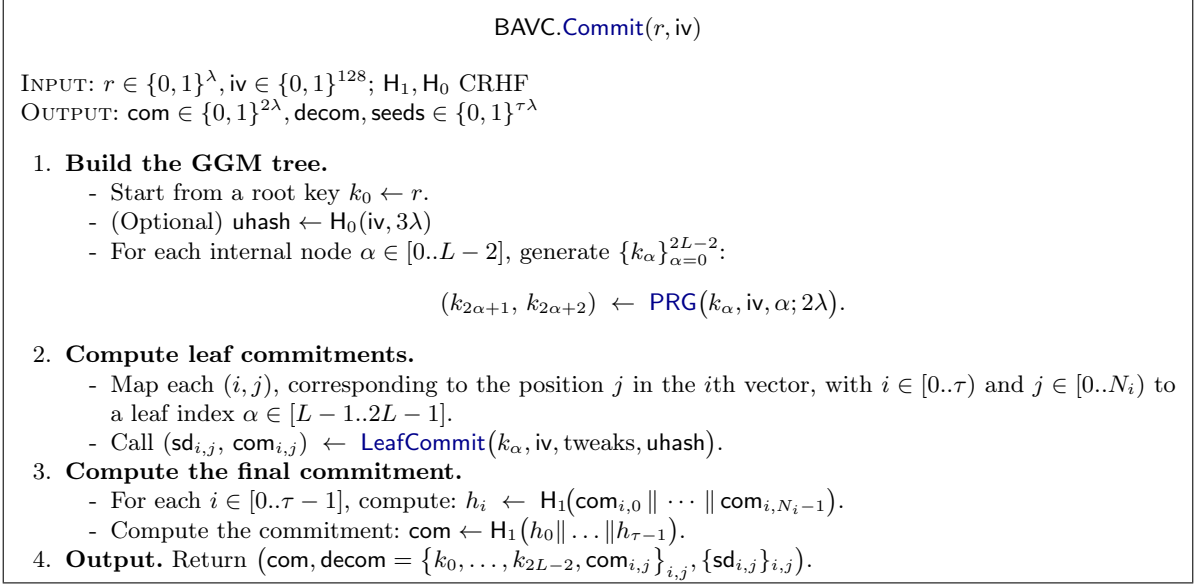


Figure 5. BAVC Commit

3.1 Batch Vector Commitments

We use our leaf commitments to build a batch vector commitment, using a GGM tree of PRGs, as shown in Figure 5. The construction works as follows: the prover first expands r by applying the length-doubling PRG **PRG** recursively in a tree-like fashion: it first expands r into two keys k_1, k_2 and then applies it again to each of the obtained keys, until a tree with L leaves containing λ -bit strings is obtained. We choose L such that the leaves can be partitioned into τ subsets of sizes $N_0, \dots, N_{\tau-1}$, giving a batch of τ all-but-one vector commitments. Each leaf index $\alpha \in \{L-1, \dots, 2L-1\}$ is then mapped using a permutation $\pi : [L-1, 2L-1] \rightarrow \{(i, j)\}_{j \in [0, N_i)}$ into a position j of the i -th vector commitment.

Then, **LeafCommit** is applied to each leaf k_α , possibly using the extra parameter $uhash$ derived from iv using a random oracle call. Each call to **LeafCommit** generates an output, called the seed $sd_{i,j}$, as well as a commitment $com_{i,j}$. The prover then hashes all the $com_{i,j}$ belonging to one vector into h_i using a random oracle, and then compresses all h_i using another random oracle call into the BAVC commitment h . The decommitment consists of all values k_α as well as all the generated $com_{i,j}, sd_{i,j}$.

To open such a commitment for $I = (\Delta_0, \dots, \Delta_{\tau-1})$, i.e. reveal all seeds committed to in the i th vector except Δ_{i-1} , the prover finds all k_α that are the roots of sub-trees of the tree generated in **Commit** that do not have indices in I . If the number of such k_j exceeds a threshold, T_{open} , then the prover aborts the opening. These k_α form the opening information $decomp_I$, together with all $com_{i,j}$ generated from those leaves of the tree that are in I . Note that the prover only reveals $com_{i,j}$ for the elements in I , not $sd_{i,j}$.

Using $decomp_I$ as well as I , the reconstruction algorithm (shown in Figure 7) can reconstruct all $sd_{i,j}$ outside of I by applying **PRG** to all k_j in $decomp_I$ in the same way **Commit** used them to generate the original tree. Then, it can apply **LeafCommit** to all the reconstructed tree leaves, yielding all $sd_{i,j}, com_{i,j}$ outside of I . Based on these recomputed $com_{i,j}$ as well as those contained in $decomp_I$ that belong to leaves in I , **Reconstruct** can apply the random oracle again to derive h as **Commit** does.

BAVC Correctness. Since the construction is essentially the same as [BBM⁺24] but with different leaf commitments, the correctness of the opening/reconstruction procedures follows from [BBM⁺24]. For completeness, we prove the construction satisfies *correctness with aborts* (Definition 4), where reconstruction succeeds as long as the opening algorithm does not abort.

¹⁴ This is merely a heuristic argument used to justify the assumption; we highlight that our formal security analysis does not model AES as an ideal cipher.

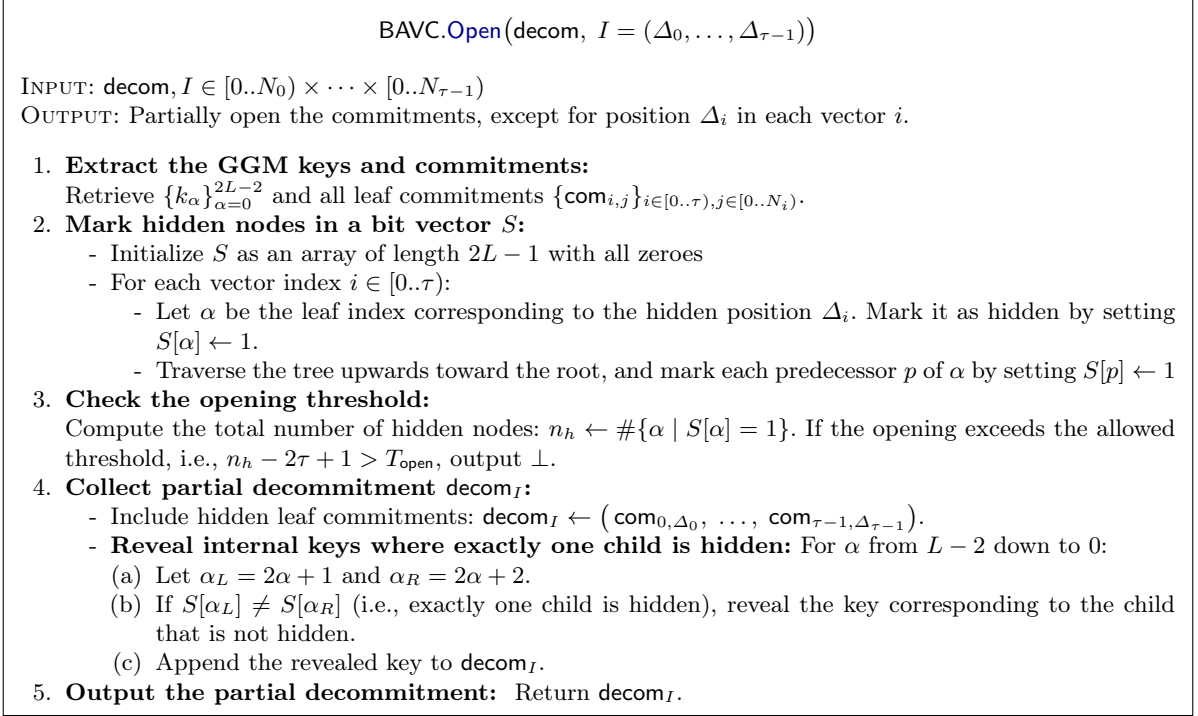


Figure 6. BAVC Open

Proposition 29 (BAVC Correctness with Aborts). *Let BAVC be the batch all-but-one vector commitment scheme described in Figures 5, 6 and 7. Then BAVC has correctness with aborts as defined in Definition 4.*

Proof. We prove that any $\text{decom}_I \neq \perp$ output by **BAVC.Open** contains exactly the information needed for **BAVC.Reconstruct** to re-compute and recover every unhidden leaf’s seed ($\text{sd}_{i,j}, j \neq \Delta_i$), as well as re-compute the final hash com .

First, we recall from **BAVC.Commit** that each internal node seed k_α of the GGM tree is expanded via a deterministic pseudorandom generator:

$$(k_{2\alpha+1}, k_{2\alpha+2}) \leftarrow \text{PRG}(k_\alpha, \text{iv}, \alpha; 2\lambda).$$

Hence, if the same node α and parent seed k_α appear in two different executions, they will produce the same child seeds. This implies that all node seeds in the unhidden part of the tree are fixed as soon as we fix $k_0 = r$ and the expansions up to those unhidden nodes.

When **BAVC.Open**(*decom*, I) runs, it marks the leaf corresponding to **BAVC.PosInTree**(i, Δ_i) for each vector i as “hidden” and includes exactly the subset of child seeds needed to re-derive all other leaves. This selective opening does not give out the seeds on a path if that path leads to a fully hidden leaf. The algorithm also checks that the total size of revealed seeds stays below some threshold T_{open} ; if not, it aborts with $\perp$. Concretely, in the final partial decommitment decom_I , generated in **BAVC.Open**, we see:

1. The τ hidden leaf commitments $\{\text{com}_{i,\Delta_i}\}_{i=0}^{\tau-1}$.
2. The seeds of children at internal nodes having “exactly one hidden child.” These seeds allow re-expansion of the unmarked subtrees, but do not reveal the seeds on the fully hidden paths.

Hence $\text{decom}_I \neq \perp$ implies **BAVC.Open** has given the verifier *sufficient* information to re-derive all unhidden leaves.

When **Reconstruct** is called on input decom_I , it does as follows:

- (a) It parses the hidden leaf commitments $\{\text{com}_{i,\Delta_i}\}$;
- (b) It re-expands the unhidden portion of the GGM tree from the revealed node seeds;

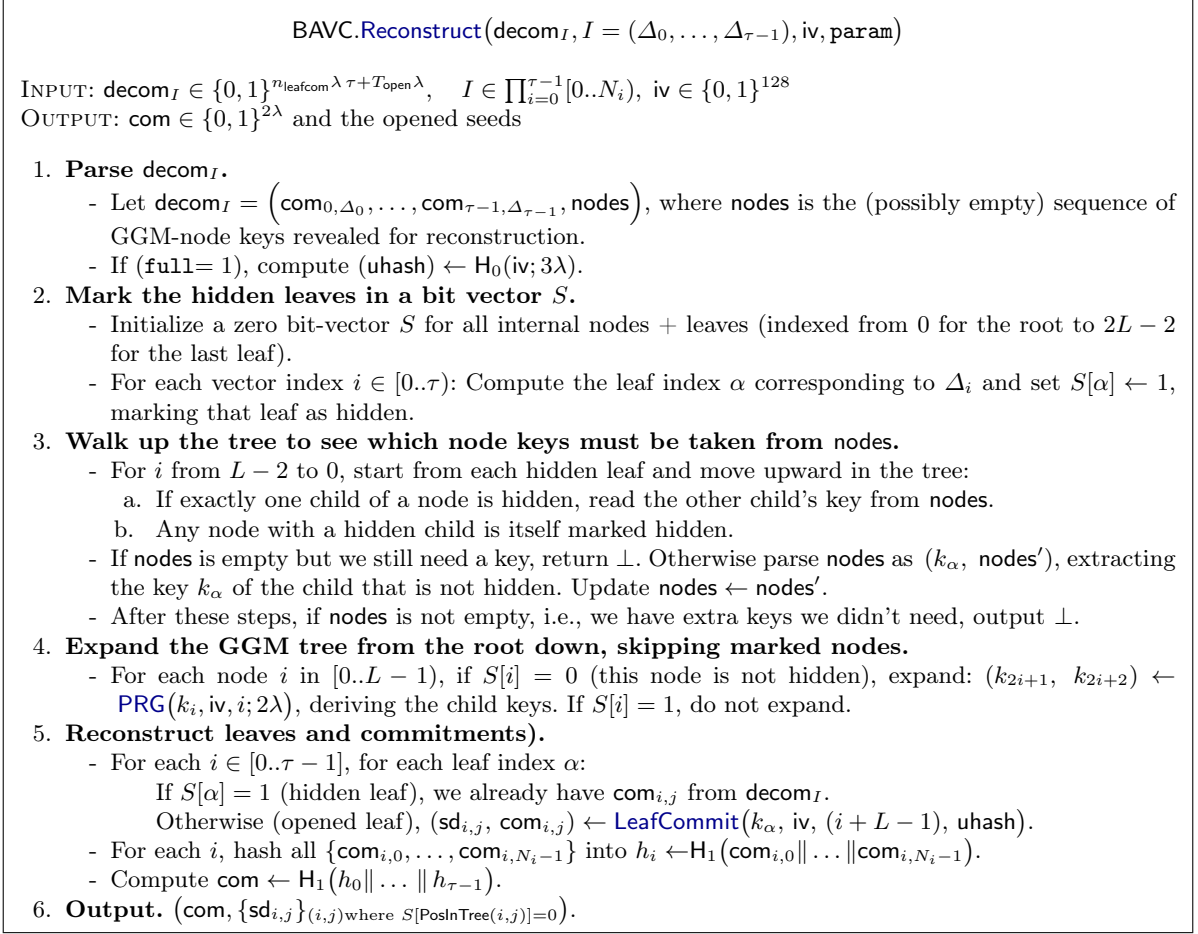


Figure 7. BAVC Reconstruct

- (c) It runs the same leaf-commit function to get $(\text{sd}_{i,j}, \text{com}_{i,j})$ for $j \neq \Delta_i$. All expansions are deterministic, so by comparing with BAVC.Commit's expansions, one sees that $\text{sd}_{i,j}$ and $\text{com}_{i,j}$ must coincide for the unhidden leaves.

Finally, BAVC.Reconstruct re-computes the vector-level partial commitments

$$h_i = H_1(\text{com}_{i,0} \parallel \dots \parallel \text{com}_{i,N_i-1})$$

and concatenates them, hashing again:

$$\text{com}' = H_1(h_0 \parallel \dots \parallel h_{\tau-1}).$$

For unhidden leaves, $\text{com}_{i,j}$ are precisely the ones re-derived before. For the hidden leaves, the partial opening decom_I has already included the correct commitments com_{i,Δ_i} . Therefore the entire set $\{\text{com}_{i,j}\}_{i,j \in [N_i], j \neq \Delta_i} \cup \{\text{com}_{i,\Delta_i}\}$ matches the set from BAVC.Commit. Hence $\text{com}' = \text{com}$, except with negligible probability in the event of a hash collision.

Other than the commitment com , the BAVC.Reconstruct algorithm also returns $\{\text{sd}_{i,j}\}_{j \neq \Delta_i}$ as re-derived from the GGM expansions and leaf commits. By the same argument as before, each such $\text{sd}_{i,j}$ is the same as in BAVC.Commit. Meanwhile, the seeds for the hidden indices (i, Δ_i) never appear in decom_I and remain unknown to the verifier.

We have that BAVC.Reconstruct(decom_I, I) re-creates exactly the same com from BAVC.Commit since every unhidden seed and leaf commit re-expands to the same value. No new information about the hidden leaves $\{\text{sd}_{i,\Delta_i}\}$ is revealed, beyond the com_{i,Δ_i} . Hence, provided Open does not abort, the batch all-but-one vector commitment satisfies correctness. $\square$

3.2 Converting BAVC to VOLEs

In previous works such as [BBM⁺24], it was shown that the vector commitment schemes also fulfill notions of binding and hiding. However, the BAVC scheme is never used directly in our work, but only after conversion into VOLE correlations which are themselves commitments. We will therefore now introduce how this conversion works, and then describe binding and hiding from a VOLE angle.

From Seeds to VOLEs. After committing to N random seeds using the all-but-one vector commitment BAVC, each seed is expanded to a longer vector of $\mathbb{F}_q$ elements using PRG. The set of N vectors is then converted into a VOLE correlation over $\mathbb{F}_N$, for $N = 2^d$, which we represent as d VOLE correlations over $\mathbb{F}_2$. This conversion requires the signer to compute, in $\mathbb{F}_N$

$$\mathbf{u} = \sum_{i=0}^{N-1} \text{PRG}(\text{sd}_i), \quad \mathbf{v} = \sum_{i=0}^{N-1} i \cdot \text{PRG}(\text{sd}_i), \quad (4)$$

where PRG expands each seed to a vector in $\{0, 1\}^{\hat{\ell}}$ and i is viewed as an element of $\mathbb{F}_N$. Here $\hat{\ell} = \ell + 3\lambda + B$, where ℓ is the witness length for the zero-knowledge proof, and $3\lambda + B$ extra commitments bits are needed to mask various consistency checks. The verifier, when given some index Δ and sd_i for all $i \neq \Delta$, will compute

$$\mathbf{q} = \sum_{i=0}^{N-1} (\Delta - i) \cdot \text{PRG}(\text{sd}_i).$$

This is a VOLE correlation, since $\mathbf{q} = \mathbf{v} + \mathbf{u} \cdot \Delta$ in $\mathbb{F}_{2^d}$.

To convert vector commitments into VOLE correlations, instead of computing Eq. (4) directly, the signer and verifier use a divide-and-conquer method [Roy22] to iteratively compute the vectors $\mathbf{v}_j$ and $\mathbf{q}_j$, which represent the j -th bits extracted from $\mathbf{v}, \mathbf{q}$. This algorithm is shown in Fig. 8. This algorithm is run by the prover using all N seeds sd_i , while the verifier will run the same algorithm where one of the seeds is unknown. It therefore sets one of the seeds to $\perp$ and ignores the $\mathbf{u}$ part of the output. For correctness, the verifier must additionally permute its seeds according to the permutation $i \mapsto i \oplus \Delta$. When run by prover and verifier on consistent inputs, the ConvertToVOLE algorithm (Figure 8) gives the following correlation guarantee:

Proposition 30. *Let $(\text{sd}_0, \dots, \text{sd}_{N-1})$ and $(\text{sd}'_0, \dots, \text{sd}'_{N-1})$ be seeds such that $\text{sd}'_i = \text{sd}_{i \oplus \Delta}$ for all $i > 0$, for some $\Delta \in [0..N)$, and $\text{sd}'_0 = \perp$. Then, if $(\delta_0, \dots, \delta_{d-1})$ is the bit decomposition of Δ ,*

$$(\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{d-1}) := \text{ConvertToVOLE}(\text{sd}_0, \dots, \text{sd}_{N-1}; \hat{\ell}),$$

and

$$(\mathbf{u}', \mathbf{q}_0, \dots, \mathbf{q}_{d-1}) := \text{ConvertToVOLE}(\text{sd}'_0, \dots, \text{sd}'_{N-1}; \hat{\ell}),$$

it holds that

$$\mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u}, \quad \text{for } j \in [0..d).$$

This proof follows from close inspection of the algorithm, and was also shown in e.g. [FAE23].

VOLE Commitments and Reconstructions. We use two additional algorithms VOLECommit and VOLEReconstruct, in which the signer and verifier apply the above transformation to each of the τ committed vectors from the BAVC. Additionally, VOLECommit outputs a set of “correction values” $\mathbf{c}_i$ that convert each of the τ VOLE instances to use the *same* vector $\mathbf{u}$. By viewing elements of $\mathbb{F}_{2^d}$ as bit strings in $\{0, 1\}^d$, the τ VOLE correlations can be combined into one big λ -bit correlation, namely

$$\mathbf{Q} = \mathbf{V} + \mathbf{u} \cdot \Delta$$

where $\mathbf{u} \in \mathbb{F}_2^{\hat{\ell}}$, $\mathbf{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$ are known to the signer, and $\Delta \in \{0, 1\}^\lambda$, $\mathbf{Q} \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$ to the verifier (where in the product $\mathbf{u} \cdot \Delta$, each bit of $\mathbf{u}$ is multiplied with all of Δ). Note that $\mathbf{Q}, \mathbf{V}$ can also be seen as length- $\hat{\ell}$ vectors of $\mathbb{F}_{2^\lambda}$ elements; this perspective is used later in the zero-knowledge proof. In the following, we detail the complete VOLECommit and VOLEReconstruct algorithms.

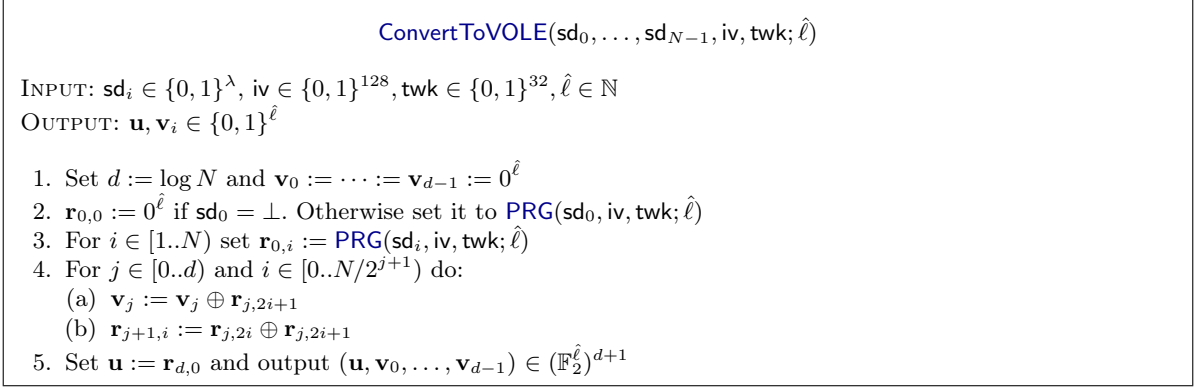


Figure 8. Seed expansion and conversion to VOLE

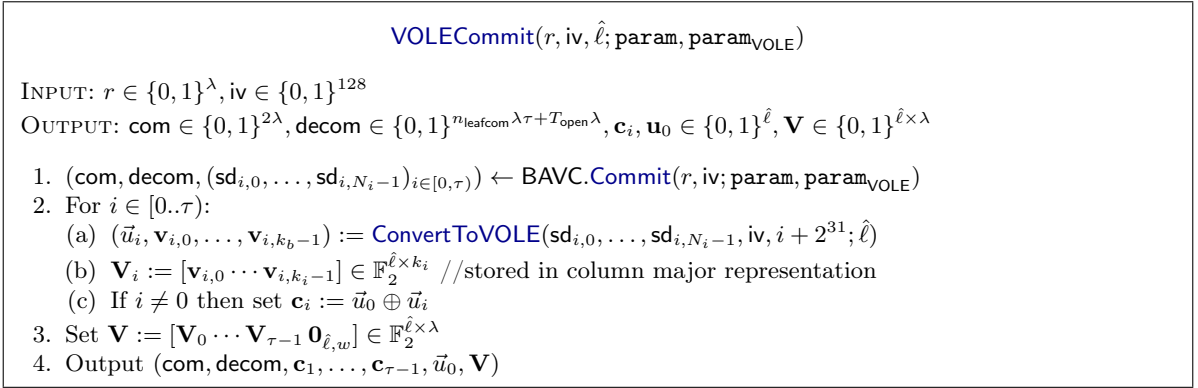


Figure 9. VOLE Commitments

VOLE Commitment. The algorithm **VOLECommit** (Figure 9) takes as input a λ -bit seed r , a 128-bit initialization vector iv , and a parameter $\hat{\ell}$. It outputs a commitment com , a decommitment decom , a set of “correction” vectors $\{\vec{c}_1, \dots, \vec{c}_{\tau-1}\}$, the secret vector $\vec{u}$ for the VOLE correlation, and a matrix $\mathbf{V}$ whose columns are the vectors $\vec{v}$ of the τ VOLEs.

First, the algorithm invokes **BAVC.Commit**($r, \text{iv}; \text{param}, \text{param}_{\text{VOLE}}$) to perform a single batch all-but-one vector commitment over all τ VOLE vectors. This returns a global commitment com , a decommitment structure decom and the per-leaf seeds $\text{sd}_{i,j}$ for each vector $i \in [0..\tau-1]$.

Next, the function loops over each vector i . For each such vector, it calls **ConvertToVOLE**($\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1}, \text{iv}, i + 2^{31}; \hat{\ell}$). This function merges the seeds $\text{sd}_{i,j}$ into one $\hat{\ell}$ -bit secret $\vec{u}_i$ and a set of column vectors $\mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,k_i-1}$. The column vectors are placed in a matrix $\mathbf{V}_i \in \mathbb{F}_2^{\hat{\ell} \times k_i}$. After doing this for all τ vectors, the procedure concatenates these sub-matrices (plus any zero-padding) into a global matrix $\mathbf{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$. This generates $\tau \cdot \hat{\ell}$ VOLE correlations with different secrets $\vec{u}_i$. To correct the sender secret to the same secret vector, the algorithm designates $\vec{u}_0$ from the first vector as the secret $\vec{u}$, and, for each $i \geq 1$, it computes $\vec{c}_i = \vec{u}_0 \oplus \vec{u}_i$. These “correction” vectors allow the other VOLE instances to share the same underlying secret $\vec{u}$. Finally, the function returns com , the decommitment decom , all correction vectors $\{\vec{c}_i\}$, the secret $\vec{u}$, and the large matrix $\mathbf{V}$.

VOLE Reconstruction. Once the signer has committed to its execution of **VOLECommit**, the verifier can use the commitments, together with the challenge that generates the Δ_i keys, to reconstruct the matching VOLE keys using **VOLEReconstruct** (Figure 10).

The algorithm **VOLEReconstruct** takes, as input, a λ -bit challenge chall_3 , a partial decommitment decom_I from the all-but-one vector commitment, a set of correction vectors $\{\mathbf{c}_i\}$, a 128-bit iv , and a VOLE length $\hat{\ell}$. It returns both the commitment com (verifying consistency with the committed values) and a matrix $\mathbf{Q}$ that encodes the receiver’s VOLE messages.

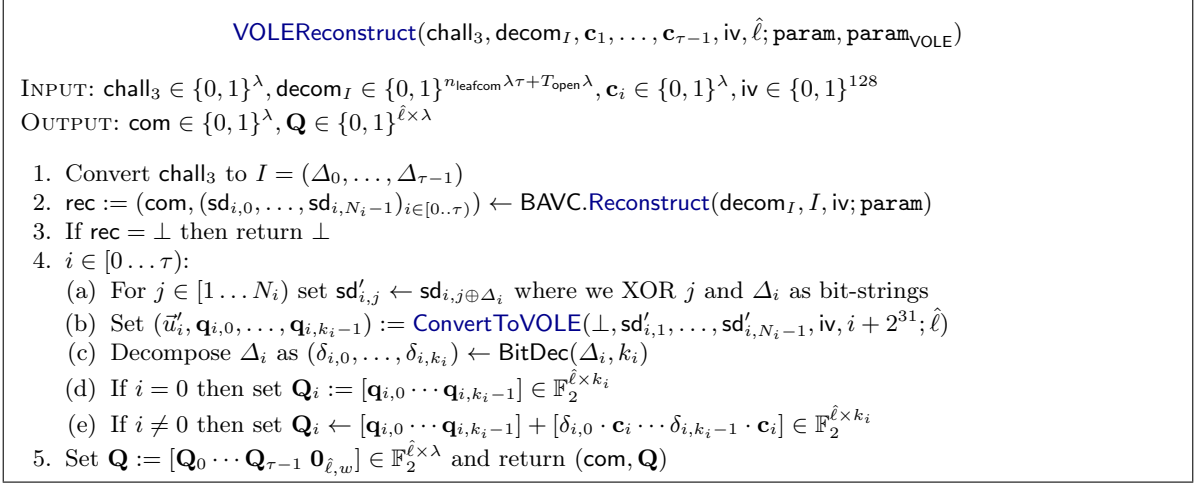


Figure 10. VOLE reconstruction

First, the algorithm decodes chall₃ into Δ₀, ..., Δ_{τ-1}. Each Δ_i indicates which leaf in the i-th vector remains hidden. Next, it partially reconstructs the batch vector commitment by invoking

$$(\text{com}, (\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1})_{i \in [0..\tau)}) \leftarrow \text{BAVC.Reconstruct}(\text{decom}_I, I, \text{iv}; \text{param}).$$

This call returns commitment com (matching the one from VOLECommit) and, for every i, all leaf seeds except for the one at index Δ_i. The algorithm then iterates over each vector i. Within each vector, it applies ConvertToVOLE on input seeds sd'_{i,j} = sd_{i,j ⊕ Δ_i}, returning (ū'_i, q_{i,0}, ..., q_{i,k_i-1}) (Proposition 30). Additionally, it interprets Δ_i as bits (δ_{i,0}, ..., δ_{i,k_i}). If i = 0, the resulting columns q_{i,j} are stored directly in Q₀. If i ≥ 1, each column q_{i,j} is “shifted” by δ_{i,j} · c_i to make it consistent with the main VOLE secret ū₀ from VOLECommit. Finally, it concatenates Q_i for all i (possibly with a zero-padding block 0 _{$\hat{\ell}, w$}) into a single matrix Q ∈ $\mathbb{F}_2^{\hat{\ell} \times \lambda}$. At the end, the function outputs the recovered commitment com and the matrix Q, which together allow the verifier to validate that the partial opening and VOLE messages correspond to the original batch commitment generated by FAEST.VOLECommit.

3.3 VOLECommit is Binding

The main security property satisfied by our scheme is a notion of *extractable binding*, which requires that there exists an extractor that is allowed to observe random oracle queries, and will successfully extract the adversary’s committed messages with high probability. Unlike [FAE23], we relax the definition to allow for an unbounded extractor, since we are only aiming to prove soundness of the resulting zero-knowledge argument and not knowledge soundness. We also note that this analysis is applied to the VOLECommit algorithm rather than just BAVC; this is only for technical reasons that help with analysing some other properties, and essentially the same argument shows binding of BAVC. Below, we prove a formal version of the following informal lemma:

Lemma 31 (Extractable Binding of VOLE Commitment, Simplified). *Define the following extractable-binding game for VOLECommit and VOLEReconstruct, a stateful adversary A, and a straight-line extractor Ext:*

1. During the commit phase, A may output arbitrarily many tuples (com, c₁, ..., c_{τ-1}, iv^{pre}). These are stored in a list.
2. During the decommitment phase, A first chooses any previously generated tuple C as well as a challenge I = (Δ₀, ..., Δ_{τ-1}) and a decommitment decom_I. The challenger will derive iv from iv^{pre} of C.
3. Then, Ext will extract a VOLE correlation (u_i^{*}, V_i^{*})_{i ∈ [0,τ)} for C (or fails). In addition, the VOLE correlation matrix Q (consisting of submatrices Q_i) of the receiver as well as the commitment com will be reconstructed from decom_I, Δ, the c_i and iv using VOLEReconstruct.

4. $\mathcal{A}$ loses if **VOLEReconstruct** failed, if $\overline{\text{com}} \neq \text{com}$ or if the extractor succeeded in extracting $\mathbf{u}_i^*, \mathbf{V}_i^*$ that form valid VOLE correlations with each $\mathbf{Q}_i$ and Δ_i . Otherwise $\mathcal{A}$ wins.

Suppose $\mathcal{A}$ makes at most Q_0, Q_1 queries to the oracles H_0, H_1 , respectively, used in **BAVC.Commit**. Moreover, suppose that $\mathcal{A}$ checks if **VOLEReconstruct** returns $\perp$ for the purported opening decom_I , and outputs $\perp$ if that happens. There is a deterministic (unbounded) straight-line extractor **Ext** (defined in the security proof for **FAEST**) such that the advantage of $\mathcal{A}$ in the above game is at most

$$(Q_1 + 1)^2 \cdot 2^{-2\lambda} + \tau Q_0 \cdot 2^{-\lambda}.$$

There is a deterministic (unbounded) straight-line extractor **Ext** (defined in the security proof for **FAEST-EM**) which is allowed to program H_4 (that derives iv from iv^{pre}), such that the advantage of $\mathcal{A}$ in the above game is at most

$$(Q_1 + 1)^2 \cdot 2^{-2\lambda} + \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[Q_4, L].$$

where the running time of $\mathcal{B}$ is roughly that of $\mathcal{A}$. The IV-based tweakable PRG $\text{PRG}_{2\lambda}$ is defined in Section 2, with restricted tweak domain $[L - 1, L + \tau - 1]$.

In our proof, we use that the random oracle calls done by $\mathcal{A}$ can be observed by the reduction. We will construct an extractor which, for each $\mathbf{C}$ generated by $\mathcal{A}$, will inspect all the random oracle calls done by the adversary and use these to recursively reconstruct the leaves from the $\text{com}_{i,j}$ used to generate $\text{com} = h$. This is done because the extractor will brute-force some preimage k_α for each $\text{com}_{i,j}$ making it inefficient. However, after switching iv to injective mode, the inefficient extraction is guaranteed to succeed unless $\mathcal{A}$ cheats by breaking collision or preimage resistance of the random oracle. Both of these events can be bounded using a standard analysis of such events happening in random oracles, leading to the advantages of the Lemma. The full proof is presented in below.

Note that $\mathcal{A}$ chooses the challenges Δ_i , hence it can cheat in the sense that only an opening for Δ_i is “known” to $\mathcal{A}$, i.e. the (extracted) commitment messages cannot be opened for any other challenge than Δ_i . However, as long as the extractor extracts those messages, this does not constitute a success for the adversary — the adversary only wins if the extracted messages disagree with the values that it can open it to later.

The requirement that $\mathcal{A}$ checks its purported decommitment can be removed by replacing a non-conforming adversary $\mathcal{A}'$ with $\mathcal{A}$ where this check is added. Due to this change, the maximal number of queries Q_0 (resp. Q_1) to H_0 (resp. H_1) increases to at most $Q_0 + 1$ (resp. $Q_1 + \tau + 1$), which for concrete security makes almost no difference, since $Q_0, Q_1 \gg \tau$.

The security game of Lemma 32 is a multi-challenge setting, i.e., the adversary can try many commitments during the Commit-Phase before deciding to open one. This allows us to avoid losses caused by hybrid arguments: a reduction can simply extract all openings, and should a decommitment ever be inconsistent, it can choose it to for the Deccommitment-Phase to win the game.

Formal Proof

Lemma 32 (Extractable Binding of VOLE Commitment). *Consider the following extractable binding game for (**VOLECommit**, **VOLEReconstruct**), a stateful adversary $\mathcal{A}$ and a straightline extractor **Ext**:*

1. *Commit-Phase: Let $\mathcal{C} = \{\}$ be an empty associative array.*
 - (a) $\mathbf{C} = (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}}) \leftarrow \mathcal{A}^{H_0, H_1, H_4}(\text{commit})$.
 - (b) $\mathbf{D} = (\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)} = \text{Ext}^{H_4}(\mathcal{L}_0, \mathcal{L}_1, \text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$, where $\mathcal{L}_b$ is a set $\{(x_i, H_b(x_i))\}$ of query-response pairs from queries $\mathcal{A}$ made to the random oracle H_b .
 - (c) Set $\mathcal{C}[\mathbf{C}] = \mathbf{D}$. Note that $\mathbf{D} = \perp$ is allowed.
 - (d) $\mathcal{A}$ can either go to Step 1a or output $\mathbf{C}$ in $\mathcal{C}$ and the game continues below.
2. *Decommitment-Phase: Parse $\mathbf{C} = (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$.*
3. $(\text{chall}_3, \text{decom}_I) \leftarrow \mathcal{A}^{H_0, H_1, H_4}(\text{open})$ where $I = (\Delta_0, \dots, \Delta_{\tau-1})$ from chall_3 .
4. *If $I = \perp$ then output 0 (failure).*
5. *Let $\text{iv} = H_4(\text{iv}^{\text{pre}})$.*
6. *For $i \in [0, \tau)$ let $(\delta_{i,0}, \dots, \delta_{i,k_i})$ be the bit decomposition of Δ_i .*
7. $(\overline{\text{com}}, \mathbf{Q}) := \text{VOLEReconstruct}^{H_0, H_1}(\text{chall}_3, \text{decom}_I, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}, \hat{\ell}; \text{param})$.

8. Parse $[\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1} \mathbf{0}_{\ell,w}] = \mathbf{Q}$ where $\mathbf{Q}_i \in \mathbb{F}_2^{\ell \times k_i}$
9. Output 0 (failure) if
 - (a) $\overline{\text{com}} = \perp$ or $\text{com} \neq \overline{\text{com}}$ (i.e., verifier rejects), or
 - (b) $\mathbf{Q}_i = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot (\mathbf{u}_i^* \oplus \mathbf{c}_i), \dots, \delta_{i,k_b} \cdot (\mathbf{u}_i^* \oplus \mathbf{c}_i)]$, for all $i \in [0, \dots, \tau)$ (i.e., extraction correct).
10. Output 1 (success) otherwise.

Suppose that the hash family $\mathcal{H}$ is $\varepsilon_{\text{uhash}} = 2^{-3\lambda}$ -almost universal. Suppose $\mathcal{A}$ makes at most Q_0, Q_1 queries to the oracles $\mathbf{H}_0, \mathbf{H}_1$, respectively. Moreover, suppose that $\mathcal{A}$ checks if **VOLEReconstruct** returns $\perp$ for the purported opening decom_I , and outputs $\perp$ if that happens. There is a deterministic (unbounded) straightline extractor **Ext** (defined in the security proof for **FAEST**) such that the advantage of $\mathcal{A}$ in the above game is at most

$$(Q_1 + 1)^2 \cdot 2^{-2\lambda} + \tau Q_0 \cdot 2^{-\lambda}.$$

There is a deterministic (unbounded) straightline extractor **Ext** which is allowed to program $\mathbf{H}_4$ (defined in the security proof for **FAEST-EM**), such that the advantage of $\mathcal{A}$ in the above game is at most

$$(Q_1 + 1)^2 \cdot 2^{-2\lambda} + \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[Q_4, L].$$

where the running time of $\mathcal{B}$ is roughly that of $\mathcal{A}$. The IV-based tweakable PRG $\text{PRG}_{2\lambda}$ is defined in Section 2, with restricted tweak domain $[L - 1, L + \tau - 1]$.

We use following lemma to simplify the proof of Lemma 32.

Lemma 33 (Random oracle graph game). *Let $H: \{0, 1\}^* \rightarrow \mathcal{Y}$ be a random oracle and consider following game with an adversary $\mathcal{A}$. The game keeps track of a directed graph $G = (V, E)$, initially empty, and proceeds as follows: The adversary can (repeatedly) query H at some x . Let $H(x) = y$.*

- If $y \in V$ but there is no edge $(x', y) \in E$, then the adversary wins. (Preimage)
- If an edge $(x', y) \in E$ exists with $x' \neq x$, then the adversary wins. (Collision)
- Else, add nodes x and y to V and add an edge $e = (x, y)$ from x to y to E .

Let $\mathcal{A}$ be an adversary which makes at most Q queries to H . Then the probability that $\mathcal{A}$ wins is bounded by

$$Q^2 / |\mathcal{Y}|.$$

Proof of Lemma 33. The chances for $\mathcal{A}$ to win with a query to H (if it has not yet won) are at most $\frac{|V|}{|\mathcal{Y}|}$. Namely, the conditions on no edge to y or an existing edge to y separate V into two disjoint sets: Let V' be the set of nodes with no edges pointing to them. Let V'' be the set of nodes with edges pointing to them (that is, images under H). The adversary wins if a fresh query x yields an edge to a node y in V' (the case $\nexists x': (x', y) \in E$), or if x yields another edge to a node in V'' (the case $\exists x': (x', y) \in E \wedge x \neq x'$). (The former is a preimage attack, the latter is a collision attack.) Clearly, the chances that the former happens are $\frac{|V'|}{|\mathcal{Y}|}$ and the chances that the latter happens are $\frac{|V''|}{|\mathcal{Y}|}$, and since V' and V'' are disjoint, the chances that either happens is $\frac{|V|}{|\mathcal{Y}|}$.

Since the set V grows by at most 2 nodes per query, we can bound the success probability of $\mathcal{A}$ by

$$\sum_{i=1}^Q \frac{2 \cdot (i-1)}{|\mathcal{Y}|} \leq \frac{Q^2}{|\mathcal{Y}|}.$$

□

Proof of Lemma 32. We first analyze the straightline extractor for **FAEST**. At the end of the proof, we explain the modifications required for **FAEST-EM**. The core of the proof is the extraction of the BAVC commitment com , i.e., the seeds $\text{sd}_{i,j}$. Transforming the seeds, given $(\text{chall}_3, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv})$ to the VOLE output $\mathbf{Q}$ is simply post-processing of the opening. For convenience, the extractor also applies the post-processing to compute the VOLE relation (or $\perp$ if too many seeds failed to extract).

We define the straightline extractor **Ext**, by looking through the lists of queries $\mathcal{L}_1$ as follows.

1. Find a preimage $(h_0 \| \dots \| h_{\tau-1})$ of com under $\mathbf{H}_1$. If there is no unique preimage, i.e., there are none or multiple preimages, immediately output $\perp$.

2. For each $i \in [0.. \tau)$:
 - (a) Find a preimage $(\text{com}_{i,1}, \dots, \text{com}_{i,N_i})$ of h_i under H_1 , where $N_i = 2^k$ if $i < \tau_1$ and 2^{k-1} otherwise (cf. Table 2). If there is no unique preimage, i.e., there are none or multiple preimages, immediately output $\perp$.
 - (b) For each $j \in [0..N_i)$, brute-force a preimage r for $\text{com}_{i,j}$ under $r \mapsto \text{LeafCommit}(r, \text{iv}, i + L - 1, \text{uhash}_i)$, where uhash_i is set as $(\text{uhash}_0, \dots, \text{uhash}_{\tau-1}) = H_0(\text{iv}; 3\lambda\tau)$ and iv is the IV used in the commit. That is, find r with $(\text{sd}_{i,j}, \text{com}_{i,j}) = \text{LeafCommit}(r, \text{iv}, i + L - 1, \text{uhash}_i)$. If there is no such preimage, set $\text{sd}_{i,j} = \perp$. If there are multiple preimages, immediately output $\perp$. Otherwise, remember $\text{sd}_{i,j}$ corresponding to $\text{com}_{i,j}$.

Then, for each of the $i \in [0, \tau)$ VOLEs, Ext distinguishes following cases:

- Case 1 (All seeds found): If all $(\text{sd}_{i,j})_{j \in [0, N_i)}$ are not $\perp$ for i , then Ext computes $\mathbf{V}_i^*$ and $\tilde{\mathbf{u}}_i$ honestly (i.e., via [ConvertToVOLE](#)) and sets $\Delta_i^* = \perp$.
- Case 2 (Missing one seed): If a single seed is missing, say $\text{sd}_{i,j^*} = \perp$, then Ext sets $\Delta_i^* = j^*$ and, as in [VOLEReconstruct](#), computes $(\tilde{\mathbf{u}}_i, \mathbf{q}'_{i,0}, \dots, \mathbf{q}'_{i,k_b-1}) = \text{ConvertToVOLE}(\perp, \text{sd}_{i,1 \oplus \Delta_i^*}, \dots, \text{sd}_{i,(N_i-1) \oplus \Delta_i^*}, \text{iv}; \hat{\ell})$ and lets $\mathbf{Q}_0 = [\mathbf{q}'_{0,0} \cdots \mathbf{q}'_{0,k_b-1}]$ and $\mathbf{u}_0^* = \tilde{\mathbf{u}}_0$ as well as $\mathbf{Q}_i = [\mathbf{q}'_{i,0} \cdots \mathbf{q}'_{i,k_b-1}] + [\delta_{i,0} \cdot \mathbf{c}_i \cdots \delta_{i,k_b-1} \cdot \mathbf{c}_i]$ and $\mathbf{u}_i^* = \tilde{\mathbf{u}}_i \oplus \mathbf{c}_i$ for $i > 0$. With that, Ext defines $\mathbf{V}_i^* = \mathbf{Q}_i + [\delta_{i,0} \cdot \mathbf{u}_i^*, \dots, \delta_{i,k_b} \cdot \mathbf{u}_i^*]$, for $i \in [0, \tau)$.
- Case 3 (Missing multiple seeds): If two or more seeds are $\perp$ (due to missing or non-unique preimages), say $\text{sd}_{i,j_1^*} = \perp = \text{sd}_{i,j_2^*}$ for $j_1^* \neq j_2^*$, then Ext outputs $\perp$, i.e., extraction fails.

This yields the output $(\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)}$ of Ext. While not part of its output, we make use of the bad challenges $(\Delta_i^*)_{i \in [0, \tau)}$ found during extraction later. Finally, to invoke Lemma 33 later, we add the following actions to Ext if extraction failed:

- If extraction failed since the preimage for com (resp., some preimage for h_i) was missing, in step 1 (resp. step 2), query $H_1(h_i)$ (resp., pick the smallest h_i without preimage and query $H_1(h_i)$). (Lemma 33 will then rule out that the adversary finds a preimage after extraction failure.)

Note that Ext will make at most one query to H_1 (per extraction).

To analyse Ext, we first consider the (graph from) the game from Lemma 33 and define by Fail_1 the (failure) event that, during the extractable binding game, a preimage attack or a collision attack succeeded in the random oracle graph for H_1 .

Consider the whole binding game as an adversary $\mathcal{A}$ to the random oracle graph game in Lemma 33, where only the subroutine $\mathcal{A}'$ of $\mathcal{A}$ makes fresh random oracle queries. Observe that if the bad event Fail_1 occurs, then $\mathcal{A}'$ wins Lemma 33. This holds because, either the adversary finds a random oracle collision, or it finds a preimage for a value x which Ext could not extract. To cover the latter case in the random oracle game of Lemma 33, Ext makes a additional query $H_1(x)$ which adds x as a node to the random oracle game, so that if the adversary finds the missing preimage of x , then the game aborts.

As a consequence, failing to look up com or h_i can trigger an extra query to H_1 by Ext. By Lemma 33 and a union bound, we therefore find

$$\Pr[\text{Fail}_1] \leq (Q_1 + 1)^2 / 2^{2\lambda}$$

As a second step, consider the failure event $\text{Fail}_{\text{uhash}}$ which by definition occurs if brute-forcing $\text{sd}_{i,j}$ finds multiple preimages. By assumption, the hash family $\mathcal{H}$ (used in [LeafCommit](#) to hash $(\text{sd}, \mathbf{x}_1)$) is $\varepsilon_{\text{uhash}}$ -almost universal for $\varepsilon_{\text{uhash}} = 2^{-3\lambda}$. Moreover, the map $r \mapsto \text{LeafCommit}(r, \text{iv}, i + L - 1, \text{uhash}_i)$ has input domain $\{0, 1\}^\lambda$ of size 2^λ for each fixed $i \in [0.. \tau)$ and iv . Hence, at most 2^λ different values $(\text{sd}, \mathbf{x}_1)$ can be hashed through $\mathcal{H}_{\text{uhash}_i}(\cdot)$ for fixed (i, iv) . Since uhash_i is a random oracle output and thus is independent of the hashed inputs, we find that for fixed i the map $r \mapsto \text{LeafCommit}(r, \text{iv}, i + L - 1, \text{uhash}_i)$ is injective except with probability at most $(2^\lambda)^2 \cdot \varepsilon_{\text{uhash}} \leq 2^{-\lambda}$. (Namely, by $\varepsilon_{\text{uhash}}$ -almost universality, for any $\mathbf{x} \neq \mathbf{y} \in \{0, 1\}^{4\lambda}$ the probability that $\mathcal{H}_{\text{uhash}_i}(\mathbf{x}) = \mathcal{H}_{\text{uhash}_i}(\mathbf{y})$ is at most $\varepsilon_{\text{uhash}}$ (over uniform uhash_i). Hence, for any set $\mathcal{S}$ of size $S = |\mathcal{S}|$, by a union bound over $\mathbf{x} \neq \mathbf{y} \in \mathcal{S}$, we have a collision with probability at most $S^2/2 \cdot \varepsilon_{\text{uhash}}$. Plugging in the size $S = 2^\lambda$ which is hashed by [LeafCommit](#) for fixed (i, iv) , and $\varepsilon_{\text{uhash}} \leq 2^{-3\lambda}$, a hash function in $\mathcal{H}$ is injective with probability at least $2^{-\lambda}$ (for uniform uhash_i).)

As we assume that the adversary runs [VOLEReconstruct](#), hence queried H_0 to learn $(\text{uhash}_0, \dots, \text{uhash}_{\tau-1})$ for iv , we find (by a union bound over all τ choices of $i \in [0.. \tau)$ and Q_0 queries to H_0) that

$$\Pr[\text{Fail}_{\text{uhash}}] \leq \tau Q_0 \cdot 2^{-\lambda}$$

In the following, we analyze the extraction conditioned on $\neg\text{Fail}$ where $\text{Fail} = \text{Fail}_1 \vee \text{Fail}_{\text{uhash}}$. Consequently, whenever the extractor does not know a preimage the game will not encounter one (either due to a query of $\mathcal{A}$ or during **VOLEReconstruct**), and likewise, there will never be multiple preimages. We observe the following:

- **com** uniquely defines $(h_0 \| \dots \| h_{\tau-1})$.
- h_i uniquely defines $(\text{com}_{i,0}, \dots, \text{com}_{i,N_i-1})$ for $i \in [0, \tau)$.
- $\text{com}_{i,j}$ uniquely defines $\text{sd}_{i,j}$.¹⁵

Thus, it is only possible for $\mathcal{A}$ to open **com** successfully (i.e., $\text{com} = \overline{\text{com}} \neq \perp$) if

- Whenever $\Delta_i^* = \perp$, i.e., h_i could be fully extracted, then

$$\mathbf{Q}_i = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot \mathbf{u}_i^* \cdots \delta_{i,k_b} \cdot \mathbf{u}_i^*].$$

This can be seen as follows: By the condition $\neg\text{Fail}$, the reconstructed seeds $\text{sd}'_{i,j}$ in **VOLEReconstruct** coincide with the (extracted) seeds $\text{sd}_{i,j \oplus \Delta_i}$ for $j \in [0, N)$, except that $\text{sd}'_{i,j} = \perp$. Proposition 30 asserts that in this case, the equality $\mathbf{Q}_i = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot \mathbf{u}_i^* \cdots \delta_{i,k_b} \cdot \mathbf{u}_i^*]$ holds, as required.¹⁶

- Whenever $\Delta_i^* \neq \perp$, the choice Δ_i of $\mathcal{A}$ is Δ_i^* . Otherwise, $\mathcal{A}$ would lose: If $\Delta_i^* \neq \perp$, then at least one commitment $\text{com}_{i,j}$ could not be extracted, hence this challenge must be used by $\mathcal{A}$ otherwise it could not successfully decommit (due to condition $\neg\text{Fail}$). If challenge Δ_i^* is used, then by condition $\neg\text{Fail}$ and Proposition 30, we again find that the equality $\mathbf{Q}_i = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot \mathbf{u}_i^* \cdots \delta_{i,k_b} \cdot \mathbf{u}_i^*]$ holds, as required.

Thus, we have shown that if **Fail** does not occur, extraction always succeeds and coincides with the opening in the Decommitment-Phase. This concludes the proof for **FAEST**.

Now, we explain the changes required for **FAEST-EM**. Essentially, we can apply the exact same argument, except that we reduce to almost injectivity (Definition 27) to ensure that extraction of the challenge commitments succeeds. The only difference, due to the different **LeafCommit**, which occurs in the proof is an extraction failure when $\text{com}_{i,j}$ has multiple preimages. To exclude this, we reduce to almost injectivity by outputting the challenge iv and $Q_{\text{out}} = L$ commitments, namely, all the $\text{com}_{i,j}$ which belong to the challenge commitment. Observe that an extraction failure due to multiple preimages for $\text{com}_{i,j}$ now implies that the reduction wins the almost injectivity game. Thus, we constructed an adversary $\mathcal{B}$ with advantage $\text{AdvInj}_{\mathcal{B}}^{\text{PRG}^{2\lambda}}[L]$ and running time roughly that of $\mathcal{A}$. $\square$

3.4 Additional Properties: Hiding and Unpredictability

As well as binding, we show that the VOLE commitments satisfy hiding and unpredictability properties.

Hiding. We define a standard hiding game, where an adversary is asked to distinguish between the (public) commitment part of the output of **VOLECommit** from a random string. The proof follows in two steps: First, we replace the hidden outputs of the BAVC commitments with true randomness. To do this, we make use of the fact that the GGM construction is a selectively secure puncturable PRF from a length-doubling PRG, and use hybrids to (eventually) randomize the unopened seeds as well as the opened commitments. One subtle detail of this analysis is that, since we rely on an AES-based PRG and AES is a PRP, a naive approach would involve a security loss from applying the PRP/PRF switching lemma. To obtain a tighter bound, we apply a Renyi divergence-like argument, which allows us to switch the AES-CTR outputs for random values with only a small constant security loss. Using this technique, we can successively replace **PRG** calls along the path from the root to each unopened leaf by truly random outputs. In a second step, we make the VOLE secret $\mathbf{u}$ uniformly random. This follows by again using the pseudorandomness of **PRG**.

¹⁵ Note that attacks on the GGM tree, i.e., on **PRG**, are not possible here, as the GGM tree is only used to compress the decommitment for **BAVC.Reconstruct**. The checks and extraction rely only on $(\text{sd}_{i,j}, \text{com}_{i,j}) = \text{LeafCommit}(\dots)$, in the notation of **BAVC.Reconstruct**.

¹⁶ More explicitly, it follows from Proposition 30 and $\mathbf{Q}_0 = \mathbf{V}_0^* \oplus [\delta_{0,0} \cdot \mathbf{u}'_0 \cdots \delta_{0,k_0-1} \cdot \mathbf{u}'_0]$ as well as $\mathbf{Q}_i = \mathbf{Q}'_i \oplus [\delta_{i,0} \cdot \mathbf{c}_i \cdots \delta_{i,k_i-1} \cdot \mathbf{c}_i] = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot \mathbf{u}'_i \cdots \delta_{i,k_i-1} \cdot \mathbf{u}'_i] \oplus [\delta_{i,0} \cdot \mathbf{c}_i \cdots \delta_{i,k_i-1} \cdot \mathbf{c}_i]$ for $i > 0$.

Unpredictability. We also need a notion of unpredictability, meaning that an adversary cannot guess the value of a commitment output by **VOLECommit** without knowing the commitment randomness. While intuitively, unpredictability follows from the hiding property, by analyzing this property separately, we are able to obtain a tighter bound.

Hiding. We now show that **VOLECommit** is hiding. To do this, we model our use of random IVs in PRGs with the following multi-target security game. The batch size L denotes the number of times the PRG is used with the same¹⁷ IV, while the number of queries Q denotes the number of independent IVs used.

Remark 34 (BAVC opening aborts). Observe that **BAVC.Open** aborts for certain challenges $I = (\Delta_0, \dots, \Delta_{\tau-1})$. These aborts depend solely on I , and are independent from **decom**. In particular, it is efficiently verifiable if a choice I causes an abort.

Lemma 35 (VOLECommit is multi-hiding). *Let H_0, H_1 be random oracles. Consider the following multi-hiding experiment for $\text{VOLECommit}^{H_0, H_1}$ and a Q -query adversary $\mathcal{A}$, defined for some arbitrary $\hat{\ell}$, param .*

1. Sample $b^* \leftarrow \{0, 1\}$
2. For $j \in [1, Q]$:
 - $r^j \leftarrow \{0, 1\}^\lambda$, $\text{iv}^j \leftarrow \{0, 1\}^{128}$
 - $(\text{com}^j, \text{decom}^j, (\mathbf{c}_i^j)_{i \in [1, \tau]}, \mathbf{u}^j, \mathbf{V}^j) := \text{VOLECommit}(r^j, \text{iv}^j, \hat{\ell}; \text{param}, \text{param}_{\text{VOLE}})$
 - Sample $\text{chall}_3^j \leftarrow \{0, 1\}^{\lambda - w_{\text{grind}}} \times 0^{w_{\text{grind}}}$ uniformly, conditioned on **BAVC.Open** not aborting for $I^j = (\Delta_0^j, \dots, \Delta_{\tau-1}^j)$ based on chall_3^j (cf. Remark 34).
 - $\text{decom}_{I^j} := \text{BAVC.Open}(\text{decom}^j, I^j)$
 - Define

$$(\bar{\mathbf{u}}^j, \bar{\mathbf{c}}_1^j, \dots, \bar{\mathbf{c}}_{\tau-1}^j) = \begin{cases} (\mathbf{u}^j, \mathbf{c}_1^j, \dots, \mathbf{c}_{\tau-1}^j) & \text{if } b^* = 0 \\ \text{random from } (\mathbb{F}_2^{\hat{\ell}})^\tau & \text{if } b^* = 1 \end{cases}$$

3. $b \leftarrow \mathcal{A}^{H_0, H_1}((\bar{\mathbf{u}}^j, (\bar{\mathbf{c}}_i^j)_{i \in [1, \tau]}, \text{decom}_{I^j}, \text{chall}_3^j, \text{com}^j)_{j \in [1, Q]})$
4. Output 1 (success) if $b = b^*$. Else output 0 (failure).

Write $\text{PRG}_{\text{len}(s)} = \text{PRG}(s; \text{len})$ and $\beta = \frac{\lambda}{128}$. Suppose that the hash family $\mathcal{H}$ is $\mathbb{F}_2^\lambda$ -hiding. Let G_i^* be the hiding game with $b^* = i$. Then we have

$$\Pr[G_0^* = 1] \leq e^{Q\tau(4\lceil \log(L) \rceil \beta^2 + 16\beta^2 N_0^2 + (\lceil \hat{\ell}/128 \rceil)^2)/2^{128}} \cdot \left(\Pr[G_1^* = 1] + Q\tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRG}_{\mathcal{B}_2}^{\text{LeafCommit}}[N_0] + \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\hat{\ell}}}[1] \right) \right)$$

where by abuse of notation, we write $\text{AdvPRG}_{\mathcal{B}_3}^{\text{LeafCommit}}$ which is

- $\text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{4\lambda}}$ in FAEST
- $\text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{2\lambda}}$ in FAEST-EM

as the advantage pertains to the respective **PRG** evaluation in **LeafCommit**. Moreover, the reductions $\mathcal{B}_1, \mathcal{B}_2$ are black-box and oblivious of the random oracles H_0, H_1 . The same holds against quantum adversaries, in which case the advantage terms are with respect to quantum adversaries.

Note that we can sample the challenges $(\text{chall}_3^j)_{j \in [1, Q]}$ before committing without changing the adversary's view (because whether or not **BAVC.Open** rejects a challenge is independent from the actual commitment). Hence, all decommitment indices for all VC commitments are known at commitment time. That is, we only assert selective security.

¹⁷ In the AES-CTR construction we use, increasing L degrades security by a factor of up to L , while increasing Q does not lead to any practical loss.

Proof. We assume that $(\text{chal}_3^j)_{j \in [1, Q]}$ is sampled before the commitments are generated, so that we are obviously in a selective security setting for BAVC openings.

Game G_0 (Real hiding experiment with $b = 0$). By definition, $G_0 = G_0^*$ is the hiding experiment with the challenge bit $b^* = 0$, i.e., the outputs $(\bar{\mathbf{u}}^j, \bar{\mathbf{c}}_1^j, \dots, \bar{\mathbf{c}}_{\tau-1}^j)$ corresponds to the generated commitment.

For simplicity, we describe the reductions first for $Q = 1$ instead of the multi-challenge setting and omit the superscript j . Since all **VOLECommit** instances are independent, it is straightforward to generalize it to $Q > 1$, as we will show in the final step.

Game G_1 (Randomize hidden GGM tree seeds). In game G_1 , we replace all hidden seeds k_{α_i} in the (de)commitment with truly random seeds.

We make use of the proof that the GGM construction is a selectively secure puncturable PRF from a length-doubling PRG, and use the hybrids to (eventually) randomize the hidden seeds. Following that proof, we successively replace **PRG** calls along the path from the root to the leaf $\alpha_i = \text{BAVC.PosInTree}(i, \Delta_i)$ (for $i \in [0, \tau)$) by truly random outputs. After at most $\tau \lceil \log(L) \rceil$ hybrid steps, the leaf seeds k_{α_i} are replaced with random values. By applying Corollary 22 and Remark 23 we arrive at

$$\begin{aligned} \Pr[G_0 = 1] &\leq (e^{(2\beta)^2/2^{128}})^{\tau \lceil \log(L) \rceil} (\Pr[G_1 = 1] + \sum_{i=1} \text{AdvPRP}_{\mathcal{B}_{1,i}}^{\text{PRG}_{2\lambda}}[1]) \\ &= e^{4\tau \lceil \log(L) \rceil \beta^2/2^{128}} (\Pr[G_1 = 1] + \tau \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1]) \end{aligned}$$

since each call to $\text{PRG}(k_{\alpha_i}, \text{iv}, \alpha; 2\lambda)$ in step 1 of **BAVC.Commit** outputs $2\beta = \frac{2\lambda}{128}$ blocks for $T = 1$ queries and we have at most $\tau \lceil \log(L) \rceil$ hybrid steps.

Game G_2 (Randomize hidden **LeafCommit** outputs). In game G_2 for **FAEST**, we replace the outputs $(\text{sd}_{i,\Delta_i}, \text{com}_{i,\Delta_i})$ corresponding to the hidden leaves $\alpha_0, \dots, \alpha_{\tau-1}$ in step 2 of **BAVC.Commit** by sampling $\text{sd}_{i,\Delta_i} \leftarrow \{0, 1\}^\lambda$ and $\text{com}_{i,\Delta_i} \leftarrow \mathcal{H}_{\text{uhash}}(0^\lambda, \mathbf{x}_1)$ for $\mathbf{x}_1 \leftarrow \{0, 1\}^{3\lambda}$.

In game G_2 for **FAEST-EM**, we instead sample $\text{com}_{i,\Delta_i} \leftarrow \{0, 1\}^{2\lambda}$. (Since we consider hidden sd_{i,Δ_i} , we do not need to sample it in **FAEST-EM**.) For simplicity, we focus on **FAEST.LeafCommit** below.

Since in game G_1 , all seeds k_{α_i} are truly random, we can apply pseudorandomness of $\text{PRG}_{4\lambda}$ to sample $(\text{sd}_{i,\Delta_i}, \mathbf{x}_1) \leftarrow \{0, 1\}^{4\lambda}$ uniformly for the corresponding to leaves $\alpha_0, \dots, \alpha_{\tau-1}$ in step 2 of **BAVC.Commit**. Again, we apply Corollary 22 and Remark 23 to arrive at

$$\begin{aligned} \Pr[G_1 = 1] &= (e^{(4\beta)^2 N_0^2/2^{128}})^{\tau} (\Pr[G_2 = 1] + \sum_{i=1}^{\tau} \text{AdvPRP}_{\mathcal{B}_{2,i}}^{\text{PRG}_{4\lambda}}[N_i]) \\ &\leq e^{16\tau\beta^2 N_0^2/2^{128}} (\Pr[G_2 = 1] + \tau \cdot \text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{4\lambda}}[N_0]) \end{aligned}$$

since each call $\text{PRG}(k_{\alpha_i}, \text{iv}, \alpha; 4\lambda)$ outputs $4\beta = \frac{4\lambda}{128}$ blocks for $T = N_i$ queries, and $N_i \leq N_0$ by definition of N_i (see Table 2). Now observe, that for uniform $\mathbf{x}_1 \in \{0, 1\}^{3\lambda}$, the output $\text{com}_{i,\Delta_i} \leftarrow \mathcal{H}_{\text{uhash}}(\text{sd}_{i,\Delta_i}, \mathbf{x}_1) \in \{0, 1\}^{3\lambda}$ is independent of sd_{i,Δ_i} by $\mathbb{F}_2^\lambda$ -hiding of $\mathcal{H}$. Hence, we can compute $\text{com}_{i,\Delta_i} \leftarrow \mathcal{H}_{\text{uhash}}(0^\lambda, \mathbf{x}_1) \in \{0, 1\}^{3\lambda}$ without changing the distribution. In particular, sd_{i,Δ_i} and com_{i,Δ_i} are stochastically independent now and can be sampled as $\text{sd}_{i,\Delta_i} \leftarrow \{0, 1\}^\lambda$ and $\text{com}_{i,\Delta_i} \leftarrow \mathcal{H}_{\text{uhash}}(0^\lambda, \mathbf{x}_1)$ for $\mathbf{x}_1 \leftarrow \{0, 1\}^{3\lambda}$ without affecting the distribution.

Coming back to **FAEST-EM**, the only difference is that instead of $\text{AdvPRP}_{\mathcal{B}_{2,i}}^{\text{PRG}_{4\lambda}}[N_i]$, we reduce to a PRG with 2λ output length, i.e., we have $\text{AdvPRP}_{\mathcal{B}_{2,i}}^{\text{PRG}_{2\lambda}}[N_i]$ in the above and the factor 16 becomes 4.

Game G_3 (Switch **ConvertToVOLE** to random). In game G_3 , we replace the PRG output of $\mathbf{r}_{0,0} = \text{PRG}(\text{sd}_{i,\Delta_i}, \text{iv}, \text{twk}; \hat{\ell})$ (where $\text{twk} = i + 2^{31}$) in Item 2 of that **ConvertToVOLE** computation called from **VOLECommit** in step 2a by true randomness. In the output of **ConvertToVOLE**, $\mathbf{u}$ equals $\mathbf{u} = \sum_{i \in [0, \tau)} \text{PRG}(\text{sd}_i, \text{iv}, i; \hat{\ell})$ and thus if $\mathbf{r}_{0,0}$ is truly random, so is $\mathbf{u}$. Hence, all $\mathbf{u}_i$ in **VOLECommit** generated by step 2a are truly random. Consequently, the outputs $\mathbf{u}_0$ and $\mathbf{c}_i = \mathbf{u}_i \oplus \mathbf{u}$ (from lines 2a and 2c) are truly random in G_3 .

By the change in game G_2 , we know that the seed sd_{i,Δ_i} which is input into **ConvertToVOLE** in step 2a of **VOLECommit** is truly random, so we can reduce to PRG security as before. We get

$$\begin{aligned} \Pr[G_2 = 1] &= (e^{(\lceil \hat{\ell}/128 \rceil)^2/2^{128}})^{\tau} (\Pr[G_3 = 1] + \sum_{i=1}^{\tau} \text{AdvPRP}_{\mathcal{B}_{3,i}}^{\text{PRG}_{\hat{\ell}}}[1]) \\ &\leq e^{\tau(\lceil \hat{\ell}/128 \rceil)^2/2^{128}} (\Pr[G_3 = 1] + \tau \cdot \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\hat{\ell}}}[1]) \end{aligned}$$

since we have $\frac{\lceil \hat{\ell} \rceil}{128}$ blocks of output in $\text{PRG}(\text{sd}_i, \text{iv}, i; \hat{\ell})$ in for the i -th run of ConvertToVOLE (for $i \in [0, \tau - 1]$), where each only has $T = 1$ queries. All in all, we have shown that modifying the single-challenge experiment from $b^* = 0$ to $b^* = 1$ is indistinguishable.

Final step: Handling $Q > 1$. Let $\mathbf{G}_4$ the multi-challenge game with $b^* = 1$. By plugging the above inequalities for games 1 to 3 in another, we see that game $\mathbf{G}_3$ (for $Q = 1$) satisfies

$$\Pr[\mathbf{G}_0 = 1] \leq e^{(4\tau \lceil \log(L) \rceil \beta^2 + 16\tau \beta^2 N_0^2 + \tau(\lceil \hat{\ell}/128 \rceil)^2)/2^{128}} \cdot \left(\Pr[\mathbf{G}_3 = 1] + \right. \\ \left. \tau \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + \tau \cdot \text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{4\lambda}}[N_0] + \tau \cdot \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\ell}}[1] \right)$$

Applying the same reasoning for a hybrid with Q commitments, we therefore arrive at

$$\Pr[\mathbf{G}_0 = 1] \leq (e^{(4\tau \lceil \log(L) \rceil \beta^2 + 16\tau \beta^2 N_0^2 + \tau(\lceil \hat{\ell}/128 \rceil)^2)/2^{128}})^Q \cdot \left(\Pr[\mathbf{G}_4 = 1] + \right. \\ \left. Q\tau \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + Q\tau \cdot \text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{4\lambda}}[N_0] + Q\tau \cdot \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\ell}}[1] \right) \\ \leq e^{Q\tau(4\lceil \log(L) \rceil \beta^2 + 16\beta^2 N_0^2 + (\lceil \hat{\ell}/128 \rceil)^2)/2^{128}} \cdot \left(\Pr[\mathbf{G}_4 = 1] + \right. \\ \left. Q\tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{4\lambda}}[N_0] + \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\ell}}[1] \right) \right)$$

This concludes the claim for the multi-challenge setting. *Note that the contribution of $\text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{4\lambda}}$ changes to $\text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{2\lambda}}$ for FAEST-EM (and the term $16\beta^2 N_0^2$ could be reduced to $4\beta^2 N_0^2$).*

Observing that we did not make use of $\mathcal{A}$'s structure in any way yields the post-quantum security statement. $\square$

Unpredictability. We define unpredictability in a slightly non-standard manner, namely we let the adversary $\mathcal{A}$ output a bit and replace that bit with 0 if it succeeded in guessing. This is tailored to our application of unpredictability, where $\mathcal{A}$ will actually be a (hybrid) game which outputs 1 if the EUF-CMA adversary succeeds. Hence, replacing the output 1 with 0 means that the EUF-CMA adversary loses the modified EUF-CMA game if it predicts a commitment.

We note that, intuitively, unpredictability follows immediately from hiding. However, to obtain a tighter bound, we observe that it is sufficient for unpredictability to randomize a single unopened leaf of the GGM tree. This randomizes its commitment and thus $\text{com} = h$. By using that $\mathcal{A}$ will likely not have made queries involving the randomized leaf commitment to the RO using a standard bound, the bound claimed in Lemma 36 follows.

Lemma 36 (VOLECommit is unpredictable). *Fix some $\hat{\ell} \in \mathbb{N}$ and admissible $\text{param}, \text{param}_{\text{VOLE}}$. Let $\mathcal{A}$ be an adversary in the following game $\text{ExpUnpred}_{b, \mathcal{A}}^{\text{VOLECommit}, \hat{\ell}, \text{param}, \text{param}_{\text{VOLE}}}(\lambda)$:*

1. $(\mathcal{L}_{\text{com}}, \text{state}) \leftarrow \mathcal{A}^{\text{H}_0, \text{H}_1}(1^\lambda)$. Here $\mathcal{L}_{\text{com}}$ is list of commitment guesses.
2. $r \leftarrow \{0, 1\}^\lambda$, $\text{iv} \leftarrow \{0, 1\}^{128}$
3. $C = (\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(r, \text{iv}, \hat{\ell}; \text{param}, \text{param}_{\text{VOLE}})$
4. If $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}) \notin \mathcal{L}_{\text{com}}$ or $b = 0$, return $b_{\mathcal{A}} \leftarrow \mathcal{A}^{\text{H}_0, \text{H}_1}(\text{state}, C)$. Else return 0.

For brevity, let $\mathbf{G}_b$ denote $\text{ExpUnpred}_{b, \mathcal{A}}^{\text{VOLECommit}, \hat{\ell}, \text{param}, \text{param}_{\text{VOLE}}}$ for $b \in \{0, 1\}$. Also let $\beta = \frac{\lambda}{128} \in \mathbb{N}$. For any adversary $\mathcal{A}$ which makes at most Q_1 queries to H_1 , and outputs lists $\mathcal{L}$ of size at most Q_{com} we have

$$\Pr[\mathbf{G}_0 = 1] \leq e^{(4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{128}} \cdot \left(\Pr[\mathbf{G}_1 = 1] \right. \\ \left. + \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}_{\mathcal{B}_2}^{\text{LeafCommit}}[1] + (2Q_1 + Q_{\text{com}}) \cdot 2^{-2\lambda} \right)$$

where by abuse of notation, we write $\text{AdvPRP}_{\mathcal{B}_2}^{\text{LeafCommit}}$ for $\text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{4\lambda}}$ in FAEST, and $\text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}$ in FAEST-EM. Moreover, the reductions $\mathcal{B}_1, \mathcal{B}_2$ are black-box and oblivious of the random oracles H_0, H_1 .

Proof. Let G_b denote $\text{ExpUnpred}_{b,A}^{\text{VOLECommit}, \ell, \text{param}, \text{param}_{\text{VOLE}}}$. Observe that G_0 and G_1 are identical unless the bad event F , that $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}) \in \mathcal{L}_{\text{com}}$, occurs in Item 4. Thus, we have

$$\Pr[G_0 = 1] \leq \Pr[G_1 = 1] + \Pr[F]$$

Hence, it suffices to bound $\Pr[F]$. To analyze $\Pr[F]$, we define the game G'_0 which is identical to G_0 and G_1 except for Step 4 which is replaced with

4. If $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}) \in \mathcal{L}_{\text{com}}$, return 1. Else return 0.

By definition, $\Pr[F] = \Pr[G'_0 = 1]$. To bound $\Pr[G'_0 = 1]$, we can follow the steps of the hiding proof. Namely, first apply the same game hops as in games G_0 to G_2 of the proof of Lemma 35 but only to randomize a single leaf, say $(i, j) = (0, 0)$ and denote the resulting games G'_1 and G'_2 . (Do not continue to G_3 , as we need not randomize $\mathbf{u}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$.) Then, the leaf commit $\text{com}_{0,0} \in \{0, 1\}^{2\lambda}$ is truly random and we have

$$\begin{aligned} \Pr[G'_0 = 1] &\leq e^{(4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{128}} \cdot \left(\Pr[G'_2 = 1] \right. \\ &\quad \left. + \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \end{aligned} \quad (5)$$

Observe here, that the reductions $\mathcal{B}_1, \mathcal{B}_2$ from Lemma 35 are fully black-box and oblivious to the oracle-access to H_0, H_1 .

Let us now bound $\Pr[G'_2 = 1]$. The probability that H_1 was queried by $\mathcal{A}$ on the random $\text{com}_{0,0} \in \{0, 1\}^{2\lambda}$ is at most $Q_1/2^{2\lambda}$. Unless that happened, h_0 is uniformly random (independent of $\mathcal{A}$'s view). Further, the probability that H_1 was queried by $\mathcal{A}$ on an input starting with the random $h_0 \in \{0, 1\}^{2\lambda}$ is at most $Q_1/2^{2\lambda}$. Unless that happened, com is uniformly random. Finally, note that when com is uniformly random, then the probability that $(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \in \mathcal{L}_{\text{com}}$ is at most $Q_{\text{com}}/2^{2\lambda}$. Hence

$$\Pr[G'_2 = 1] \leq (2Q_1 + Q_{\text{com}}) \cdot 2^{-2\lambda}. \quad (6)$$

Putting things together, we find that

$$\begin{aligned} \Pr[G_0 = 1] &\leq \Pr[G_1 = 1] + e^{(4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{128}} \cdot \left((2Q_1 + Q_{\text{com}}) \cdot 2^{-2\lambda} \right. \\ &\quad \left. + \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \end{aligned}$$

which then yields the claimed bound. $\square$

4 The Optimized FAEST Signature Scheme

We now use properties of VOLE commitments and the AES S-box to optimize AES circuit evaluation. Based on this, we introduce the three algorithms of our modified FAEST signature. The description is based on the most simple setting of AES-128 as OWF. Other security levels use a different OWF, while FAEST-EM additionally uses a different instance of [LeafCommit](#) as previously described.

4.1 Optimized Constraints in FAEST

If we want to show that (VOLE) commitments to an AES key k allow to verify that $\text{AES-128}(k, x) = y$ for publicly known x, y , then computing AES-128 can be separated into two phases: first, the AES key scheduling is applied to k to compute commitments to all AES round keys. Then, the actual AES round function is applied iteratively. Here, commitments to round inputs and the corresponding round key must produce commitments to the output of each round. The initial round input is x , while the round output of round i becomes round input to round $i + 1$. In this work we only focus on optimizing the round function and not the key scheduling. The latter only contains a small share of the total number of S-boxes and doesn't lend itself to our optimization.

In FAEST, the prover creates bit commitments for the output of each S-box, while commitments to the inputs can be derived linearly from commitments to outputs of previous AES rounds and the round

key. Then to check the S-box, the bit commitments are first lifted to commitments over $\mathbb{F}_{2^8}$. Using that the AES S-box simply inverts its input x to produce the output y , FAEST checks the degree-2 constraint that $x \cdot y = 1$ over $\mathbb{F}_{2^8}$ which also requires $x \neq 0$. This check uses that VOLE commitments can be multiplied (which increases their degree) and attests that the product of the commitments to x and y is a degree-2 commitment to 1. Our approach is to check constraints of degree up to 3 and makes use of the field norm $N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}$. Here, we use that the norm for every $x \in \mathbb{F}_{2^8}$ is in $\mathbb{F}_{2^4}$ and can thus be represented using 4 instead of 8 bit commitments. We also use that computing x^2 for $x \in \mathbb{F}_{2^8}$ is an $\mathbb{F}_2$ -linear operation on the bit decomposition of x , which allows us to obtain degree-1 commitments to all 2^i -powers of x .

Some Rounds Need Less Bits. We can split the AES rounds into *even* and *odd* rounds. In every *even* round, rather than committing to the full 8-bit S-box output, we commit only to a 4-bit value $c = a^{-17}$, which is the inverse of $N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(a) = a^{17}$ of the S-box input $a \in \mathbb{F}_{2^8}$. To reconstruct a commitment of the full inverse a^{-1} one can multiply the commitment of c with the commitment to a^{16} (which can be computed from the commitments of the bit decomposition of a by the observation on squaring mentioned above). The S-box relation can be checked with the degree-3 constraint:

$$c \cdot a^2 \cdot a^{16} = a.$$

In *odd* rounds, the prover commits to the entire 8-bit inverse $y = x^{-1}$ and verifies it via $x^2 \cdot y = x$ and $x \cdot y^2 = y$. This checks the AES S-box input-output relation while allowing for S-box inputs¹⁸ of 0, as observed in [BBM⁺24, DH23].

Back to Bits. However, this approach doesn't get us commitments to the individual bits of $c \cdot a^{16}$, but rather a single commitment as an element of $\mathbb{F}_{2^8}$. This means we can only perform $\mathbb{F}_{2^8}$ -homomorphic operations on the commitment, as opposed to $\mathbb{F}_2$ -homomorphic operations on the individual bits, which is a problem because the linear parts of the AES round function is $\mathbb{F}_2$ -linear but not $\mathbb{F}_{2^8}$ -linear. We solve this by computing the field trace $Tr_{\mathbb{F}_{2^8}/\mathbb{F}_2}$ of values related to $c \cdot a^{16}$, which computes $\mathbb{F}_2$ -linear functions on the bit representation of $c \cdot a^{16}$. Using this, we recover commitments to (linear functions of) the bit decomposition which can be used to compute commitments to the inputs x of all S-boxes of the *odd* rounds, as required. In addition, squaring is $\mathbb{F}_2$ -linear, but not $\mathbb{F}_{2^8}$ -linear, so we cannot simply compute a commitment to x^2 from a commitment to x without increasing the degree of the VOLE commitment. The naive approach to compute x^2 as $x \cdot x$ would result in a degree-4 commitment to x^2 , which makes $x^2 y = x$ a degree-5 constraint and introduces additional communication and computation. To avoid this, we compute x^2 directly by applying the appropriate operations to the $\mathbb{F}_2$ -conjugates of the output of the previous layer of inversions. This essentially means that our construction performs another AES round computation on a "squared" state to derive x^2 , where constants such as the round key are likewise squared. This yields a direct degree-2 commitment to x^2 . Fig. 11 provides an overview of the AES proof.

4.2 The FAEST Algorithms

We now describe the three signature algorithms in more detail.

Key Generation. The FAEST key generation algorithm (Figure 12) selects a parameter set **param**, which determines the security parameter λ , the output length, and the OWF F as mentioned above. It iteratively samples a candidate secret key k until the two least significant bits of k are both non-zero. Once a valid k is found, a random plaintext x is sampled, then encrypted under k to compute

$$y = F_x(k).$$

The key generation algorithm outputs the secret key $\text{sk} = k$ and the public key $\text{pk} = (x, y)$. Choosing the key k as is done in our key generation algorithm implies that some public keys cannot be generated from any valid secret key.

¹⁸ We also use this check to verify the S-boxes in the key schedule.

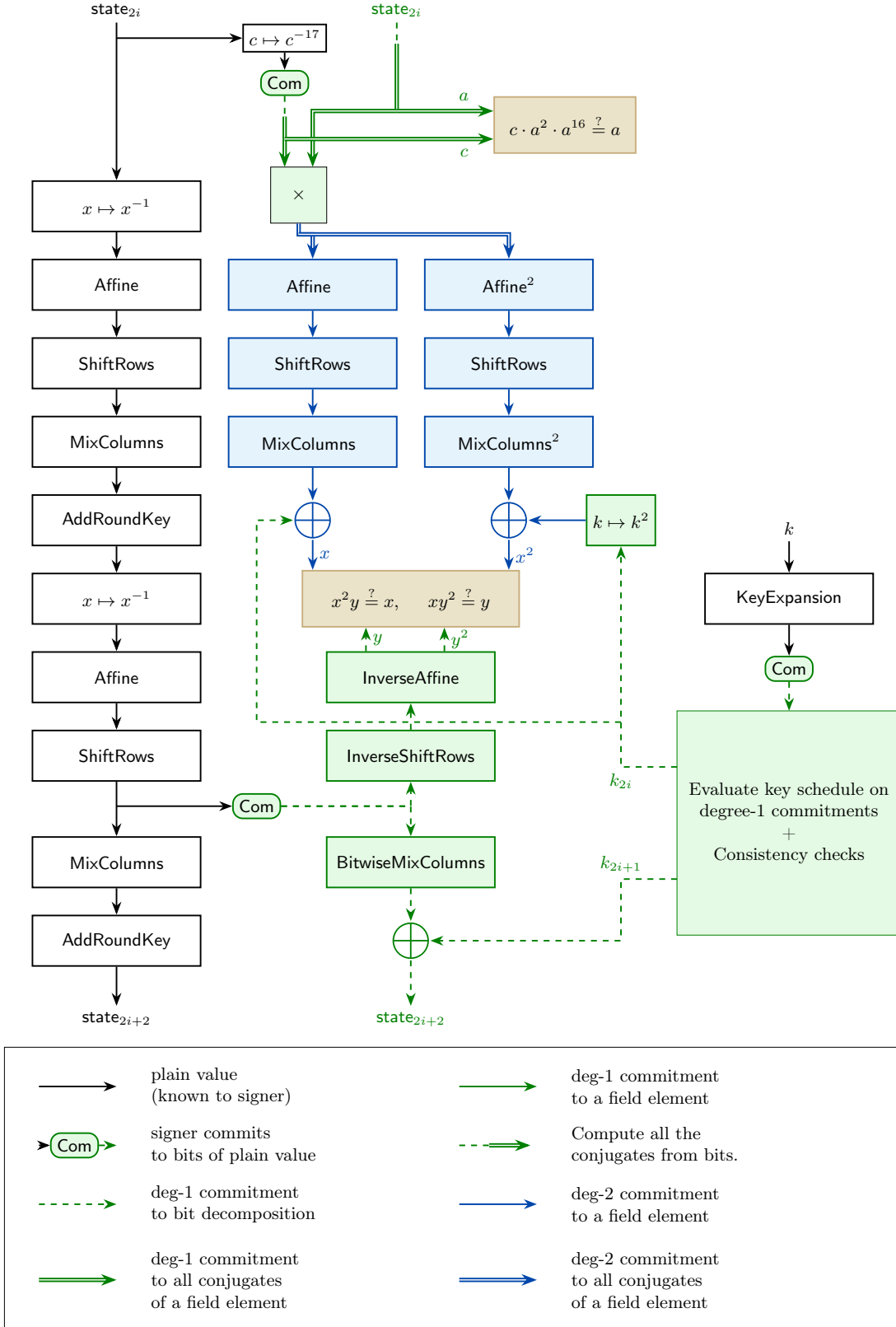


Fig. 11. A diagrammatic overview of the QuickSilver proof of knowledge of two consecutive AES rounds in FAEST. The FAEST-EM version is similar, except that the AES key is public, so the key schedule is evaluated in the clear, rather than on degree-1 VOLE commitments.

FAEST.KeyGen(param)

1. **Generate a key.**
 - Pick a random $\mathbf{k} \in \{0, 1\}^\lambda$. If $\mathbf{k}[0] \wedge \mathbf{k}[1] \neq 0$, discard it and pick a new $\mathbf{k}$.
2. **Select a public plaintext and ciphertext.**
 - Sample a random $\mathbf{x} \in \{0, 1\}^{N_{\text{st-bits}}}$, as the plaintext part of the public key.
 - Use the one-way function $F_{\mathbf{k}}$ (derived from $\mathbf{k}$) to compute $\mathbf{y} = F_{\mathbf{k}}(\mathbf{x})$.
3. **Output.** Set secret key $\text{sk} := \mathbf{k}$ and public key $\text{pk} := (\mathbf{x}, \mathbf{y})$. Output (sk, pk) .

Figure 12. FAEST KeyGen

Signing Algorithm. To sign a message msg using sk (Figure 13), the parameters param and public key pk , we first derive a unique value μ from (pk, msg) using a hash function. This value, together with the secret key sk and an optional value ρ (which can be used to randomize signature generation) is used to derive the randomness r for the VOLE commitment generation using another hash function call. This also generates iv^{pre} , which is hashed again to obtain the IV iv for the [LeafCommit](#) function. Then [VOLECommit](#)(r, iv) is called to generate the VOLE commitment com , decommitment decom , correction values $\mathbf{c}_i$, and VOLE relation $\mathbf{u}, \mathbf{V}$. Let $\mathbf{w}$ be the extended witness for the OWF proof, derived from sk .

The first message of the prover in the underlying ZK proof system is formed by $(\mu, \text{com}, \mathbf{c}_i, \text{iv}^{\text{pre}})$ and commits it to the VOLE relation. To generate a signature, the message is hashed due to the Fiat-Shamir transform to compute the challenge chall_1 . The challenge is used by the signer to show that its VOLE commitment is consistent by computing the linear hash [VOLEHash](#) keyed with chall_1 of $(\mathbf{u}, \mathbf{V})$, which produces $(\tilde{\mathbf{u}}, \tilde{\mathbf{V}})$. In addition, the signer also generates $\mathbf{d} = \mathbf{u} \oplus \mathbf{w}$ which commits it to the witness $\mathbf{w}$ in the underlying Zero-Knowledge proof. It also uses $\mathbf{d}$ to correct the VOLE commitments generated from $\mathbf{u}, \mathbf{V}$ to commitments to $\mathbf{w}$. Later, the verifier can use $\mathbf{d}$ to derandomize the random VOLE commitments to commitments to $\mathbf{w}$ as well.

In the interactive proof, $(\tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$ forms the second message, so these are hashed together with chall_1 to produce chall_2 . The signer now evaluates the AES circuit as described in the previous subsection (using an algorithm called [OWFProve](#)¹⁹) on the VOLE commitments to $\mathbf{w}$. As a result, it has to open multiple degree-3 relations that are supposedly commitments to 0. To amortize all such openings of VOLE commitments, [OWFProve](#) uses the linear [ZKHash](#) keyed by chall_2 to produce the degree-2 polynomial with coefficients $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2$. Again, these would be sent in an actual interactive proof, so the signer hashes these together with chall_2 and a counter to produce the final challenge chall_3 . Note that it will check if the last w_{grind} bits of chall_3 are 0, otherwise it increases the counter and computes the hash again.

Finally, the signer decodes chall_3 into the opening indices I and generates the decommitment decom_I . This produces the signature

$$\sigma := \left((\tilde{c}_i)_{i=1..t}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_2, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr} \right).$$

Verification Algorithm. The verification algorithm (Figure 14) takes as input the signature σ (in the same format as above) as well as pk , the message msg and the parameters param . It checks that chall_3 ends with at least w_{grind} -many 0s and computes (μ, iv) as the signing algorithm does. Next, it computes the verifier's share $\mathbf{Q}$ of the VOLE commitments by running [VOLEReconstruct](#) on decom_I , chall_3 , the correction values $\mathbf{c}_i$ and the IV iv . This also outputs com (which is identical to the value generated by the signer if decom_I was computed honestly), so the verifier can recompute chall_1 . It then applies [VOLEHash](#) keyed by chall_1 to $\mathbf{Q}$, producing the hash $\tilde{\mathbf{Q}}$. If the signer was honest and by the linearity of the hash, then

$$\tilde{\mathbf{V}} = \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda-1} \cdot \tilde{\mathbf{u}}]$$

must hold where $\text{chall}_3 = \delta_0 \parallel \dots \parallel \delta_{\lambda-1}$. This allows the verifier to recompute chall_2 . Then, the verifier corrects his VOLE commitment keys $\mathbf{Q}$ using $\mathbf{d}$ to commitment keys for the witness, and applies the AES circuit evaluation outlined above using an algorithm called [OWFVerify](#). This algorithm again hashes the commitment keys of all openings using [ZKHash](#) keyed with chall_2 . If the signer was honest, then the

¹⁹ We do not formally define this algorithm and its analogue for verification, as it is simply the circuit evaluation of the OWF based on VOLE commitments.

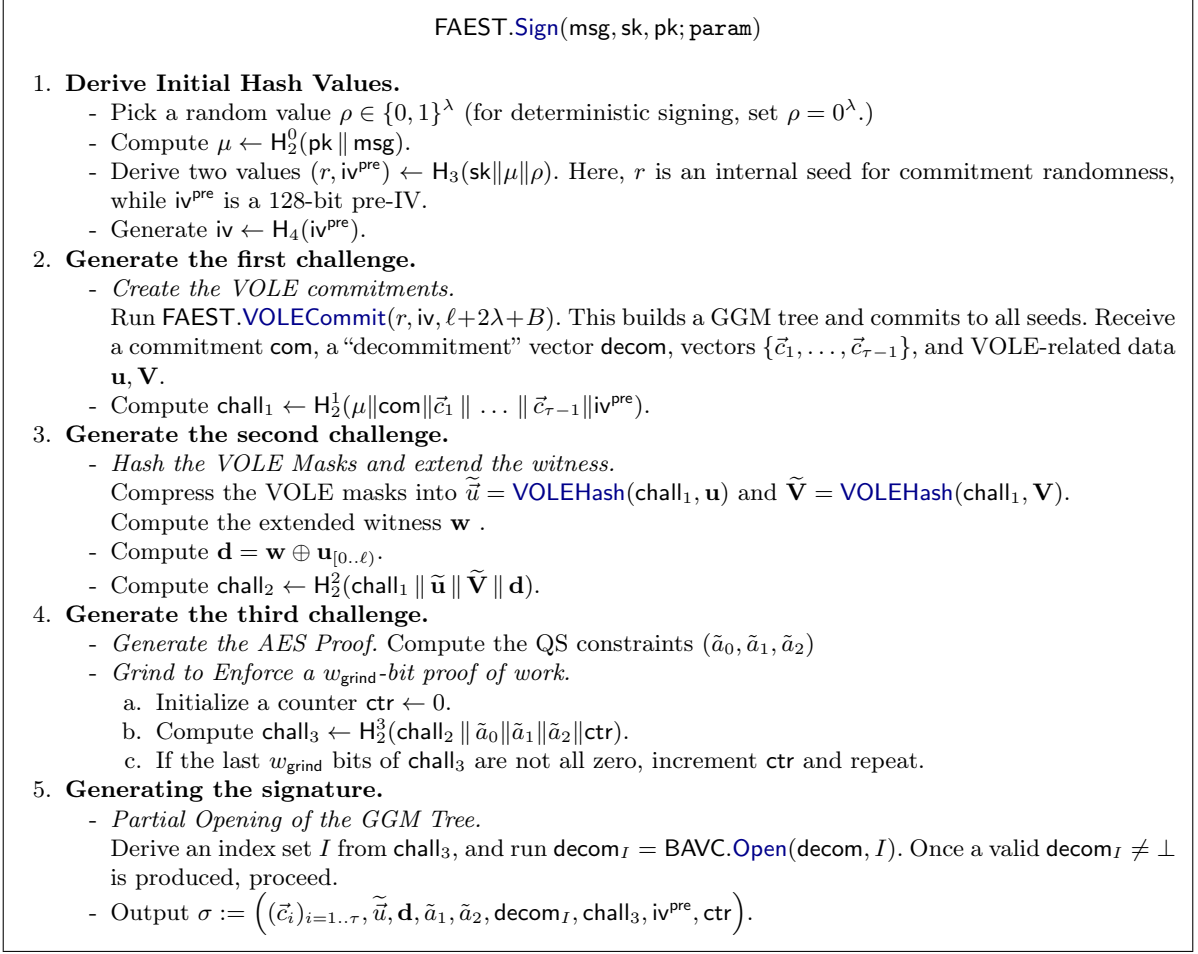


Figure 13. FAEST Signing Algorithm

output of this operation is a key that can be corrected to $\tilde{a}_0$ based on $\tilde{a}_1, \tilde{a}_2$ from the signature as well as **chall**₃. This $\tilde{a}_0$ is then used together with $\tilde{a}_1, \tilde{a}_2, \text{chall}_2$ and **ctr** to recover **chall**₃ which is compared against the **chall**₃ included in the signature.

5 EUF-CMA Security of FAEST and FAEST-EM

Building on the analysis of Section 3, we now argue security of the overall signature scheme. Our EUF-CMA proof works in the following steps:

1. We first build a reduction from EUF-CMA security (which allows an attacker to query signatures) to EUF-KO security (where forgeries must be provided given only the public key).
2. We replace the public key with a public key that, with high probability, does not form a valid OWF instance.
3. We turn the resulting EUF-KO forgeries into an attack on the interactive proof (IP) underlying FAEST. Because the OWF instance is likely invalid, the attacker must break the soundness of the proof system.
4. We turn the IP into an IOP by using an inefficient extractor to obtain the underlying VOLE correlations (which become IOP strings).
5. Finally, we show Round-by-Round (RBR) soundness of this IOP.

We first give an overview and provide the full proof in Section 5.1 to 5.3.

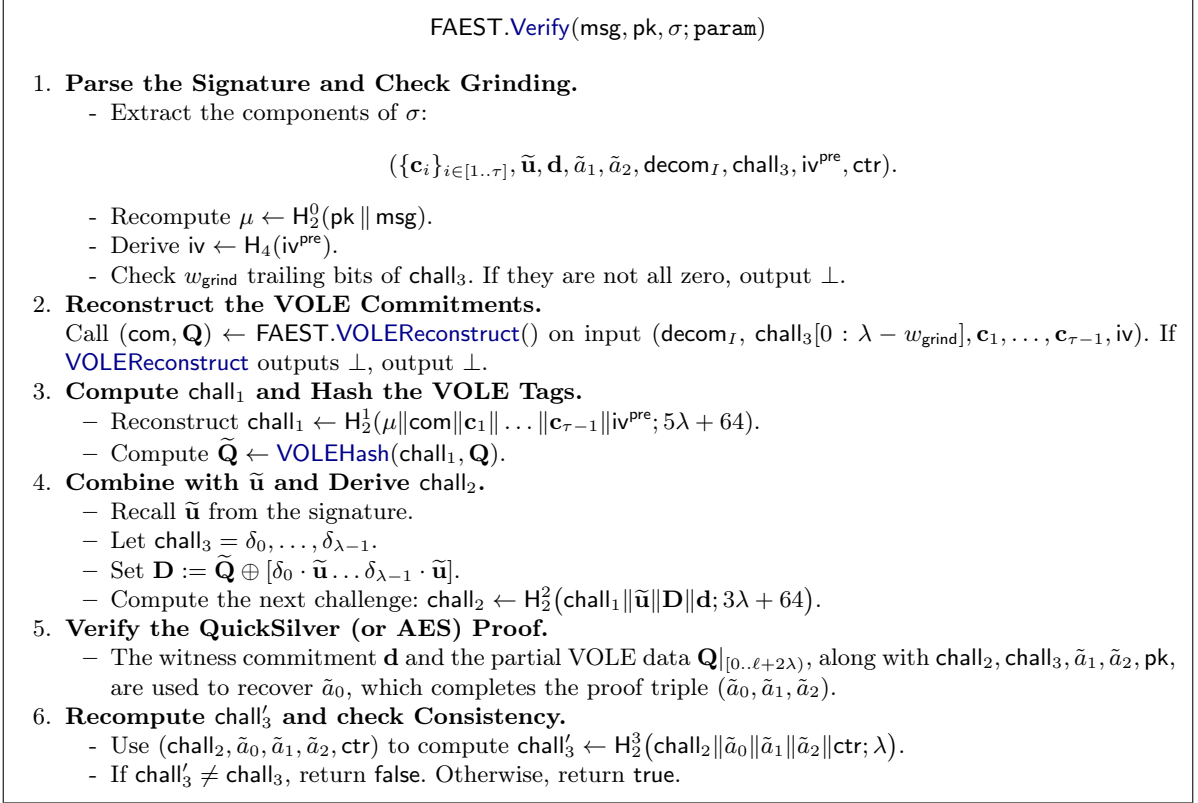


Figure 14. FAEST Verification

EUF-CMA to EUF-KO. We begin with what is a standard technique, namely emulating a signing oracle without access to the signing key by using that FAEST is an honest-verifier zero-knowledge proof. Simulating a signature can be done by running an honest-verifier zero-knowledge simulator and programming the simulated challenges into the Fiat-Shamir random oracles H_2^1, H_2^2, H_2^3 . By assumption, the resulting proofs are indistinguishable from real signatures except that programming of the random oracles may fail. We note here a technical detail in our proof, namely, that the fixed blocksize of 128 bits of AES necessitates a “divergence based” approach for hiding and simulatability, and relies on the upper bound of 2^{64} signature queries, independent of the security parameter. To address that programming of random oracles does not lead to aborts, we use the unpredictability of BAVC commitments: because the adversary will not have queried the input belonging to the programmed output beforehand if it is unpredictable, it can actually be programmed.

Aside from this, changing from EUF-KO to EUF-CMA is completely standard, and summarized in the following theorem, which is proven in Section 5.3.

Theorem 37. *Let $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as random oracles and H_3 be a PRF. Then for any PPT adversary $\mathcal{A}$ which makes $Q_{\text{sig}} \leq 2^{64}$ queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to $H_0, H_1, H_2^0, H_2^1, H_2^2$ and H_2^3 respectively,*

$$\begin{aligned}
\text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-KO}^{\text{FAEST}} \\
&+ 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \\
&+ 2 \cdot Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRG}^{\text{LeafCommit}}_{B_4}[N_0] + \text{AdvPRP}^{\text{PRG}}_{B_5}[1] \right) \\
&+ 2 \cdot Q_{\text{sig}} \cdot \left(\frac{2Q_1 + Q_{2,0} + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,1} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,2} + Q_{2,3}}{2^{3\lambda+64}} \right)
\end{aligned}$$

For FAEST-EM, the reduction and bound holds.

Moving to Unconditional Soundness. For FAEST and FAEST-EM, the key generation algorithm `KeyGen` excludes keys with bits $\mathbf{k}[0] = \mathbf{k}[1] = 1$. Thus, $\frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})} = \frac{3}{4}$ and $\left(1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}\right)^{-1} = 4$.

Each signature is a Fiat-Shamir transformed VOLE-in-the-Head proof of knowledge of this secret key $\mathbf{k}$ to the public key $\text{pk} = (\mathbf{x}, \mathbf{y})$. The proven relation asserts $\text{OWF}_{\mathbf{x}}(\mathbf{k}) = \mathbf{y}$ and $\mathbf{k}[0] \cdot \mathbf{k}[1] = 1$, and hence at least $\frac{1}{4}$ th of all $\mathbf{y}$ have no preimage.

Our reduction switches to such a pk , for which there exists no secret key and the statement is false. This allows us to reduce to *soundness instead of knowledge soundness*, permitting the use of an inefficient extractor in the security argument. We describe this approach in more detail in Section 5.1. This diverts from the original analysis of FAEST and follows an idea of Katz and Wang [KW03].

Proving EUF-KO Security. Our security reduction for EUF-KO, which we fully describe in Section 5.2, goes through several steps to eventually reduce security of the Fiat-Shamir transformed VOLE-in-the-Head proof system to the round-by-round soundness of a interactive oracle proof (IOP):

1. In Lemma 45 we convert any attacker successfully generating EUF-KO forgeries into an attacker that wins in a state restoration-sound (SRS) interactive proof security game. Note that this conversion is standard as SRS can be seen as an interactive analogue of Fiat-Shamir NIZKs. The verifier of this SRS game is constructed as an interactive version of the FAEST signature verification algorithm. To build the attacker against the SRS game, we follow a known strategy and bound the probability that calls to the Fiat-Shamir Random Oracles $H_2^0, H_2^1, H_2^2, H_2^3$ made by the EUF-KO adversary are not occurring in the order in which an honest signer performs them.
2. Since the proven statement is false, we can run the inefficient extractor of BAVC commitments from Lemma 32. This extractor directly outputs the VOLE correlations supposedly used by the prover, if they exist. Access to these VOLE correlations allows us to convert the SRS IP verifier from the previous step to an SRS IOP verifier, where the oracle is formed by the VOLE correlation. This IOP verifier performs the same verification steps (`VOLEHash`, `ZKHash`, `OWFVerify`) as the IP verifier.
3. Finally, we prove RBR soundness of the IOP verified by the verifier constructed in the previous step. This IOP verifier only performs information-theoretic checks. We use the VOLE consistency check argument of [Roy22] (using Lemma 11) for `VOLEHash`, check correctness of many VOLE commitment openings using `ZKHash` (using Lemma 12 for analysis), and the QuickSilver check [YSWW21] for correctness of circuit evaluation, similar to [BBD⁺23]. We then apply the theorem that round-by-round (RBR) soundness implies state-restoration soundness, finishing the proof.

The EUF-KO proof yields the following theorem, proven in Section 5.2:

Theorem 38. *Let $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as random oracles and (param) be parameters of the FAEST signature. Let $\mathcal{A}$ be an adversary which, for simplicity, runs `FAEST.Verify` on its purported forgery and outputs $\perp$ if verification fails. Let $Q_0, Q_1, Q_{2,i}, Q_3$ and Q_4 denote upper bounds on the number of queries of $\mathcal{A}$ to $H_0, H_1, H_2^i, H_3, H_4$, respectively, and let $Q_2 = Q_{2,1} + Q_{2,2} + Q_{2,3}$. Suppose that $\hat{\ell} \leq 2^{13}, \ell' \leq 2^{13}$ and $2 \log_2(\tau) \leq \min\{B, 50\}$. Then*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}} \leq \left(1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}\right)^{-1} \cdot (\text{AdvSnd}_{\mathcal{B}_2}^{\text{FAEST}} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}})$$

where `OWF` is the `OWF` (or more precisely, `PRG`) defined by $\text{param}_{\text{OWF}}$, and

$$\begin{aligned} \text{AdvSnd}_{\mathcal{B}_2}^{\text{FAEST}} &\leq (Q_{2,0} + Q_{2,1} + Q_{2,1} + Q_{2,3} + 4)^2 / 2^{2 \cdot \lambda} + \frac{(Q_1 + 1)^2}{2^{2 \cdot \lambda}} \\ &\quad + \text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} + (Q_{2,1} + Q_{2,1} + Q_{2,3} + 9) \cdot 3/2^\lambda \end{aligned}$$

with

- $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \leq \tau Q_0 \cdot 2^{2\lambda} \cdot \varepsilon_{\text{uhash}} \leq \tau Q_0 \cdot 2^{-\lambda}$ for FAEST.
- $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \leq (Q_{\text{AES}} + 1)^2 \cdot 2^{-2\lambda}$ for FAEST-EM.

The difference in FAEST and FAEST-EM for the security proof lies in the extractability assumption needed for (leaf) commitments (and the choice of `OWF`). For FAEST, our construction is statistically secure (in the random oracle model), but this requires a universal hash with 3λ output length per leaf commit. For FAEST-EM, we make a stronger assumption (Definition 27) on $\text{PRG}_{2\lambda}$, which allows us to keep the 2λ output length, which is minimal by the birthday bound.

5.1 Switching to False Statements

Theorem 39 (FAEST is EUF-KO). *Let H_0, H_1 and $H_2^0, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as random oracles and $(\text{param}, \text{param}_{\text{OWF}}, \text{param}_{\text{VOLF}})$ be parameters of the FAEST signature. Let $\mathcal{A}$ be an adversary which, for simplicity, runs FAEST.Verify on its purported forgery and outputs $\perp$ if verification fails. Let $Q_0, Q_1, Q_{2,i}, Q_3$ and Q_4 denote upper bounds on the number of queries of $\mathcal{A}$ to $H_0, H_1, H_2^i, H_3, H_4$, respectively, and let $Q_2 = Q_{2,1} + Q_{2,2} + Q_{2,3}$. Suppose that $\ell \leq 2^{13}, \ell' \leq 2^{13}$ and $2 \log_2(\tau) \leq \min\{B, 50\}$. Then*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}} \leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot (\text{AdvSnd}_{\mathcal{B}_2}^{\text{FAEST}} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}})$$

where OWF is the OWF (or more precisely, PRG) defined by $\text{param}_{\text{OWF}}$, and

$$\begin{aligned} \text{AdvSnd}_{\mathcal{B}_2}^{\text{FAEST}} &\leq (Q_{2,0} + Q_{2,1} + Q_{2,1} + Q_{2,3} + 4)^2 / 2^{2 \cdot \lambda} \\ &\quad + \frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \\ &\quad + (Q_{2,1} + Q_{2,1} + Q_{2,3} + 9) \cdot 3 / 2^\lambda \end{aligned}$$

with

- $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \leq \tau Q_0 \cdot 2^{2\lambda} \cdot \varepsilon_{\text{uhash}} \leq \tau Q_0 \cdot 2^{-\lambda}$ for FAEST.
- $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \leq (Q_{\text{AES}} + 1)^2 \cdot 2^{-2\lambda}$ for FAEST-EM.

Remark 40. As in Lemma 32, the assumption that $\mathcal{A}$ checks that the forgery verifies, ensures that the game does not query undefined values of any random oracle for the forgery verification. For a general $\mathcal{A}$, we can always replace it by an $\mathcal{A}'$ where the check is added. This modification increases the number of random oracle queries to at most $Q_0 + 1, Q_1 + \tau + 1, Q_{2,1} + 1, Q_{2,2} + 1, Q_{2,3} + 1$ for $H_0, H_1, H_2^1, H_2^2, H_2^3$ respectively. Since $Q_0, Q_1, Q_2 \gg \tau$, the effect on concrete security is minimal.

To show EUF-KO security, we need the following technical fact.

Lemma 41. *Let $F: \mathcal{K} \rightarrow \mathcal{Y}$ be a PRG and let $\mathcal{A}$ be any (potentially unbounded) distinguisher, i.e., some predicate $\mathcal{A}: \mathcal{Y} \rightarrow \{0, 1\}$. Then*

$$\Pr_{y \leftarrow \text{im}(F)}[\mathcal{A}(y) = 1] \leq \frac{|\text{dom}(F)|}{|\text{im}(F)|} \cdot \Pr_{x \leftarrow \text{dom}(F)}[\mathcal{A}(F(x)) = 1]$$

At a high level, Lemma 41 relates the uniform distribution over the image of the PRG F and its image distribution, and allows us to switch from the former to the latter.

Proof. Since any predicate $\mathcal{A}$ defines a set $E = \{\mathcal{A}(y) = 1\} \subseteq \mathcal{Y}$, the claim will follow from ∞ -divergence (Lemma 21) of the probability distributions I and J over $\mathcal{Y}$, where I is the uniform distribution over $\text{im}(F)$ and J is the distribution given by $F(x)$ for uniform $x \leftarrow \mathcal{X}$. Clearly, if $\Pr[I = y] = 0$ then also $\Pr[J = y] = 0$, hence $\Pr[I = y] / \Pr[J = y] \leq \infty$. Moreover, for $y \in \text{supp}(I)$, we have $\Pr[I = y] = \frac{1}{|\text{im}(F)|}$ and $\Pr[J = y] \geq \frac{1}{|\text{dom}(F)|}$. Thus

$$\frac{\Pr[I = y]}{\Pr[J = y]} \leq \frac{|\text{dom}(F)|}{|\text{im}(F)|}$$

and the claim follows by Lemma 21. $\square$

Proof of Theorem 39. We argue via game hops. We prove the result for FAEST first, and then explain what changes for FAEST-EM. We write G_i for the output of the i -th game.

Game G_0 : This is the real EUF-KO security game. As in the EUF-KO game, we let sk, pk be the generated challenge secret and public key, respectively. We have

$$\Pr[G_0 = 1] = \text{AdvEUFKO}_{\mathcal{A}}^{\text{FAEST}}$$

Game G_1 : In this game, we change $\mathbf{pk} := (\mathbf{x}, \mathbf{y})$ from honestly generated to truly random, i.e., to $\mathbf{x} \leftarrow \{0, 1\}^{N_{\text{st-bits}}}$, $\mathbf{y} \leftarrow \{0, 1\}^{N_{\text{st-bits}}}$. By a straightforward reduction to PRG security, we obtain an adversary whose running time is roughly that of the EUF-KO game. We have

$$|\Pr[G_1 = 1] - \Pr[G_0 = 1]| \leq \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}}[1]$$

as there is a straightforward reduction to a distinguisher $\mathcal{B}_1$ which gets a single sample $\mathbf{y}$ that is either the OWF-output $\mathbf{y} = \text{OWF}_{\mathbf{x}}(\mathbf{k}) := F_{\mathbf{k}}(\mathbf{x})$ (for secret $\mathbf{k} \leftarrow \{0, 1\}^\lambda$ and public parameters $\mathbf{x} \leftarrow \{0, 1\}^{N_{\text{st-bits}}}$) or a random $\mathbf{y} \leftarrow \{0, 1\}^{N_{\text{st-bits}}}$.

Splitting Game G_1 : Now, we derive a bound on the probability $\Pr[G_1 = 1]$ by distinguishing two events: The event I , that $\mathbf{y} \in \text{im}(\text{OWF}_{\mathbf{x}})$, i.e., $(\mathbf{x}, \mathbf{y})$ is a valid public key, and the negated event $\neg I$, where there exists no key such that $F_{\mathbf{k}}(\mathbf{x}) = \mathbf{y}$.

Define G_1^* as G_1 , except that it returns 0 if $\mathbf{pk}$ is valid, i.e., $\mathbf{pk} \in \mathcal{L}_{\mathcal{R}}$ is language of public keys. Observe that

$$p = \Pr[G_1^* = 1] = \Pr[G_1 = 1 \wedge \neg I]$$

and G_1^* is exactly the experiment in Corollary 50 and p the respective success probability. It follows that

$$\Pr[G_1 = 1 \mid \neg I] = \frac{\Pr[G_1^* = 1]}{\Pr[\neg I]},$$

by conditional probability and since the game outputs 0 if $\mathbf{pk} \in \mathcal{L}$, i.e., if I occurs. On the other hand, we have from Lemma 41

$$\Pr[G_1 = 1 \mid I] \leq \frac{|\text{dom}(\text{OWF})|}{|\text{im}(\text{OWF}_{\mathbf{x}})|} \cdot \Pr[G_0 = 1]$$

the left hand side is G_1 with a uniformly random image $\mathbf{y}$ of $\text{OWF}_{\mathbf{x}}$ and the right hand side is the distribution of $\mathbf{y}$ under $\text{OWF}_{\mathbf{x}}$. Note that we omit $\mathbf{x}$ from domain and codomain of OWF , since these are independent of the parameter $\mathbf{x}$. Thus, we derive

$$\begin{aligned} \Pr[G_1 = 1] &= \Pr[I] \cdot \Pr[G_1 = 1 \mid I] + \Pr[\neg I] \cdot \Pr[G_1 = 1 \mid \neg I] \\ &\leq \Pr[I] \cdot \frac{|\text{dom}(\text{OWF})|}{|\text{im}(\text{OWF}_{\mathbf{x}})|} \Pr[G_0 = 1] + \Pr[\neg I] \cdot \frac{\Pr[G_1^* = 1]}{\Pr[\neg I]} \\ &\leq \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|} \Pr[G_0 = 1] + \Pr[G_1^* = 1] \end{aligned}$$

where the last step uses $\Pr[I] = \frac{|\text{im}(\text{OWF}_{\mathbf{x}})|}{|\text{cod}(\text{OWF})|}$. (This holds, since a uniformly random $\mathbf{pk} \leftarrow \text{cod}(\text{OWF})$ lies in $\text{im}(\text{OWF})$ with probability $\frac{|\text{im}(\text{OWF}_{\mathbf{x}})|}{|\text{cod}(\text{OWF})|}$.) By the first game hop, we have

$$\Pr[G_0 = 1] \leq \Pr[G_1 = 1] + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}} \leq \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|} \Pr[G_0 = 1] + \Pr[G_1^* = 1] + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}}$$

and resolving for $\Pr[G_0 = 1]$ yields

$$\Pr[G_0 = 1] \leq \frac{1}{1 - \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}} \cdot (\Pr[G_1^* = 1] + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}}) \quad (7)$$

which yields the claim after plugging in the advantage bound for $\Pr[G_1^*]$ from Corollary 50. $\square$

5.2 EUF-KO

Interactive Proof Induced by FAEST. We present the IP derived from the FAEST signature scheme. For $k \in \mathbb{N}$, let denote the set of $\mathcal{UF}(k)$ of functions from $\{0, 1\}^*$ to $\{0, 1\}^k$, and sampling from $\mathcal{UF}(k)$ is implemented via lazy sampling.

$\text{Exp}_{\text{IP}}^{\text{sr}}((s_i)_{i \in [k]}, \text{rnd}, \tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}})$

```

1:  $H \leftarrow \mathcal{UF}(2\lambda)$ 
2: while  $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$  doesn't exit:
3:    $\tilde{\mathcal{P}}_{\text{IP}}^{\text{srH}}$  outputs  $(\mathbb{x}, (\alpha_1, \dots, \alpha_i), (\text{st}_1, \dots, \text{st}_i))$  where  $\text{st}_i \in \{0, 1\}^{s_i}$ 
4:   Set  $\rho_i \leftarrow \text{rnd}_i(\mathbb{x}, (\alpha_1, \dots, \alpha_i), (\text{st}_1, \dots, \text{st}_i))$ 
5:   Give  $\rho_i$  to  $\tilde{\mathcal{P}}_{\text{IP}}^{\text{srH}}$ 
6:    $\tilde{\mathcal{P}}_{\text{IP}}^{\text{srH}}$  exits with output  $(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]})$ 
7:   for  $i \in [k]$ , set  $\rho_i \leftarrow \text{rnd}_i(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]})$ 
8:   return  $(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$ 

```

Fig. 15. The state-restoration game for IPs.

$\mathcal{V}_{\text{IP}}^{\text{H}_0, \text{H}_1, \text{H}_4}(\mathbb{x}, (\alpha_1, \dots, \alpha_4), (\rho_1, \dots, \rho_4))$

1. **Parse the input and check grinding.**
 - Write $\mathbb{x} = (\text{pk}, \text{param}, \text{param}_{\text{OWF}})$, $\alpha_1 = (\text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}})$, $\alpha_2 = (\tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d})$, $\alpha_3 = (\tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2)$, $\alpha_4 = (\text{decom}_I)$.
 - Set $\text{chall}_1 := \rho_1, \text{chall}_2 := \rho_2, \text{chall}_3 := \rho_3$.
 - Set $(\delta_0, \dots, \delta_{\lambda-1}) := \text{chall}_3$ and $\text{iv} \leftarrow \text{H}_4(\text{iv}^{\text{pre}})$.
 - Check w_{grind} trailing bits of chall_3 . If they are not all zero, return false.
2. **Reconstruct the VOLE Commitments.**
 Compute $(\widehat{\text{com}}, \mathbf{Q}) \leftarrow \text{VOLEReconstruct}(\text{decom}_I, \text{chall}_3[0 : \lambda - w_{\text{grind}}], \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv})$. If **VOLEReconstruct** outputs $\perp$ or $\widehat{\text{com}} \neq \text{com}$, return false.
3. **Hash the VOLE Tags.**
 Compute $\tilde{\mathbf{Q}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{Q})$.
4. **Combine with $\tilde{\mathbf{u}}$.**
 If $\mathbf{D} \neq \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda-1} \cdot \tilde{\mathbf{u}}]$ then return false.
5. **Verify the QuickSilver (or AES) Proof.**
 The witness commitment $\mathbf{d}$ and the partial VOLE data $\mathbf{Q}|_{[0..\ell+2\lambda]}$, along with $\text{chall}_2, \text{chall}_3, \tilde{a}_1, \tilde{a}_2, \text{pk}$, are used to compute $\hat{a}_0$.
6. **Check Consistency.**
 If $\hat{a}_0 = \tilde{a}_0$ return true, otherwise false.

Figure 16. The FAEST interactive proof verifier.

Definition 42 ([CY24, Definition 13.2.1]). The state-restoration experiment for $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$, in the random oracle model, with salt sizes $s_i \in \mathbb{N}$, randomness functions $\text{rnd} = (\text{rnd}_i)_{i \in [k]} \in (\mathcal{UF}(r_i))_{i \in [k]}$, and IP state-restoration prover $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ is defined in Fig. 15. We say that $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ is t -move if it exits the **while** loop after at most t iterations.

Definition 43 ([CY24, Definition 13.2.2]). $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ has state-restoration soundness error $\epsilon_{\text{IP}}^{\text{sr}}$ if for every salt sizes $s_i \in \mathbb{N}$, move budget $k \in \mathbb{N}$, k -round malicious IP state-restoration prover $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$,

$$\Pr \left[\begin{array}{l} \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathcal{V}_{\text{IP}}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1 \end{array} \middle| \begin{array}{l} \text{rnd} = (\text{rnd}_i)_{i \in [k]} \leftarrow (\mathcal{UF}(r_i))_{i \in [k]} \\ (\mathbb{x}, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \leftarrow \\ \text{Exp}_{\text{IP}}^{\text{sr}}((s_i)_{i \in [k]}, \text{r}, \tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}) \end{array} \right] \leq \epsilon_{\text{IP}}^{\text{sr}}((s_i)_{i \in [k]}, k)$$

In Figure 16 we define the IP verifier $\mathcal{V}_{\text{IP}}$ with $k = 4$ for FAEST explicitly. Note that, when interacting with any honest prover, the protocol running with $\mathcal{V}_{\text{IP}}$ has a very large completeness loss. This is because chall_3 must be zero in the last w_{grind} bits, which is only true with small probability in the honest SRS experiment from Definition 42. This is, however, of no importance for the proof, which only considers a reduction to the state-restoration soundness of the FAEST IP.

From EUF-KO to SR Soundness. We begin with a proposition that we need in the construction of the reduction:

Proposition 44 (Random oracle list game). Let $n \in \mathbb{N}$ and for $i \in \{0, \dots, n+1\}$ let $\mathcal{Y}_i, \mathcal{Y}'_i \subset \{0, 1\}^*$ be finite sets. For each $i \in \{0, \dots, n\}$ let $H_i: \mathcal{Y}_i \times \mathcal{Y}'_i \rightarrow \mathcal{Y}_{i+1}$ be a random oracle. Define $Y = \min_{i=1}^n |\mathcal{Y}_i|$.

Consider the following game with an adversary $\mathcal{B}$: For $i \in [1..n]$ the game keeps track of initially empty lists $L_i \subset \mathcal{Y}_i$ and proceeds as follows: For $i \in \{0, \dots, n\}$ the adversary can (repeatedly) query each H_i on an input x, x' , producing an output y :

1. When querying $y = H_i(x, x')$, if $y \in L_{i+1}$ then the adversary wins.
2. Thereafter if $i \geq 1$ then add x to L_i .

Let $\mathcal{B}$ be an adversary which makes at most Q queries to the oracles $H_0, \dots, H_n$. Then the probability that $\mathcal{B}$ wins is bounded by

$$Q^2 / (2 \cdot Y).$$

Proof of Proposition 44. Clearly, $\mathcal{B}$ can only win by querying $H_0, \dots, H_{n-1}$. At the first query, the adversary cannot win because each L_i is empty. At the second query, it can win with probability at most $\max_{i=1}^n \frac{1}{|\mathcal{Y}_i|} \leq 1/Y$ if the previous query was to $H_1, \dots, H_n$. This is because the output of the oracle to which the query is now made is uniformly random in the respective $\mathcal{Y}_i$ and will match the single element on a list (if it exists) with probability $1/|\mathcal{Y}_i|$. At the third query, this probability is at most $\max_{i=1}^n \frac{2}{|\mathcal{Y}_i|} \leq 2/Y$ as the chance is maximal if both previous queries were to the same H_i . By a union bound, $\mathcal{B}$ will win during any of the queries with probability at most

$$\sum_{i=1}^Q \frac{i-1}{Y} \leq \frac{Q^2}{2 \cdot Y}. \quad \square$$

We now prove that any attacker against the EUF-KO security of FAEST can be used to construct an attacker against an SRS IP with verifier $\mathcal{V}_{\text{IP}}$.

Lemma 45. Let $\mathcal{R}$ be the relation such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ if $\mathbf{x}$ is a public key and $\mathbf{w}$ the corresponding signing key for the FAEST signature scheme (with parameters $\text{param}, \text{param}_{\text{OWF}}, \text{param}_{\text{VOLE}}$ which will be left implicit). Let $\mathcal{A}$ be an algorithm with access to random oracles $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_3, H_4$ playing the EUF-KO game of Definition 2 with random public key $\mathbf{pk} = (\mathbf{x}, \mathbf{y}) \leftarrow \{0, 1\}^{2\lambda}$. Suppose $\mathcal{A}$ wins the EUF-KO game for $\mathbf{pk} \notin \mathcal{L}_{\mathcal{R}}$ with probability

$$p = \Pr[\text{ExpEUF}_{\mathcal{A}}^{\text{SIG}}(\mathbf{pk}) = 1 \wedge \mathbf{pk} \notin \mathcal{L}_{\mathcal{R}} \mid \mathbf{pk} \leftarrow \{0, 1\}^{2\lambda}].$$

Suppose further that $\mathcal{A}$ makes at most $Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}$ queries to each of the random oracles $H_2^0, H_2^1, H_2^2, H_2^3$, respectively. Then there exists a 4-round probabilistic algorithm $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ (and sizes $s_1, \dots, s_4$) for the relation $\mathcal{R}$ with access to the random oracles $H_0, H_1, H_2^0, H_3, H_4$ that wins the state-restoration game of Definition 43 with probability

$$\epsilon_{(\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}, \mathcal{V}_{\text{IP}})}^{\text{sr}}((s_i)_{i \in [4]}, 4) \geq p - (Q_{2,0} + Q_{2,1} + Q_{2,1} + Q_{2,3} + 4)^2 / 2^{2 \cdot \lambda}$$

against $\mathcal{V}_{\text{IP}}$. Moreover, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ makes at most $Q_{2,1} + Q_{2,1} + Q_{2,3} + 4$ queries to the SRS game and its running time is roughly that of $\mathcal{A}$.

Proof. We define a malicious prover $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ against the state-restoration soundness based on $\mathcal{A}$. Note that $\mathcal{A}$ is non-interactive and interacts with the random oracles $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_3, H_4$ during the EUF-KO game. $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ is defined for an interactive game, where the messages from the SRS experiment will replace the responses from H_2^1, H_2^2, H_2^3 .

Initially, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ samples $\mathbf{pk} \leftarrow \{0, 1\}^{2\lambda}$, just as in EUF-KO with random public key. Then, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ starts $\mathcal{A}$ with the $\mathbf{pk}$ as well as $\text{param}, \text{param}_{\text{OWF}}, \text{param}_{\text{VOLE}}$ as inputs. Further, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ runs the EUF-KO game with $\mathcal{A}$ and simulates access to the random oracles. Whenever $\mathcal{A}$ makes a query to any of $H_0, H_1, H_2^0, H_3, H_4$, the reduction $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ observes it and forwards it to a random oracles it has access to itself, passing the response back to $\mathcal{A}$. For queries to the random oracles H_2^1, H_2^2, H_2^3 , the reduction $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ runs the following *RO simulation*:

- When $\mathcal{A}$ makes a query $\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}$ to H_2^1 ,

1. $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ checks if μ was previously returned to $\mathcal{A}$ as a response to a query to H_2^0 ; if not, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ returns a randomly sampled chall_1 to $\mathcal{A}$ and puts μ on the bad list.
2. Otherwise, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ recovers the pk and msg that were queried to H_2^0 to create an instance $\mathbb{x}$, and outputs $(\mathbb{x}, (\alpha_1), (\text{st}_1))$ to $\text{Exp}_{\text{IP}}^{\text{sr}}$, where $\alpha_1 = \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}$ and $\text{st}_1 = \mu$ serves as salt. It receives ρ_1 from $\text{Exp}_{\text{IP}}^{\text{sr}}$ and passes $\text{chall}_1 = \rho_1$ to $\mathcal{A}$ as the response from H_2^1 .
- When $\mathcal{A}$ makes a query $\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d}$ to H_2^2 ,
 1. $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ checks if chall_1 was previously returned to $\mathcal{A}$ as a response from H_2^1 ; if not, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ returns a randomly sampled chall_2 to $\mathcal{A}$ and puts chall_1 on the bad list.
 2. Otherwise, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ recovers the partial transcript $(\mathbb{x}, (\alpha_1), (\text{st}_1))$ that received $\rho_1 = \text{chall}_1$ as response, and it outputs $(\mathbb{x}, (\alpha_1, \alpha_2), (\text{st}_1, \text{st}_2))$ to $\text{Exp}_{\text{IP}}^{\text{sr}}$, where $\alpha_2 = \tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d}$ and $\text{st}_2 = \text{chall}_1$ serves as salt. It receives ρ_2 from $\text{Exp}_{\text{IP}}^{\text{sr}}$ and returns $\text{chall}_2 = \rho_2$ to $\mathcal{A}$ as the response from H_2^2 .
- When $\mathcal{A}$ makes a query $\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}$ to H_2^3 ,
 1. $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ checks if chall_2 was previously returned to $\mathcal{A}$ as a response from H_2^2 ; if not, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ returns a randomly sampled chall_3 to $\mathcal{A}$ and puts chall_2 on the bad list.
 2. Otherwise, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ recovers the partial transcript $(\mathbb{x}, (\alpha_1, \alpha_2), (\text{st}_1, \text{st}_2))$ that received $\rho_2 = \text{chall}_2$ as response, and it outputs $(\mathbb{x}, (\alpha_1, \alpha_2, \alpha_3), (\text{st}_1, \text{st}_2, \text{st}_3))$, where $\alpha_3 = \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2$ and $\text{st}_3 = (\text{chall}_2, \text{ctr})$. It receives $\rho_3 \in \{0, 1\}^\lambda$ from $\text{Exp}_{\text{IP}}^{\text{sr}}$ returns $\text{chall}_3 = \rho_3$ to $\mathcal{A}$ as the response from H_2^3 .

At the end of the EUF-KO game, $\mathcal{A}$ terminates with $\text{msg}, \sigma = ((\mathbf{c}_i)_{i \in [1.. \tau)}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_2, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$. $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$, upon receiving this, runs $\text{FAEST.Verify}(\text{msg}, \text{pk}, \sigma; \text{param}, \text{param}_{\text{OWF}})$ to check if the signature σ is valid. As FAEST.Verify interacts again with the random oracles $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_3, H_4$ the algorithm $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ will, as before, forward queries to $H_0, H_1, H_2^0, H_3, H_4$ to the random oracles that it has access to. For queries to H_2^1, H_2^2, H_2^3 it does the following:

- When Verify makes the query $\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}$ to H_2^1 , $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ checks if μ was put on the bad list during *RO simulation*. If yes, then $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ aborts. Otherwise it outputs $(\mathbb{x}, (\alpha_1), (\text{st}_1))$ to $\text{Exp}_{\text{IP}}^{\text{sr}}$, where $\alpha_1 = \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}$ and $\text{st}_1 = \mu$ serves as salt. It receives ρ_1 from $\text{Exp}_{\text{IP}}^{\text{sr}}$ and passes $\text{chall}_1 = \rho_1$ to Verify as the response from H_2^1 .
- When Verify makes the query $\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d}$ to H_2^2 , $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ checks if chall_1 was put on the bad list during *RO simulation*. If yes, then $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ aborts. Otherwise it outputs $(\mathbb{x}, (\alpha_1, \alpha_2), (\text{st}_1, \text{st}_2))$ to $\text{Exp}_{\text{IP}}^{\text{sr}}$, where $\alpha_2 = \tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d}$ and $\text{st}_2 = \text{chall}_1$ serves as salt. It receives ρ_2 from $\text{Exp}_{\text{IP}}^{\text{sr}}$ and returns $\text{chall}_2 = \rho_2$ to Verify as the response from H_2^2 .
- When Verify makes the query $\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}$ to H_2^3 , $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ checks if chall_2 was put on the bad list during *RO simulation*. If yes, then $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ aborts. Otherwise it outputs $(\mathbb{x}, (\alpha_1, \alpha_2, \alpha_3), (\text{st}_1, \text{st}_2, \text{st}_3))$, where $\alpha_3 = \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2$ and $\text{st}_3 = (\text{chall}_2, \text{ctr})$. It receives $\rho_3 \in \{0, 1\}^\lambda$ from $\text{Exp}_{\text{IP}}^{\text{sr}}$ returns $\text{chall}_3 = \rho_3$ to Verify as the response from H_2^3 .

By construction, the outputs of the simulated ROs during Verify are consistent with those of the simulated ROs during the EUF-KO game with $\mathcal{A}$.

If FAEST.Verify rejects the signature, then $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ aborts. Else, define $\alpha_1 = (\text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}})$, $\alpha_2 = (\tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d})$, $\alpha_3 = (\tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2)$ and $\alpha_4 = (\text{decom}_I)$. Then exit the while loop in $\text{Exp}_{\text{IP}}^{\text{sr}}$ and output $(\text{pk}, (\alpha_1, \alpha_2, \alpha_3, \alpha_4), (\mu, \text{chall}_1, \text{chall}_2 \parallel \text{ctr}, \perp))$ to the experiment.

Note that the outputs of the ROs H_2^1, H_2^2, H_2^3 which $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ simulates towards $\mathcal{A}$ are identically distributed to those of the outputs of the random oracles in the EUF-KO game. This follows from Definition 42, which replaces the random oracles with outputs of randomness functions (that map each input to a uniformly random output, just like a random oracle) and because RO outputs are chosen uniformly at random in the bad list case as well. Thus, running $\mathcal{A}$ inside $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ does not change the probability of $\mathcal{A}$ outputting a valid forgery when compared with the EUF-KO security experiment (for any choice of pk). Also, note that $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ never aborts during the interaction with $\mathcal{A}$ that creates the forgery but potentially after obtaining the forgery attempt from $\mathcal{A}$.

Assume that $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ outputs a value to the experiment $\text{Exp}_{\text{IP}}^{\text{sr}}$ without aborting. By construction, Verify must have accepted the signature generated by $\mathcal{A}$. $\mathcal{V}_{\text{IP}}$ runs the same algorithms to verify the messages as FAEST.Verify and equally checks that the same w_{grind} bits of chall_3 are 0. It also uses the exact same random challenges as Verify due to the simulation of the RO responses by $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ during Verify .

The differences in verification between both algorithms are the following:

$\mathcal{V}_{\text{IP}}$ compares the output $\hat{a}_0$ of `OWFVerify` with the $\tilde{a}_0$ contained in the input message α_3 . It also compares the output $\widetilde{\text{com}}$ of `VOLEReconstruct` with the value com contained in the input α_1 ; while `Verify` hashes the output $\tilde{a}_0$ of `OWFVerify` (and other messages) using H_3^2 to generate chall'_3 , which it compares with chall_3 . It also hashes the output com of `VOLEReconstruct` using H_2^1 to generate chall_1 .

All the inputs $\mathbf{d}, \mathbf{Q}, \tilde{a}_1, \tilde{a}_2, \text{pk}; \text{param}, \text{param}_{\text{OWF}}$ to `OWFVerify` are identical in `Verify`, $\mathcal{V}_{\text{IP}}$ as both use the same challenges. Since `Verify` accepts, we have that the value $\tilde{a}_0$ which $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ generates from `OWFVerify` (using the challenges $\text{chall}_2, \text{chall}_3$) and outputs to the experiment $\text{Exp}_{\text{IP}}^{\text{sr}}$ is identical to the RO input to H_3^2 that `Verify` makes to generate $\text{chall}'_3 = \text{chall}_3$. As $\mathcal{V}_{\text{IP}}$ uses the same challenges as `Verify`, the output $\hat{a}_0$ of `OWFVerify` in $\mathcal{V}_{\text{IP}}$ must therefore be identical to the $\tilde{a}_0$ it obtained as input. A similar argument can be made for $\widetilde{\text{com}}$: there, all inputs except chall_3 to `VOLEReconstruct` are fixed. As the signature verifies, `Verify` used the challenge chall_3 for verification, which made it output com that is placed in α_1 . But since $\mathcal{V}_{\text{IP}}$ uses the same inputs to `VOLEReconstruct`, its output $\widetilde{\text{com}}$ will be identical to com . We conclude that, if $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ does not abort the experiment, then, unless $\text{pk} \in \mathcal{L}_{\mathcal{R}}$, it will win the experiment with $\mathcal{V}_{\text{IP}}$ due to consistency of RO simulation with the outputs of the random functions of the SRS experiment.

$\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ aborts the experiment if it previously marked an input to a RO as bad during the simulation of ROs towards $\mathcal{A}$, or if `Verify` aborts. Aborting in the latter case does not decrease the success probability of $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ so we only analyze the former. We invoke Proposition 44: if an input query $x = \mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}$ to H_2^1 matches a μ that was added to the bad list, then μ was not output by H_2^0 before (the same argument follows identically for H_2^2, H_2^3). If $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ aborts due to the bad list event for H_2^1 during `Verify`, then $\mathcal{A}$ must have output a forgery tuple msg, σ (for a predefined pk) such that $H_2^0(\text{pk} \parallel \text{msg}) = \mu$ and where $x = \mu \parallel \dots$ was queried to H_2^1 before $\text{pk} \parallel \text{msg}$ was queried to H_2^0 (if it was not queried by $\mathcal{A}$ during the EUF-KO game it will be queried by `Verify`). By modeling $H_2^0, \dots, H_2^3$ as the random oracles in Proposition 44, the abort probability due to this event can be at most $(Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2 / 2^{2\lambda}$. Here we count the RO queries made by $\mathcal{A}$ and the additional 4 that are possibly made during `Verify`. Since Y is set to be the size of the minimum of the output spaces of H_2^0, H_2^1, H_2^2 we have that $Y = 2^{2\lambda}$ as $\mu \in \{0, 1\}^{2\cdot\lambda}$ is the smallest response of these ROs.

Taken, the probability, that $\text{pk} \notin \mathcal{L}_{\mathcal{R}}$ and $\mathcal{A}$ outputs a valid forgery for a uniformly random pk , is exactly p . Thus, the probability that $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ wins the SRS experiment is at least

$$p - (Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2 / 2^{2\lambda}$$

which concludes the proof. $\square$

Note that the constructed $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ runs $\mathcal{A}$ once and forwards requests to random oracles. In particular, if $\mathcal{A}$ is a PPT algorithm, then so is $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$. $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ also ignores potential preimage and collision attacks on H_4 that allow $\mathcal{A}$ to find signatures that are identical except for differing iv^{pre} . This is out of scope as we do not claim to achieve strong unforgeability.

From SRS IP to SRS IOP. We now define a simplified notion of an IOP (following [BBD⁺23]), and will present a version of $\mathcal{V}_{\text{IP}}$ that works as an IOP by idealizing the BAVC. We then show a reduction from SRS IP to SRS IOP.

Definition 46 (Interactive Oracle Proof (IOP)). A (public-coin) interactive oracle proof (IOP) [BCS16, RRR16] for NP-relation $\mathcal{R}$ in the random oracle model is a pair of PPT ITMs $\Pi = (\mathcal{P}, \mathcal{V})$, which are defined as follows: The prover $\mathcal{P}$ throughout the protocol sends strings $\bar{m}_i = (m_i, g_i)$ in each round, where $m_i \in \{0, 1\}^*$ is called message and $g_i \in \{0, 1\}^*$ is called oracle. The verifier $\mathcal{V}$ learns m_i in round i , but not g_i . It sends random challenges $\rho_i \in \mathcal{C}_i$ in round i in response. In a final round, the receiver learns all g_i and outputs $b = \mathcal{V}_{\text{IOP}}(\mathbb{X}, ((m_1, \dots, m_k), (g_1, \dots, g_k), (\rho_1, \dots, \rho_{k-1})))$.

Definition 46 can easily be reconciled with state-restoration soundness. Namely, for any IOP protocol we simply let the message α_i be defined as $\alpha_i := (m_i, g_i)$. Then the resulting construction is still an IOP, as the state-restoration verifier, as defined in the state-restoration soundness error Definition 43, is only ever called after all messages $\alpha_1, \dots, \alpha_k$ (and the salts $\text{st}_1, \dots, \text{st}_k$) have been fixed in the experiment. This fulfills the requirements from Definition 46.

We now rewrite $\mathcal{V}_{\text{IP}}$ as an IOP which we call $\mathcal{V}_{\text{IOP}}$. We write $\text{AdvSRSIOP}_{\mathcal{A}}^{\mathcal{V}_{\text{IOP}}}$ for the advantage of an adversary $\mathcal{A}$ against state restoration soundness of the IOP with verifier $\mathcal{V}_{\text{IOP}}$.

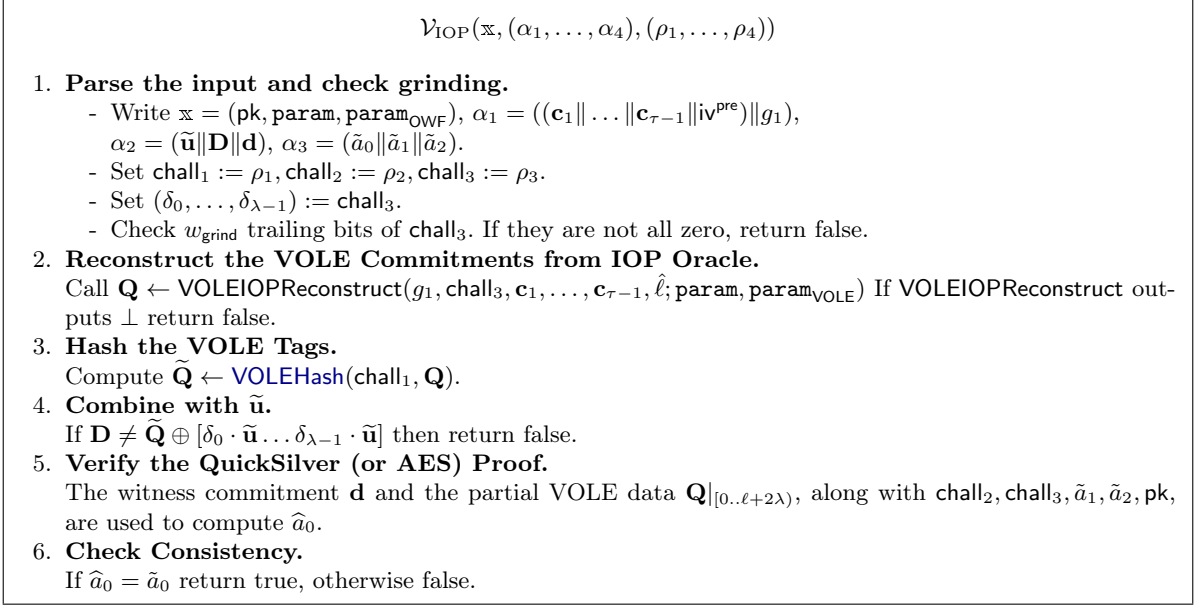


Figure 17. The FAEST IOP verifier.

The algorithm is defined in Figure 17.

The main difference to $\mathcal{V}_{\text{IP}}$ is that the commitment com contained in α_1 is now replaced by the IOP oracle string g_1 that contains all the $\mathbf{u}_i, \mathbf{V}_i$ pairs corresponding to the committed seeds as supposedly generated by VOLECommit . Therefore, no opening decom_I for the commitments must be sent in α_4 . Moreover, this also means that we use a new algorithm that reconstructs the VOLEs from the oracle string g_1 instead of the commitment and openings. We call this algorithm $\text{VOLEIOPReconstruct}$ and it is defined in Figure 18.

$\text{VOLEIOPReconstruct}$ differs from VOLEReconstruct as follows: Based on the BAVC commitment, the original algorithm uses BAVC.Reconstruct to obtain the $\text{sd}_{i,j}$, except for those at the unopened positions defined in $\Delta_0, \dots, \Delta_{\tau-1}$, and uses these, together with ConvertToVOLE , to compute the matrix $\mathbf{Q}$. This is an algorithm that can fail. In comparison, $\text{VOLEIOPReconstruct}$ simply takes $\mathbf{u}_i, \mathbf{V}_i$ as input and computes $\mathbf{Q}$ based on the relation that ConvertToVOLE generates according to Proposition 30. Therefore, $\text{VOLEIOPReconstruct}$ will succeed in computing $\mathbf{Q}$ as long as it obtains a valid oracle string g_1 as input.

Lemma 47. *Let $\mathcal{R}$ be the relation such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ if $\mathbb{x}$ is a public key and $\mathbb{w}$ the corresponding signing key for the FAEST signature scheme (with parameters $\text{param}, \text{param}_{\text{OWF}}, \text{param}_{\text{VOLE}}$ which will be left implicit). Let $\mathcal{A}$ be an algorithm with access to random oracles $\text{H}_0, \text{H}_1, \text{H}_4$ which wins the SRS soundness game defined in Definition 43 for $\mathcal{R}$ against $\mathcal{V}_{\text{IP}}$ with probability at least $p = \epsilon_{(\mathcal{A}, \mathcal{V}_{\text{IP}})}^{\text{sr}}((s_i)_{i \in [4]}, 4)$. Suppose $\mathcal{A}$ makes at most Q_0 queries to the random oracle H_0 and Q_1 to the random oracle H_1 .*

Then there exists a 4-round probabilistic algorithm $\tilde{\mathcal{P}}_{\text{IOP}}$ that wins the state-restoration game in Definition 43 with probability at least

$$\text{AdvSRSIOP}_{\mathcal{A}}^{\mathcal{V}_{\text{IOP}}} \geq p - \left(\frac{(Q_1 + 1)^2}{2^{2\lambda}} + \tau Q_0 \cdot 2^{-\lambda} \right)$$

against $\mathcal{V}_{\text{IOP}}$. For FAEST-EM, the success probability is

$$\text{AdvSRSIOP}_{\mathcal{A}}^{\mathcal{V}_{\text{IOP}}} \geq p - \left(\frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[Q_4, L] \right)$$

using the extractor from 32 that reprograms H_4 . If $\mathcal{A}$ makes β queries to the SRS game, then $\tilde{\mathcal{P}}_{\text{IOP}}$ makes $\beta + 1$ queries.

Note that we do not require $\tilde{\mathcal{P}}_{\text{IOP}}$ to be polynomial-time and in fact it won't be. This is however not a problem, since $\mathbb{x}$ is not a valid instance anyways so the reduction cannot trivially win the security game by finding a witness.

```

VOLEIOPReconstruct( $g_1, \text{chall}_3, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \hat{\ell}; \text{param}, \text{param}_{\text{VOLE}}$ )


---


INPUT:  $g_1 \in \{0, 1\}^{\hat{\ell} \times \tau} \times \{0, 1\}^{\hat{\ell} \times k_0} \times \dots \times \{0, 1\}^{\hat{\ell} \times k_{\tau-1}}, \text{chall}_3 \in \{0, 1\}^\lambda$ 
OUTPUT:  $\mathbf{Q} \in \{0, 1\}^{\hat{\ell} \times \lambda}$ 
1:  $I = (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{DecodeAllChall}_3(\text{chall}_3)$ 
2: if  $I = \perp$  then return  $\perp$ 
3: else do
4:   Write  $(\mathbf{u}_0, \dots, \mathbf{u}_{\tau-1}, \mathbf{V}_0, \dots, \mathbf{V}_{\tau-1}) := g_1$ 
5:   for  $i \in [0 \dots \tau)$  do
6:      $(\delta_{i,0}, \dots, \delta_{i,k_i}) \leftarrow \text{BitDec}(\Delta_i, k_i)$ 
7:     Consider  $\mathbf{V}_i$  column-wise as  $(\mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,k_i-1}) := \mathbf{V}_i$ 
8:     for  $j \in [0 \dots k_i)$  do
9:        $\mathbf{q}_{i,j} := \mathbf{v}_{i,j} \oplus \delta_{i,j} \cdot \mathbf{u}_i$ 
10:    if  $i = 0$  then
11:       $\mathbf{Q}_i := [\mathbf{q}_{i,0} \dots \mathbf{q}_{i,k_i-1}] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$  // stored in column major representation
12:    else
13:       $\mathbf{Q}_i \leftarrow [\mathbf{q}_{i,0} \dots \mathbf{q}_{i,k_i-1}] + [\delta_{i,0} \cdot \mathbf{c}_i \dots \delta_{i,k_i-1} \cdot \mathbf{c}_i] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$ 
14:     $\mathbf{Q} := [\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1} \mathbf{0}_{\hat{\ell},w}] \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$  //  $\mathbf{0}$  is  $(\hat{\ell} \times w)$  bits of zero-padding
15:  return  $\mathbf{Q}$ 

```

Fig. 18. VOLE reconstruction for the FAEST IOP.

Proof of Lemma 47. We now construct the algorithm $\tilde{\mathcal{P}}_{\text{IOP}}$ and prove its success probability. The intuitive idea is that $\tilde{\mathcal{P}}_{\text{IOP}}$ will forward most of the messages from $\mathcal{A}$ directly to the experiment, with the difference that it will extract all inputs from the BAVC com and convert them into an oracle string.

Let $\mathcal{A}$ be the algorithm described in the statement, then $\tilde{\mathcal{P}}_{\text{IOP}}$ works as follows: It runs an instance of $\mathcal{A}$ towards which it simulates the SRS IP experiment as defined in Figure 15. $\tilde{\mathcal{P}}_{\text{IOP}}$ itself runs within an analogous SRS IOP experiment.

- For the hash function $\mathbf{H}_0$, $\tilde{\mathcal{P}}_{\text{IOP}}$ simulates a random oracle using lazy sampling and records all query-response pairs on the list $\mathcal{L}_0 := (x_i, \mathbf{H}(x_i))$. Similarly, responses for queries to $\mathbf{H}_1$ will be simulated with lazy sampling and stored on $\mathcal{L}_1$ by $\tilde{\mathcal{P}}_{\text{IOP}}$. $\tilde{\mathcal{P}}_{\text{IOP}}$ also simulates $\mathbf{H}_4$ using lazy sampling.
- Whenever $\mathcal{A}$ outputs $(\mathbb{x}, (\alpha_1), (\text{st}'_1))$, with $\alpha_1 := (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$, $\tilde{\mathcal{P}}_{\text{IOP}}$ runs the extractor algorithm $\text{Ext}(\mathcal{L}_0, \mathcal{L}_1, \text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$ from Lemma 32 to obtain $\mathbf{D} = (\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)}$. $\tilde{\mathcal{P}}_{\text{IOP}}$ sets $g_1 := \mathbf{D}$ regardless of whether extraction was successful. It then sets $m_1 := ((\mathbf{c}_1 \| \dots \| \mathbf{c}_{\tau-1} \| \text{iv}^{\text{pre}}) \| g_1)$, $\text{st}_1 := \text{st}'_1 \| \text{com}$ and outputs $(\mathbb{x}, (m_1), (\text{st}_1))$ to the IOP SRS experiment and returns to $\mathcal{A}$ what the experiment returns.
- Whenever $\mathcal{A}$ outputs $(\mathbb{x}, (\alpha_1, \alpha_2), (\text{st}'_1, \text{st}_2))$ we compute m_1, st_1 as above and output $(\mathbb{x}, (m_1, (\alpha_2 \| \emptyset)), (\text{st}_1, \text{st}_2))$ to the experiment and return to $\mathcal{A}$ what the experiment returns.
- Similarly, whenever $\mathcal{A}$ outputs $(\mathbb{x}, (\alpha_1, \alpha_2, \alpha_3), (\text{st}'_1, \text{st}_2, \text{st}_3))$ we compute m_1, st_1 as above and output $(\mathbb{x}, (m_1, (\alpha_2 \| \emptyset), (\alpha_3 \| \emptyset)), (\text{st}_1, \text{st}_2, \text{st}_3))$ to the experiment and return to $\mathcal{A}$ what the experiment returns.
- Similarly, whenever $\mathcal{A}$ outputs $(\mathbb{x}, (\alpha_1, \alpha_2, \alpha_3, \alpha_4), (\text{st}'_1, \text{st}_2, \text{st}_3, \text{st}_4))$ we compute m_1, st_1 as above and output $(\mathbb{x}, (m_1, (\alpha_2 \| \emptyset), (\alpha_3 \| \emptyset), (\alpha_4 \| \emptyset)), (\text{st}_1, \text{st}_2, \text{st}_3, \text{st}_4))$ to the experiment and return $\emptyset$ to $\mathcal{A}$.
- Whenever $\mathcal{A}$ exits the loop and outputs $(\mathbb{x}, (\alpha_1, \alpha_2, \alpha_3, \alpha_4), (\text{st}'_1, \text{st}_2, \text{st}_3, \text{st}_4))$, $\tilde{\mathcal{P}}_{\text{IOP}}$ first computes g_1, m_1, st_1 as above. Then we first output $(\mathbb{x}, (m_1, (\alpha_2 \| \emptyset), (\alpha_3 \| \emptyset)), (\text{st}_1, \text{st}_2, \text{st}_3))$ to the IOP SRS experiment to obtain ρ_3 .

Then, let $\rho_3 := \text{chall}_3$, $\alpha_4 := \text{decom}_I$ and $\alpha_1 := (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$. Additionally, compute

$$(\widehat{\text{com}}, \widehat{\mathbf{Q}}) := \text{FAEST.VOLEReconstruct}(\text{decom}_I, \text{chall}_3, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}}; \text{param}).$$

If $\widehat{\text{com}} = \perp$ or $\widehat{\text{com}} \neq \text{com}$ or $g_1 = \perp$ while $\widehat{\text{com}} = \text{com}$ then abort. In case no abort happens, exit the loop with the IOP SRS game and output $(\mathbb{x}, (m_1, (\alpha_2 \| \emptyset), (\alpha_3 \| \emptyset), (\alpha_4 \| \emptyset)), (\text{st}_1, \text{st}_2, \text{st}_3, \text{st}_4))$ to the experiment.

The distribution of ρ_i and random oracle responses that $\mathcal{A}$ obtains from $\tilde{\mathcal{P}}_{\text{IOP}}$ is perfectly indistinguishable from those sent to $\mathcal{A}$ in the original SRS IP experiment. This is achieved by making com part of st_1 used to generate the random function response ρ_1 , which is necessary as an attacker might use different com with identical $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}}$ as part of some attack strategy. Thus, the behaviour of $\mathcal{A}$ towards $\tilde{\mathcal{P}}_{\text{IOP}}$ is perfectly indistinguishable from the behaviour it shows towards the regular experiment, in particular with regards to generating accepting transcripts.

The constructed reduction aborts whenever $\mathcal{V}_{\text{P}}$ would trivially abort, namely if the reconstructed $\widehat{\text{com}}$ is incorrect and the commitment therefore did not open correctly. The reduction also aborts when the extractor is not successful and returns $\perp$ but **VOLEReconstruct** outputs $\mathbf{Q}$. Otherwise, the behaviour of $\mathcal{V}_{\text{P}}$ and $\mathcal{V}_{\text{IOP}}$ only differs when $\mathcal{A}$ outputs a commitment com that it can successfully open with decom_I but where Ext extracted the wrong $(\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)}$. However, observe that $\tilde{\mathcal{P}}_{\text{IOP}}$ plays exactly the security game of Lemma 32 with $\mathcal{A}$. There, the winning condition of the attacker is either that the extracted $(\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)}$ are inconsistent with the output of **VOLEReconstruct** or that the extractor failed to generate $(\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)}$ but **VOLEReconstruct** returns $\mathbf{Q}$. By Lemma 32 the probability of the attacker winning the game is at most $\frac{(Q_1+1)^2}{2^{2\lambda}} + \tau Q_0 \cdot 2^{-\lambda}$, as claimed above. Note that $\tilde{\mathcal{P}}_{\text{IOP}}$ makes only one extra query to the SRS game it interacts with in order to recover ρ_3 .

□

FAEST IOP is RBR-sound. Looking at the Chiesa–Yogev book [CY24];²⁰ round-by-round (knowledge) implies state-restoration (knowledge) soundness with a $t + k$ multiplicative loss, where k is the round complexity of the proof system, and t is the “move budget” of the malicious state-restoration prover (we should think of $t = O(2^\lambda)$ because it’s the number of times the malicious prover can restore the state of the verifier, i.e. query the hash function). See [CY24], Theorems 31.2.1 and 31.3.1.

To handle the QROM as well, we define an augmented notion of round-by-round soundness.

Definition 48 (Round-by-round soundness, based on [CCH⁺18]). For a IP Π for a language $\mathcal{L}$, let RBRState be a state function mapping an instance and a transcript prefix to a Boolean value, with the following properties. For $\mathbb{x} \notin \mathcal{L}$,

1. $\text{RBRState}(\mathbb{x}, \emptyset) = 0$, where $\emptyset$ denotes the empty transcript prefix.
2. If for some instance $\mathbb{x}$ and a transcript prefix trc that is empty ($i=0$) or ends in the i -th challenge with $\text{RBRState}(\mathbb{x}, \text{trc}) = 0$, then for any prover message pmsg_{i+1} it holds that

$$\Pr_{\text{chall}_{i+1}} [\text{RBRState}(\mathbb{x}, \text{trc} | \text{pmsg}_{i+1} | \text{chall}_{i+1}) = 1] \leq \epsilon,$$

where chall_{i+1} is sampled uniformly from the appropriate challenge space, and the round-by-round soundness error ϵ depends on the security parameter only.

3. For any complete transcript trc , if $\text{RBRState}(\mathbb{x}, \text{trc}) = 0$ then the verifier rejects.

We say Π has round-by-round (RBR) soundness if for all no-instances $x \notin \mathcal{L}$ and all transcripts trc , ϵ is upper bounded by a negligible function of the security parameter. We say that a challenge chall_{i+1} is isolated if given $\text{RBRState}(\mathbb{x}, \text{trc}) = 0$, the event $\text{RBRState}(\mathbb{x}, \text{trc} | \text{pmsg}_{i+1} | \text{chall}_{i+1}) = 1$ depends on chall_{i+1} only. We say Π is RBR-sound with passive prefix up to chall_i if for a partial transcript trc ending in chall_{j-1} , given $\text{RBRState}(\mathbb{x}, \text{trc}) = 0$, the event $\text{RBRState}(\mathbb{x}, \text{trc} | \text{pmsg}_{i+1} | \text{chall}_{i+1}) = 1$ is independent of the prefix of trc ending in chall_m for $m = \min(i, j - 1)$.

Lemma 49. Let $\mathcal{R}$ be the relation such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ if $\mathbb{x}$ is a public key and $\mathbb{w}$ the corresponding signing key for the FAEST signature scheme (with parameters $\text{param}, \text{param}_{\text{OWF}}, \text{param}_{\text{VOLE}}$ which will be left implicit).

Let $\hat{\ell} \leq 2^{13}, \ell' \leq 2^{13}, \varepsilon_v = 2^{-\lambda-B}(1 + 2^{B-50}), \varepsilon_{\text{zk}} = 2^{-\lambda}(1 + 2^{-38})$ and $2 \log_2(\tau) \leq \min\{B, 50\}$. The FAEST IOP with verifier $\mathcal{V}_{\text{IOP}}$ has round-by-round soundness with soundness error $3/2^\lambda$, where $\mathbb{x}$ is not a valid instance of $\mathcal{R}$

²⁰ <https://github.com/hash-based-snargs-book/hash-based-snargs-book/blob/main/snargs-book.pdf>

Proof of Lemma 49. For the first challenge we can directly bound the probability by Lemma 11, for the second by Lemma 12 and for the last by Lemma 1.

The FAEST IOP is a 3-round IOP; in the first round, $\mathcal{P}_{\text{IOP}}$ sends

$$m_1 = (\mathbf{c}_1 \| \dots \| \mathbf{c}_{\tau-1} \| \text{iv}^{\text{pre}}) \| g_1 = (\mathbf{u}_i, \mathbf{V}_i)_i.$$

From this message, RBRState can reconstruct $\mathbf{u}, \mathbf{V}$ and apply $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$ to correct all “small” VOLE instances $\mathbf{u}_i, \mathbf{V}_i$ to the same secret as $\mathbf{u}_0$. If the secret of any small VOLE instances differs from $\mathbf{u}_0$ then the verifier should later reject.

After this, $\mathcal{V}_{\text{IOP}}$ responds with $\text{chall}_1 = \rho_1$. We now analyze how likely chall_1 will break soundness in this round by flipping RBRState from 0 to 1. This means that the hashes indicate that the secret committed in each VOLE instance is identical, when in fact they are different. We import following claim from [BBD⁺23, Theorem 2].

Claim 1. For all $(\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0..\tau)}$, $\mathbf{C} = [\mathbf{c}_1 \dots \mathbf{c}_{\tau-1}] \in \mathbb{F}_2^{\ell \times (\tau-1)}$, the probability that the consistency check fails for independently sampled VOLEHash challenges chall_1 is at most $\varepsilon_v \binom{\tau}{2}$.

Proof. See proof of [BBD⁺23, Theorem 2]. $\square$

Upon receiving chall_1 , the prover responds with $m_2 = (\tilde{\mathbf{u}} \| \mathbf{D} \| \mathbf{d})$. Using $\mathbf{d}$, we can reconstruct the witness $\mathbf{w}$ and run ZK.OWFProve inside RBRState (i.e. apply the computation to the committed witness). The commitments that are opened are those collected in $\langle \mathbf{z} \rangle_\lambda^3$ where z is supposedly 0. If any value committed in the commitments is non-zero then the verifier should later abort.

After the second round, $\mathcal{V}_{\text{IOP}}$ responds with $\text{chall}_2 = \rho_2$. We now analyze how likely non-zero commitments are hashed to a commitment of zero, thus breaking soundness and flipping RBRState from 0 to 1.

Claim 2 (ZK Hash check failure). Let $\mathbf{h} = \text{ZKHash}(\text{chall}_2, (\mathbf{x}_0, x_1)) = \mathbf{r}^\top \mathbf{x}_0 + x_1$. Let $\mathbf{e} \in \mathbb{F}_{2^\lambda}$ be the degree 3 coefficients of $\langle \mathbf{z} \rangle_\lambda^3$.

Define the bad event (“ZK Hash check fails”) as $\mathbf{e} \neq 0$ but $\mathbf{r} \cdot \mathbf{e} = 0$, i.e. the event where ZK Hash corrects an error into a valid VOLE correlation. The probability that the ZK Hash check fails for an independent chosen hash chall_2 is at most ε_{zk} .

Proof. This follows from Lemma 12 showing the ε_{zk} -universality of ZKHash . $\square$

In the third round, $\mathcal{P}_{\text{IOP}}$ sends

$$\alpha_3 = (\tilde{a}_0 \| \tilde{a}_1 \| \tilde{a}_2).$$

These define a degree 2 polynomial that is supposedly an opening of the ZKHash of $\langle \mathbf{z} \rangle_\lambda^3$ as computed in OWFProve . Assume that the output of ZKHash is a degree-3 polynomial. To this, $\mathcal{V}_{\text{IOP}}$ responds with $\text{chall}_3 = \rho_3$ (which is the random sampling of the QuickSilver challenge Δ). By Lemma 1 the soundness error of this step is exactly $3/2^\lambda$. Again, RBRState flips from 0 to 1 for the 3 challenges which breaks soundness.

Since $\varepsilon_{\text{zk}} = 2^{-\lambda}(1 + 2^{-38}) < 2^{-\lambda+1}$, we have that $3/2^\lambda$ is always larger than the soundness error of the last round is always larger than the soundness error of the second round. Moreover, we have that $\binom{\tau}{2} < \tau^2$, so as long as $2 \log_2(\tau) \leq \min\{B, 50\}$ we also have that

$$\binom{\tau}{2} 2^{-\lambda-B}(1 + 2^{B-50}) < 2^{2 \log_2(\tau)} 2^{-\lambda-B}(1 + 2^{B-50}) < 2 \cdot 2^{-\lambda}$$

and the soundness error of the first round is always smaller than the soundness error of the last round. $\square$

Finally, let us gather the chain from SRS security to RBR security in one corollary.

Corollary 50. Consider the setting of Lemma 45 for adversary $\mathcal{A}$ against EUF-KO with random public key, and recall

$$p = \Pr[\text{ExpEUFKR}_{\mathcal{A}}^{\text{SIG}}(\text{pk}) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}} \mid \text{pk} \leftarrow \{0, 1\}^{2\lambda}].$$

In particular, $\mathcal{A}$ makes at most $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_4$ queries to each of the random oracles $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4$ respectively. Suppose that $\hat{\ell} \leq 2^{13}, \ell' \leq 2^{13}$ and $2\log_2(\tau) \leq \min\{B, 50\}$. Then we have

$$\begin{aligned} p \leq & (Q_{2,0} + Q_{2,1} + Q_{2,1} + Q_{2,3} + 4)^2 / 2^{2\cdot\lambda} \\ & + \frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \\ & + (Q_{2,1} + Q_{2,1} + Q_{2,3} + 9) \cdot 3/2^\lambda \end{aligned}$$

where OWF is the OWF (or more precisely, PRG) defined by $\text{param}_{\text{OWF}}$, and

- $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \leq Q_0 \cdot 2^{2\lambda} \cdot \varepsilon_{\text{uhash}} \leq Q_0 \cdot 2^{-\lambda}$ for FAEST.
- $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}} \leq \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[Q_4, L]$ for FAEST-EM.

Proof. First, we apply Lemma 45 to turn $\mathcal{A}$ into an SRS adversary $\mathcal{A}_1$ with $(Q_{2,1} + Q_{2,1} + Q_{2,3} + 4)$ as an interactive proof (IP) system; success is reduced by at most $(Q_{2,0} + Q_{2,1} + Q_{2,1} + Q_{2,3} + 4)^2 / 2^{2\cdot\lambda}$. Secondly, we apply Lemma 47 to turn the SRS IP adversary into an SRS IOP adversary $\mathcal{A}_2$ which requires one additional query to the SRS IOP game; success is reduced by at most $\frac{(Q_1+1)^2}{2^{2\lambda}} + \text{AdvLeafColl}_{\mathcal{B}'}^{\text{LeafCommit}}$. Finally, we use [CY24, Theorem 31.2.1] to reduce the success of adversary $\mathcal{A}_2$ against SRS which makes at most $(Q_{2,1} + Q_{2,1} + Q_{2,3} + 5)$ queries, to RBR soundness. (Note here, that we only need to apply RBR soundness when $\text{pk} \notin \mathcal{L}_{\mathcal{R}}$; otherwise, the SRS game returns 0.) From Lemma 49 and the assumed bound on $\hat{\ell}, \ell', \tau, B$, we know that $\varepsilon_{\text{IOP}}^{\text{rbr}} \leq 3/2^\lambda$. Moreover, the IOP has 4 moves, and therefore we can bound success of $\mathcal{A}_2$ by

$$(Q_{2,1} + Q_{2,1} + Q_{2,3} + 9) \cdot 3/2^\lambda$$

This yields the claim. $\square$

5.3 EUF-CMA

Theorem 51 (FAEST is EUF-CMA secure). Let ρ be chosen uniformly at random. Let $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as random oracles and H_3 be a PRF²¹. Then for any PPT adversary $\mathcal{A}$ which makes $Q_{\text{sig}} \leq 2^{64}$ queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to $H_0, H_1, H_2^0, H_2^1, H_2^2$ and H_2^3 respectively,

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}] \leq & 2 \cdot \text{AdvEUF-KO}^{\text{FAEST}} \\ & + 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \\ & + 2 \cdot Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}_{\mathcal{B}_3}[1] + \text{AdvPRG}^{\text{LeafCommit}}_{\mathcal{B}_4}[N_0] + \text{AdvPRP}^{\text{PRG}_i}_{\mathcal{B}_5}[1] \right) \\ & + 2 \cdot Q_{\text{sig}} \cdot \left(\frac{2Q_1 + Q_{2,0} + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,1} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,2} + Q_{2,3}}{2^{3\lambda+64}} \right) \end{aligned}$$

For FAEST-EM, the reduction and bound holds.

Note that Theorem 51 only applies to randomized signing where ρ is chosen uniformly by the signer. After proving the statement, we will summarize how the proof has to be adjusted when setting $\rho = 0^\lambda$ in Corollary 52.

Proof. Let $\mathcal{A}$ be an arbitrary adversary against the EUF-CMA security of FAEST. We define a sequence of games which begins with $\mathcal{A}$ playing the real EUF-CMA game, where the signing oracle $\mathcal{O}_{\text{sig}}$ uses the real secret key sk to compute signatures, and ends with $\mathcal{O}_{\text{sig}}$ simulating signatures without using sk .

We then define the reduction $\mathcal{B}$ playing in the EUF-KO game to play the role of $\mathcal{O}_{\text{sig}}$ for $\mathcal{A}$ in the final game of the sequence and to output the forgery that $\mathcal{A}$ outputs. From this definition, it follows that $\mathcal{B}$ is successful in the EUF-KO game whenever $\mathcal{A}$ is in the final game.

The sequence of games is defined as follows:

²¹ Specifically, $H_3(\text{sk} \parallel \mu \parallel \rho) = \text{PRF}(\rho, \text{sk} \parallel \mu)$ is a PRF keyed by ρ and applied to $\text{sk} \parallel \mu$.

G_1 : $\mathcal{A}$ plays the EUF-CMA game with a real signature oracle. Clearly, we have:

$$\Pr[G_1 = 1] = \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}]. \quad (8)$$

G_2 : $\mathcal{O}_{\text{sig}}$ samples $r \in \{0, 1\}^\lambda$ and $\text{iv}^{\text{pre}} \in \{0, 1\}^{128}$ at random instead of computing $H_3(\text{sk} \parallel \mu \parallel \rho)$.

The difference between this game and the previous one reduces to the PRF security of H_3 with secret key ρ . Since we need to change (r, iv) in every query to $\mathcal{O}_{\text{sig}}$, we run a hybrid argument and obtain:

$$\Pr[G_1] \leq \Pr[G_2] + Q_{\text{sig}} \cdot \text{AdvPRF}_{\mathcal{A}_{\text{PRF}}}^{H_3}. \quad (9)$$

G_3 : $\mathcal{O}_{\text{sig}}$ samples $\text{chall}_1 \in \{0, 1\}^{5\lambda+64}$ at random in every query. When $\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}$ is queried to H_2^1 , game G_3 abort if this query to H_2^1 has already been made during the game; otherwise program H_2^1 to output chall_1 on this input.

We argue indistinguishability by reduction to unpredictability as stated in Lemma 36. We use a hybrid argument, where $G_{3,i}$ implements the abort only for the first i queries to $\mathcal{O}_{\text{sig}}$. Clearly, $G_{3,0} = G_2$ and $G_{3,Q_{\text{sig}}} = G_3$. Moreover, $G_{3,i-1}$ and $G_{3,i}$ only differ in the i -th $\mathcal{O}_{\text{sig}}$ query, and only if $\mathcal{A}$ predicted the query $\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}$. The i -th reduction $\mathcal{A}_{\text{unpred},i}$ to unpredictability plays unpredictability game in Lemma 36 such that it emulates $G_{3,i}$, except that for the i -th $\mathcal{O}_{\text{sig}}$ -query, it outputs its current state and the list $\mathcal{L}_{2,1}$ of queries made to $H_{2,1}$, receives $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1})$ from the unpredictability experiment (instead of running **VOLECommit**), and then continues from there. By construction, $G_{3,i-1} = 1 \iff \text{ExpUnpred}_{0, \mathcal{A}_{\text{unpred}}}^{\text{VOLECommit}, \hat{\ell}, \text{param}, \text{param}_{\text{VOLE}}} = 1$ and $G_{3,i} = 1 \iff \text{ExpUnpred}_{1, \mathcal{A}_{\text{unpred},i}}^{\text{VOLECommit}, \hat{\ell}, \text{param}, \text{param}_{\text{VOLE}}} = 1$. Thus, from Lemma 36 and a hybrid argument we get

$$\begin{aligned} \Pr[G_3 = 1] &\leq e^{Q_{\text{sig}} \cdot (4 \lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{128}} \cdot \left(\Pr[G_2 = 1] \right. \\ &\quad \left. + Q_{\text{sig}} \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}_{\mathcal{B}_2}^{\text{LeafCommit}}[1] + \frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} \right) \right) \end{aligned} \quad (10)$$

G_4 : $\mathcal{O}_{\text{sig}}$ samples $\text{chall}_2 \in \{0, 1\}^{3\lambda+64}$ at random in every query. When $\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d}$ is queried to H_2^2 , abort if this query to H_2^2 has already been made during the game; otherwise program H_2^2 to output chall_2 on this input.

G_4 can only abort if a query starting with chall_1 has been made to H_2^2 ; since chall_1 is fresh because of G_3 , by a hybrid argument, we get:

$$\Pr[G_3 = 1] \leq \Pr[G_4 = 1] + Q_{\text{sig}} \cdot \frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}.$$

G_5 : $\mathcal{O}_{\text{sig}}$ samples $\text{chall}_3 \in \{0, 1\}^\lambda$ at random in every query in every iteration of **FAEST.Sign**, until eventually the loop ends with $\text{chall}_3[\lambda - w_{\text{grind}}] \neq 0^{w_{\text{grind}}}$ and $I = \text{DecodeChall}_3(\text{chall}_3; \text{param})$ such that $\text{decom}_I \neq \perp$. When $\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b} \parallel \text{ctr}$ is queried to H_2^3 , abort if this query to H_2^3 has already been made during the game; otherwise program H_2^3 to output chall_3 on this input.

G_5 can only abort if a query starting with chall_2 has been previously made to H_2^3 ; since chall_2 is fresh because of G_4 , by a hybrid argument, we get:

$$\Pr[G_4 = 1] \leq \Pr[G_5 = 1] + Q_{\text{sig}} \cdot \frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}}.$$

Observe that the relevant elements of this game are now compatible with the hiding experiment for **VOLECommit** described in Lemma 35.

G_6 : $\mathcal{O}_{\text{sig}}$ replaces the call to **VOLECommit** by uniformly sampling $\mathbf{u} \in \mathbb{F}_2^{\hat{\ell}}$ and $(\mathbf{c}_i)_{i \in [1..\tau]}$ and computing $\mathbf{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$ such that the VOLE relation holds with the Δ and $\mathbf{q}_i$ values that result from chall_3 ; it does so in every signature query.

In the reduction, we sample chall_3 as in game G_5 , except that we replace the successful challenge chall_3 (where $\text{chall}_3[\lambda - w_{\text{grind}}] = 0^{w_{\text{grind}}}$ and $\text{decom}_I \neq \perp$) with the challenge of the hiding game. This game perfectly corresponds to game G_5 when $b = 0$, i.e. the real **VOLECommit** is used, or G_6 when $b = 1$. To see this here, note that $\text{decom}_I \neq \perp$ is derivable given only I (Remark 34), and thus embedding chall_3 obtained from the hiding game for the successful counter ctr poses no problem as

decom_I is not needed to check if chall_3 satisfies $\text{chall}_3[\lambda - w_{\text{grind}}] \neq 0^{w_{\text{grind}}}$ and $\text{decom}_I \neq \perp$. Moreover, since game G_5 establishes that programming works (or the game aborts), we can sample all chall_3 ahead of time. Hence, the non-adaptive nature of hiding as in Lemma 35 also poses no problem, and conceptual changes suffice to facilitate the reduction.

By Lemma 35 and a hybrid argument, the change in success is therefore bounded as

$$\Pr[G_5 = 1] \leq e^{Q_{\text{sig}} \cdot \tau \left(4 \lceil \log(L) \rceil \beta^2 + 16 \beta^2 N_0^2 + (\lceil \ell/128 \rceil)^2 \right) / 2^{128}} \cdot \left(\Pr[G_6 = 1] + Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}^{2\lambda}}[1] + \text{AdvPRG}_{\mathcal{B}_4}^{\text{LeafCommit}}[N_0] + \text{AdvPRP}_{\mathcal{B}_5}^{\text{PRG}^\ell}[1] \right) \right)$$

After this change, the distribution of $(\mathbf{u}, \mathbf{V})$ is uniform (modulo the VOLE relation) and independent of the VOLE commitments $(\mathbf{c}_i)_{i \in [1..\tau]}$.

G₇: In every query, $\mathcal{O}_{\text{sig}}$ samples $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$ at random (modulo the VOLE relation with $\tilde{\mathbf{Q}}$ and Δ) instead of computing **VOLEHash** during the signing. It then adjusts the last $\lambda + B$ elements of $\mathbf{u}$ and rows of $\mathbf{V}$ to match $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$ when **VOLEHash** is applied under challenge chall_1 to $\mathbf{u}, \mathbf{V}$.

Since the last $\lambda + B$ elements of $\mathbf{u}$ and rows of $\mathbf{V}$ are uniformly random (modulo the VOLE relation) because of G_6 , and **VOLEHash** is $\mathbb{F}_2^{\ell+\lambda}$ -hiding by Lemma 11, this game is perfectly indistinguishable from the previous one.

G₈: In every query, $\mathcal{O}_{\text{sig}}$ samples $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2$ at random (modulo the VOLE relation) instead of computing **ZKHash**. It then computes **OWFProve**, but adjusts the $\mathbf{u}, \mathbf{V}$ in rows $[\ell..\ell + 2\lambda]$ to match $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2$ when **ZKHash** is applied under challenge chall_2 to $\mathbf{a}_0 \| u_0^*, \mathbf{a}_1 \| v_0^* + u_1^*, \mathbf{a}_2 \| v_1^*$.

Since the respective rows of $\mathbf{u}$ and $\mathbf{V}$ are uniformly random (modulo the VOLE relation) because of G_6 , and **ZKHash** is $\mathbb{F}_2^\ell$ -hiding by Lemma 12, the distribution of $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$ is unchanged and this game is perfectly indistinguishable from the previous one.

G₉: Now that the computation of $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2$ is independent of $\mathbf{w}$, $\mathcal{O}_{\text{sig}}$ will sample $\mathbf{d}$ uniformly at random in every query instead of computing $\mathbf{d} := \mathbf{w} + \mathbf{u}_{[0..\ell]}$.

This does not change the distribution of $\mathbf{d}$ because $\mathbf{u}$ is uniform since G_6 and the first ℓ elements were not used in games G_7 and G_8 to produce their outputs, hence this game is perfectly indistinguishable from the previous one.

In G_9 , we see that the distribution of σ produced by $\mathcal{O}_{\text{sig}}$ no longer depends on the secret key sk and the success probability of $\mathcal{A}$. However, we cannot yet reduce to EUF-KO as we must show that none of the points programmed into the RO by the simulator has aided the attacker in creating a forgery. For H_2^0 to be useful in a forgery we know that, as we only want to achieve weak EUF-CMA security, the input has to be different while still generating the same μ as msg must differ. Thus, any aid on a programmed point is due to a successful multi-instance second preimage attack on the outputs of H_2^0 generated during signature simulation. Assuming that the attack was not successful on H_2^0 , an attacker might succeed with the same strategy on H_1^1 or H_2^2 with the inputs of the other hash functions being different. To address this, we add an additional hybrid which aborts whenever the forgery uses RO queries that depend on RO points programmed by the simulation.

G₁₀: Let L_0 be the list of all queries made to H_2^0 . Equivalently, let L_1 be the list of all queries made to H_2^1 and L_2 be the list of all queries made to H_2^2 . Note that these lists contain all queries, both added by the reduction and made by $\mathcal{A}$. In this hybrid, we abort whenever there is a collision $(x_0, y), (x_1, y) \in L_0$ with $x_0 \neq x_1$ or any such collision in L_1 or L_2 where one query was programmed by the reduction and the other generated by $\mathcal{A}$.

The abort probability can be bounded by

$$\frac{Q_{2,0} \cdot Q_{\text{sig}}}{2^{2\lambda}} + \frac{Q_{\text{sig}} \cdot Q_{2,1}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} \cdot Q_{2,2}}{2^{3\lambda+64}}.$$

All terms trivially follow using standard bounds on multi-instance second preimage resistance, and then we apply a union bound.

G_{10} does not contain any dependency on H_2^3 . To see why this is the case, notice that the input to H_2^3 contains chall_2 which itself depends on chall_1 and μ . By assumption, for a successful forgery, it must be that neither μ nor chall_1 or chall_2 in the forgery are outputs programmed by the simulator (otherwise

this was already captured in the above hybrid). Thus, the forgery must contain an input chall_2 to H_2^3 that was not queried before. Hence, none of the Q_{sig} programmed points in H_2^3 can be used to create a valid forgery.

Taking together the intermediate advantages, we arrive at

$$\begin{aligned}
\Pr[G_0 = 1] &\leq \rho \cdot \left(\Pr[G_{10} = 1] \right. \\
&\quad + Q_{\text{sig}} \cdot \text{AdvPRF}^{H_3} \\
&\quad + Q_{\text{sig}} \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] + \frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} \right) \\
&\quad + Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}} \right) \\
&\quad + Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRG}^{\text{LeafCommit}}_{B_4}[N_0] + \text{AdvPRP}^{\text{PRG}_{\ell}}_{B_5}[1] \right) \\
&\quad \left. + Q_{\text{sig}} \cdot \left(\frac{Q_{2,0}}{2^{2\lambda}} + \frac{Q_{2,1}}{2^{5\lambda+64}} + \frac{Q_{2,2}}{2^{3\lambda+64}} \right) \right) \\
&= \rho \cdot \Pr[G_{10} = 1] \\
&\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \\
&\quad + \rho \cdot Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRG}^{\text{LeafCommit}}_{B_4}[N_0] + \text{AdvPRP}^{\text{PRG}_{\ell}}_{B_5}[1] \right) \\
&\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\frac{2Q_1 + Q_{2,0} + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,1} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,2} + Q_{2,3}}{2^{3\lambda+64}} \right)
\end{aligned}$$

where

$$\rho = e^{Q_{\text{sig}} \cdot (4\lceil \log(L) \rceil \beta^2 + 16\beta^2) / 2^{128} + Q_{\text{sig}} \tau \cdot (4\lceil \log(L) \rceil \beta^2 + 16\beta^2 N_0^2 + (\lceil \ell / 128 \rceil)^2) / 2^{128}}$$

is bounded by 2 for all proposed parameter sets. **Observe also, that in the reduction up until now, there are no differences for FAEST or FAEST-EM.**

At this point, we reduce to EUF-KO security. Formally, game G_{10} still programs the random oracle, and thus does not fit into the EUF-KO experiment (which itself provides access to the random oracles and verifies EUF-KO w.r.t. its own oracles). We explain how to bridge this formal gap. After the changes in games G_1 through G_{10} , the reduction in G_{10} always simulates signing oracle queries and aborts if the adversary guessed a programmed query beforehand or finds a collision. Thus, we can define an alternative adversary $\mathcal{A}'$ playing game a modified game G'_{10} as follows. By definition, $\mathcal{A}'$ never queries $\mathcal{O}_{\text{sig}}$ and instead runs signature simulation and programming of the random oracle queries “in its head” (including the abort conditions, which are verifiable from the queries alone, hence by $\mathcal{A}'$). By construction, $\mathcal{A}'$ never makes queries to programmed random oracle inputs. As a consequence, G'_{10} does not (need to) program its random oracles nor does it need to answer any signature queries. Moreover, observe that the probability that G_{10} with $\mathcal{A}$ outputs 1 is identical to the probability that G'_{10} with $\mathcal{A}'$ outputs 1, as we have just moved some code from the game into the adversary. Thus, from G'_{10} a direct reduction to EUF-KO security is possible at this point, and we have $\Pr[G_{10} = 1] = \Pr[G'_{10} = 1] \leq \text{AdvEUFKO}_{\mathcal{A}'}^{\text{FAEST}}$. (Note also that the number of random oracle queries of $\mathcal{A}'$ is at most the number of queries of $\mathcal{A}$.)

We note here, that there is a formal compositional problem: The EUF-CMA reduction programs random oracles, which interferes with the EUF-KO claims. However, these problems are of formal nature. If we would instead include the EUF-KO proof here, then the advantage terms due to collisions, pre-image attacks, or guesses of random oracle outputs would be the same. $\square$

Now, we consider the deterministic version of FAEST and prove how its reduction from EUF-CMA to EUF-KO works. The proof of Theorem 51 breaks down in G_2 where we reduce to the pseudorandomness of H_3 based on the uniformity of ρ . This does not work if ρ is public and fixed. To salvage the proof, we rely on the randomness of sk and a stronger assumption on H_3 , namely, that sk can serve as a PRF secret even when pk is known to the distinguisher. For simplicity, we model H_3 as a random oracle.²² Observe

²² It is possible to rely on a standard-model assumption with a slightly more complex proof. But we already use random oracles in several places.

that until $\mathcal{A}$ queries H_3 on input $\text{sk} \parallel \mu \parallel 0^\lambda$, programming $H_3(\text{sk} \parallel \mu \parallel 0^\lambda)$ to provide a uniformly random output as done in Game G_2 still works. We argue that we can w.l.o.g. assume that $\mathcal{A}$ does not query H_3 at such a point. For this, we replace $\mathcal{A}$ with an adversary $\mathcal{A}'$ which acts as follows: It internally runs $\mathcal{A}$. However, for each query $\text{sk} \parallel \mu \parallel 0^\lambda$ to H_3 it checks if $\widehat{\text{sk}}$ is a valid secret key for pk , i.e. if $\mathbf{y} = \text{OWF}_{\mathbf{x}}(\widehat{\text{sk}})$ where $\text{pk} := (\mathbf{x}, \mathbf{y})$. For any query $\widehat{\text{sk}} \parallel \mu \parallel 0^\lambda$ where $\widehat{\text{sk}}$ is not a valid secret key for pk , $\mathcal{A}'$ implements H_3 honestly (i.e., returns randomness). However, if such a secret key is queried by $\mathcal{A}$ to H_3 , then $\mathcal{A}'$ uses $\widehat{\text{sk}}$ to sign a random message (that was not signed by $\mathcal{O}_{\text{sig}}$ previously) and outputs it as a forgery to the security game.

Computationally, the overhead of $\mathcal{A}'$ is small, i.e. $t_{\mathcal{A}'} \approx t_{\mathcal{A}}$ as it only additionally runs **FAEST.Sign**, and it only requires additional queries to the random oracles $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3$ and H_4 proportional to one execution of **FAEST.Sign**. Moreover, the input distribution to $\mathcal{A}$ does not change and thus its success probability in creating a forgery does not decrease. Hence, $\mathcal{A}'$ will create a forgery with at least the same probability as $\mathcal{A}$. Therefore, we can assume $\mathcal{A}$ makes no such successful queries to H_3 and we can apply the same hybrid steps as in the proof of Theorem 61.

Corollary 52 (Deterministic FAEST is EUF-CMA secure). *Let $\rho := 0^\lambda$. Let $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_3$ and H_4 be modeled as random oracles. Then for any PPT adversary $\mathcal{A}$ which makes $Q_{\text{sig}} \leq 2^{64}$ queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}$ and Q_3 queries to $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3$ and H_3 respectively, there is a PPT adversary $\mathcal{A}'$ with almost the same running time, which makes $Q'_i \approx Q_i$, $Q'_{i,j} \approx Q_{i,j}$ queries, differing only by one honest **FAEST.Sign** computation which $\mathcal{A}'$ additionally computes, such that*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}} \\ &+ 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvOWF}^{\text{OWF}} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \\ &+ 2 \cdot Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRG}^{\text{LeafCommit}}_{\mathcal{B}_4}[N_0] + \text{AdvPRP}^{\text{PRG}_i}_{\mathcal{B}_5}[1] \right) \\ &+ 2 \cdot Q_{\text{sig}} \cdot \left(\frac{2Q'_1 + Q'_{2,0} + Q'_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q'_{2,1} + Q'_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q'_{2,2} + Q'_{2,3}}{2^{3\lambda+64}} \right) \end{aligned}$$

For FAEST-EM, the reduction and bound holds.

6 QROM Proofs

In this section, we present the techniques that facilitate a proof of tight EUF-CMA-security of VOLEitH-based Fiat-Shamir signatures. We will organize this section along 3 challenges that arise in this security proof, and present the tools we develop to solve them. We describe general versions of the tools in Section 6.1 to 6.3. Lemmas tailored to FAEST and the its full QROM security proof for FAEST can be found in Section 6.4 to 6.11.

6.1 Challenge 1: No Tight Generic Reduction for Fiat-Shamir

In the ROM, there is a security property of a public-coin interactive proof system (IP) IP that reduces tightly to the soundness of its Fiat-Shamir transformation $\text{FS}[\text{IP}]$: State-Restoration Soundness. This flavor of soundness is essentially just an interactive model of ROM security for $\text{FS}[\text{IP}]$. This is, unfortunately, unlikely to have any reasonable analogue in the QROM: the interaction for IP is *classical* (i.e., non-quantum) and the quantum-accessible random oracle is quantum. Any plausible generalization of the reduction would thus require techniques involving quantum measurement leading to unacceptable reduction losses.

One way around this problem would be to avoid a reduction proof altogether by exploiting a purely statistical property of the underlying IP, like round-by-round (RBR) soundness. For a large class of digital signature scheme (DSS) constructions that includes FAEST, this poses a problem: The natural IP IP_0 that yields the DSS upon Fiat-Shamir transformation is still in the (Q)ROM, and can thus not be RBR-sound. The culprit here are hash-based commitments modeled in the (Q)ROM. For efficiency, they are usually not statistically-binding, e.g., when tree commitments are used, like BAVC presented in Section 3.

A somewhat counterintuitive but conceptually simple solution to this problem is to reconceptualize the DSS as a (non-standard) FS transformation of a different IP IP , by viewing every QRO call as a Fiat-Shamir hash, including the ones for generating commitments. Whenever IP_0 has a *classical* “straightline extractor” that extracts a for the *honest* prover given a list of its RO queries, IP must have RBR soundness²³.

We now give a toy example of a signature scheme to illustrate the technique (see Section 6.6 for the case of FAEST). On a high level, the IP IP_0 underlying any VOLEitH and MPCitH signature schemes work as follows:

1. Create a secret sharing of the private key sk into shares $s_0, \dots, s_{L-1}$.
2. Commit to the shares in a way that allows opening certain subsets of them, producing a commitment com
3. Create a proof $\tilde{\pi}$ that the secret-shared sk is valid for the public key pk , that can be verified using certain subsets of the shares s_i
4. Send $(\text{com}, \tilde{\pi})$ to the Verifier and receive challenge $\text{chall} \subset [L]$.
5. Open $(s_i)_{i \in \text{chall}}$

In general, there might be additional rounds of interaction, e.g., to sample keys for universal hash functions as in FAEST. For now, consider the simplest case where $\text{com} = (\text{com}_0, \dots, \text{com}_{L-1})$, $\text{com}_i = \text{H}_1(s_i, r_i)$ is a hash-based commitment to s_i using a hash function H_1 modeled as a RO, and chall is the only verifier message. The Fiat-Shamir DSS based on this IP then computes $\text{chall} = \text{H}_2(\text{pk}, \text{msg}, \text{com}, \tilde{\pi})$. If we are only interested in soundness²⁴, we can replace all calls to H_1 by a round of interaction with the verifier, i.e., Line 2 above is replaced with

- 2.’ For $i = 0, \dots, L - 1$, sample r_i . Send $(s_i, r_i)_{i \in [L]}$ to the verifier and receive challenge $(\text{com}_i)_{i \in [L]}$.

This defines a 2-challenge IP IP . We now consider a non-standard variant of the Fiat-Shamir transformation where the first challenge is derived using parallel application of the RO H_1 . This transformation recovers the original DSS.

For more efficient commitments like BAVC used in FAEST, more rounds have to be added as some of the RO queries need to happen sequentially, which requires replacing them with sequential rounds of interaction.

6.2 Challenge 2: Proving Tight Soundness of Fiat-Shamir from RBR Soundness

After re-conceptualizing a DSS as the FS transformation of a RBR-sound IP, we face the next challenge: To the best of our knowledge, there is no theorem available in the literature tightly proving the soundness of the FS transformation of a RBR-sound IP. Fortunately, the framework for proving query complexity bounds in the QROM presented in [CFHL21] was used for a similar problem before [AHJ⁺23], and provides the necessary technical tool. In this section, we use this tool to prove soundness of the FS transformation of a RBR-sound IP.

We begin by formalizing the necessary properties of the non-standard FS transformation from Section 6.1. We define two properties of RO-based functions that make them fit for deriving challenges in a Fiat-Shamir transformation.

Definition 53. A function $f^{\text{H}} : \mathcal{X} \rightarrow \mathcal{Y}$ that can be computed using L queries to another function $\text{H} : \mathcal{X}' \rightarrow \mathcal{Y}'$ is collision resistance preserving, if the following holds. For all $x_0, x_1 \in \mathcal{X}$ such that $x_0 \neq x_1$ and $f^{\text{H}}(x_0) = f^{\text{H}}(x_1)$ there are $x'_0, x'_1 \in \mathcal{X}'$ such that $x'_0 \neq x'_1$ and

1. H is queried on input x'_i when $f^{\text{H}}(x_i)$ is computed for $i \in \{0, 1\}$.
2. $\text{H}(x'_0) = \text{H}(x'_1)$.

The function is sensitive if i) $|\{f^{\text{H}_{x' \mapsto u}}(x) | u \in \mathcal{Y}'\}| \in \{1, |\mathcal{Y}'|\}$ (SN1) and ii) for $x \neq x'$, $f^{\text{H}}(x)$ does not make the same queries to H as $f^{\text{H}}(x')$ (SN2).

²³ In this generality, there might be some freedom in how to define the verifier of IP, so the claim here is to be understood as the existence of a Verifier for IP such that it has RBR soundness

²⁴ and if we in particular are fine with having no chance of any flavor of zero knowledge

Note that $f^H(x_0, x_1, \dots, x_{L-1}) = (H(x_0), H(x_1), \dots, H(x_{L-1}))$ has both properties. We go on to state and prove the main theorem of this section. For ease of presentation, we formulate the theorem for the FS transformation for proofs, rather than signatures, and denote problem instances by $\mathbb{x}$ instead of $\mathbf{pk}$.

Theorem 54 (RBR Soundness of IP $\Rightarrow$ Soundness of FS[IP]). *Let IP be an IP with ℓ challenges $(\text{chall}_i)_{i \in [\ell]}$. Let $S \subset [\ell]$, $|S| = \ell_1$, such that IP is RBR sound with RBR soundness error 0 for challenges chall_i where $i \in S$, and RBR soundness error ε else. Let f^{H_1} be a collision-resistance-preserving and sensitive function making L queries to H_1 , and let FS[IP] be the Fiat-Shamir transformation of IP where challenges chall_i for $i \in S$ are derived using f^{H_1} , and challenges chall_i are derived using H_2^i for $i \notin S$. Denote by $H_2 = (H_2^i)_{i \in [\ell] \setminus S}$ an oracle that allows querying any of the H_2^i . For any static QROM adversary $\mathcal{A}^{H_1, H_2}$ against FS[IP] that makes a total of $\tilde{Q}$ quantum queries to its oracles,*

$$\text{AdvSnd}_{\mathcal{A}}^{\text{FS[IP]}} \leq 30 \cdot Q^3 \cdot 2^{-l_{\text{out}}} + 10 \cdot Q^2 \cdot \varepsilon$$

where l_{out} is the output length of H_1 and H_2^i and $Q = \tilde{Q} + 2\ell + 2(L-1)\ell_1$

Proof. We instantiate the random oracles via domain separation of a random oracle H with sufficiently large output length for this proof, by setting $H_i^j = H(i||j||\cdot)$ truncated to the correct output length, with appropriate encoding of the number prefixes, where $H_1^0 := H_1$. For ease of notation, we define f_i^H as the function used to derive chall_i . Let T^{IP} be the set of transcripts for IP. To use Lemma 25, we define the following predicate on pairs of partial transcripts $\text{trc} = (\text{msg}_1, \text{chall}_1, \dots, \text{chall}_i)$,

$$\begin{aligned} \text{pred}_{\mathbb{x}, H}(\text{trc}) = & \text{RBRState}(\mathbb{x}, \text{trc}) \wedge \neg \text{RBRState}(\mathbb{x}, \text{trc}_{i-1}) \\ & \wedge (\forall j = 1, \dots, i : \text{chall}_j = f_j^H(\text{chall}_{j-1} || \text{msg}_j)). \end{aligned}$$

Here we have defined the truncation $\text{trc}_{i-1} = (\text{msg}, \mu, \text{msg}_1, \text{chall}_1, \dots, \text{chall}_{i-1})$ and set $\text{chall}_0 = \iota$, the empty string. If $\text{pred}_{\mathbb{x}, H}(\text{trc}, \mu)$, we say (trc, μ) is *bad* for H . We further define a predicate $P : (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}} \rightarrow \{0, 1\}$ by $P(D) := \text{there exists bad } (\text{trc}, \mu) \text{ for } D$. P has $(\ell + \ell_1 \cdot (L-1))$ -local witnesses, as the list of random oracle inputs queried when evaluating $\text{pred}_{\mathbb{x}, H}(\text{trc})$ is a witness if $\text{pred}_{\mathbb{x}, H}(\text{trc})$. We call i the length of the witness. In the following we denote by D_i^j the partial functions obtained in the same way as H_i^j are obtained from H . We also define the collision predicate P_C by setting $P_C(D)$ iff there exist inputs x, x' and indices i, j such that $D_i^j(x) = D_i^j(x') \neq \perp$. The collision predicate clearly has 2-local witnesses. We construct algorithm $\mathcal{B}^H$ as follows. $\mathcal{B}^H$ runs $\pi \leftarrow \mathcal{A}^H$ and recomputes the challenges. Denote the completed transcript by trc . If there exists i such that $\text{RBRState}(\mathbb{x}, \text{trc}_i)$ and $\neg \text{RBRState}(\mathbb{x}, \text{trc}_{i-1})$, output the predicate witness z , else, output $\perp$. By RBR soundness (Lemma 57) we have

$$\Pr_{\pi \leftarrow \mathcal{A}^H(\mathbb{x})} [\text{verifier accepts } \pi] \leq \Pr_{z \leftarrow \mathcal{B}^H} [\langle z, P, H \rangle] \leq \Pr_{z \leftarrow \mathcal{B}^H} [\langle z, P \vee P_C, H \rangle].$$

We now aim to bound the quantity

$$\max_{\substack{x, D: \\ |D| \leq k, \neg P'(D)}} \Pr_{u \leftarrow \mathcal{Y}} [P'(D[x \mapsto u])]$$

for $P' = P \vee P_C$. In the following, we omit the domain of the maximum. Here, D is a partial function with the same domain and range as H . We distinguish different cases depending on which domain x is from. We derive bound assuming the query does not derive the last challenge. If it derives the last challenge, the bounds also hold as a strict subset of the options for making the database go bad exist in this case.

1. x is in the domain of H_1 . As the RBR soundness error of the challenges in whose derivation H_1 is involved in is 0, so there are two ways to fulfill $P'(D[x \mapsto u])$, i) if a collision occurs, i.e., $\exists x' : D_1^0(x) = D_1^0(x') \neq \perp$, or ii) if there exists x, y such that $f^{D_1^0[x \mapsto u]}(x) = y$ and y is the challenge part of some x' such that $\exists j : D_2^j(x') \neq \perp$, or $f^{D_1^0}(x') \neq \perp$. By the sensitivity of f , we have

$$\Pr_{u \leftarrow \mathcal{Y}} [P'(D[x \mapsto u])] \leq 3|D|2^{-l_{\text{out}}}.$$

2. x is in the domain of H_2^j for some j . For D with $\neg P'(D)$, there are 3 ways to fulfil $P'(D[x \mapsto u])$. The first option is that there exists x' in the domain of H_2^j , $x' \neq x$ such that $D(x') = u$ (i.e. a new collision has formed). The second option is that a new witness of length $i' > i$ for P is completed. In this case, there exists pmsg_{i+1} such that $f_{i+1}(u \parallel \text{pmsg}_{i+1}) \neq \perp$. The third option is that a new witness of length i for P is completed. In this case, it is necessary that there exists a partial witness (which is unique by $\neg P_C(D)$ and the collision-resistance-preservingness of f) with the following properties. Let $\text{chall}_{i-1} = f_{i-1}^D(z_{\text{last}})$, where z_{last} is the last entry of z , let trc be the transcript obtained from z , and let trc_{i-2} be its truncation to the second-to-last challenge. Then
- (a) z is consistent, i.e., recomputing the chall_i that appear in the entries of z using D succeeds
 - (b) $\text{RBRState}(\mathbb{x}, \text{trc}_{i-2}) = 0$
 - (c) there exists pmsg_i such that $x = 2\|i\|\text{chall}_{i-1}\|\text{pmsg}_i$ and $\text{RBRState}(\mathbb{x}, (\text{pmsg}_1, \text{chall}_1, \dots, \text{pmsg}_i, u)) = 1$.

In summary, we have

$$\Pr_{u \leftarrow \mathcal{Y}}[P'(D[x \mapsto u])] \leq 2|D| \cdot 2^{-l_{\text{out}}} + \varepsilon.$$

where the term $2|D| \cdot 2^{-l_{\text{out}}}$ is from the first and second option, we have used RBR soundness to bound the probability

$$\Pr_{u \leftarrow \mathcal{Y}}[\text{RBRState}(\mathbb{x}, (\text{pmsg}_1, \text{chall}_1, \dots, \text{pmsg}_i, u))] \leq \varepsilon.$$

In conclusion, we have

$$\max_{u \leftarrow \mathcal{Y}} \Pr[P'(D[x \mapsto u])] \leq \max(3|D| \cdot 2^{-l_{\text{out}}}, 2|D| \cdot 2^{-l_{\text{out}}} + \varepsilon) \leq \frac{3\tilde{Q}}{2^{l_{\text{out}}}} + \varepsilon := \delta.$$

Using Eq. (11), and applying Lemma 25 to the $(\tilde{Q} + \ell + (L-1)\ell_1)$ -query algorithm $\mathcal{B}$, recalling that P has $(\tilde{Q} + \ell + (L-1)\ell_1)$ -local witnesses, we obtain for any invalid instance $\mathbb{x} \notin \mathcal{L}$,

$$\begin{aligned} & \Pr_{\pi \leftarrow \mathcal{A}^{\text{H}_1, \text{H}_2}(\mathbb{x})}[\mathcal{V}_{\text{FS}[\text{IP}]}(\mathbb{x}, \pi) = 1] \\ & \leq \left(\sum_{k=1}^{\tilde{Q} + 2\ell + 2(L-1)\ell_1} \sqrt{10 \max_{u \leftarrow \mathcal{Y}} \Pr[P(D[x \mapsto u])]} \right)^2 \leq \left(\sum_{k=1}^{\tilde{Q} + 2\ell + 2(L-1)\ell_1} \sqrt{10\delta} \right)^2 \\ & = 10(\tilde{Q} + 2\ell + 2(L-1)\ell_1)^2 \delta \\ & = 30(\tilde{Q} + 2\ell + 2(L-1)\ell_1)^3 2^{-l_{\text{out}}} + 10(\tilde{Q} + 2\ell + 2(L-1)\ell_1)^2 \varepsilon. \end{aligned} \quad \square$$

6.3 Challenge 3: Reprogramming a QRO at a Computationally Unpredictable Input

The third challenge arises when upgrading EUF-KO security to EUF-CMA security. For FS signatures, reductions of breaking EUF-KO to breaking EUF-CMA are based on the following rough strategy. The IP underlying the FS DSS has some form of simulability, usually honest-verifier zero knowledge (HVZK). In the following, we assume the IP has HVZK. To show that an adversary with access to a signing oracle can be simulated without using the private key, simulated proofs are generated using the HVZK simulator. The random oracle(s) are then re-programmed to be consistent with the simulated proofs being honest signatures. This needs to happen adaptively – the outputs to be re-programmed are only revealed to the reduction once a query to the signing oracle is made. To show that this re-programming is indistinguishable, it is a necessary ingredient to show that it is hard to predict the inputs where the RO will be re-programmed, and thus the adversary will not have queried these inputs in advance. In the QROM, this step is facilitated by the adaptive reprogramming lemma (also known as resampling lemma) [GHHM21]. Here, it is exploited that the inputs where the re-programming happens are *statistically* unpredictable in the sense that they have min-entropy conditioned on the adversary's view.

In the corresponding step in the ROM security proof of FAEST, however, it is used that the BAVC commitment in a FAEST signature is *computationally* unpredictable. The reduction is to an unpredictability game where the adversary has to output a list of guesses, and wins if the commitment generated by the challenger is in the list. Such an argument cannot be generalized to the QROM setting using the

techniques of [GHHM21] directly. It is natural to instantiate the QRO using a compressed oracle for the reduction, but to reduce to the list-unpredictability game a quantum measurement needs to be performed.

We chose a different way to solve this problem. To retain as much tightness as possible, while also achieving some modularity in the security proof, we define quantum-list unpredictability, where only a two-outcome measurement is performed. This two-outcome measurement checks whether the commitment generated by the challenger is in a quantum register the adversary has submitted. Quantum-list unpredictability is a clean security property that can be evaluated for any cryptographic algorithm (not just commitments), and it facilitates a tight bound on the distinguishability between a security game and a hybrid that is aborted if a reprogrammed input was previously queried.

An additional advantage of this technique is, that a (small) multiplicative loss, like the one arising in proofs using Rényi-divergence arguments, need not be converted into an additive loss. This allows for better parameters. To facilitate this type of reduction, our definition of quantum-list unpredictability has a slightly more complicated syntax than it would appear necessary.

The natural definition of quantum-list unpredictability is as follows.

Definition 55 (Quantum-List Unpredictability). *Let $\mathcal{B}$ be an algorithm, possibly with access to one or more (Q)ROs. The quantum-list unpredictability game $\text{ExpUnpred}_{b,\mathcal{A}}^{\mathcal{B}}$ between an adversary $\mathcal{A}$ and a challenger is as follows.*

1. $(L, \text{state}) \leftarrow \mathcal{A}(1^\lambda)$, where L is a quantum register holding a (superposition of) list(s) $\mathcal{L}_{\text{com}}$ of guesses for the output of $\mathcal{B}$.
2. $C \leftarrow \mathcal{B}(1^\lambda)$.
3. Set $b' = 1$. If $b = 1$
 - (a) apply the binary measurement to L that tests membership of C in the list $\mathcal{L}_{\text{com}}$ the register L contains (in superposition) to obtain output b'' ($b'' = 1$ indicates “found”).
 - (b) If $b'' = 1$, set $b' = 0$.
4. If $b' = 1$, set $b' \leftarrow \mathcal{A}^{\text{H}_0, \text{H}_1}(\text{state}, L, C)$.
5. Output b'

We say $\mathcal{B}$ is (γ, ε) -quantum-list unpredictable if

$$\Pr[\text{ExpUnpred}_{0,\mathcal{A}}^{\mathcal{B}} = 1] \leq \gamma \left(\Pr[\text{ExpUnpred}_{1,\mathcal{A}}^{\mathcal{B}} = 1] + \varepsilon \right).$$

We prove the quantum-list unpredictability of **VOLECommit** used in **FAEST** in Section 6.10, using the adaptive reprogramming lemma from [GHHM21] and a gentle measurement argument to analyze the measurement in Line 3.a.

6.4 QROM Security of FAEST

Using these techniques, we obtain a EUF-CMA security bound for **FAEST** in the QROM providing meaningful concrete post-quantum security for **FAEST**.

Theorem 56 (FAEST is EUF-CMA-secure in the QROM). *Let $\text{H}_0, \text{H}_1, \text{H}_2^1, \text{H}_2^2, \text{H}_2^3$ and H_4 be modeled as quantum-accessible random oracles and H_3 be a PRF. Then for any QPT adversary $\mathcal{A}$ which makes Q_{sig} queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to $\text{H}_0, \text{H}_1, \text{H}_2^1, \text{H}_2^2$ and H_2^3 , respectively,*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-KO}_{\mathcal{A}}^{\text{FAEST}} \\ &+ 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{\text{H}_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{PRG}_{4\lambda}}[1] \right) \\ &+ Q_{\text{sig}} \cdot \left(2 \left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}} \right) \cdot 2^{-\lambda} + 2Q_{2,1} \cdot 2^{-2\lambda} + 3\sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}} + 3\sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}}} \right), \end{aligned}$$

where $Q = Q_0 + Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4 + 3\tau + 19$ and

$$\text{AdvEUF-KO}_{\mathcal{A}}^{\text{FAEST}} \leq \left(1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})} \right)^{-1} \cdot (10 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot \tau \cdot 2^{-\lambda} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}}).$$

The security bound has a multiplicative loss of ≈ 8 with respect to the PRG security of AES-CTR, and the additive loss terms are all at most a Grover’s search advantage larger than their counterparts in Theorems 37 and 38. See Corollary 62 for a bound for **FAEST-EM**.

6.5 Proof Overview

In the following sections, we present a proof of EUF-CMA security (Theorem 56) in the quantum-accessible random oracle model. The proof follows the following strategy.

The strategy for reducing breaking EUF-CMA to EUF-KO (Theorem 61) is identical to that in the ROM proof. Whenever a random oracle needs to be reprogrammed to a fresh, random output at a location that has min-entropy given the adversary’s view, we use the adaptive reprogramming lemma (also called “resampling lemma”) [GHHM21]. When replacing chall_1 by a random value, however, the corresponding input to H_1 is only computationally unpredictable for the adversary. To exploit this computational unpredictability in a clean but tight way, we prove the unpredictability of the BAVC commitments with “quantum guesses” (see Lemma 60).

For proving EUF-KO security, we employ a lossy key argument to reduce from soundness of FAEST as a non-interactive argument. This step is identical to the ROM reduction. We would then like to follow the same strategy as in the ROM reduction: switch to an interactive version of FAEST and then exploit round-by-round (RBR) soundness. It is, however, unclear how to perform these two steps *separately* in the QROM: The equivalence of soundness of the Fiat-Shamir (FS) transformation of a given interactive proof system (IP) and its state restoration soundness seems to be unlikely in this setting, and the natural IP underlying FAEST does not have RBR soundness without idealizing the BAVC commitment. Instead we go one step further and formulate FAEST as a (slightly non-standard) FS transformation of an IP with more rounds, interpreting every random oracle call, including the ones inside BAVC.Commit in FAEST as a FS challenge generation. This way, we get to a RBR-sound IP in one step, as the commitments in the this IP are the statistically-binding leaf commitments (with the tree commitment that comes on top interpreted as rounds of interaction). This facilitates a QROM reduction via compressed-oracle-based search bound techniques [CFHL21, AHJ+23] (Lemma 59).

6.6 FAEST as Non-Standard Fiat-Shamir Transformation of 7-Challenge IP (6-Challenge for FAEST-EM).

We begin by formulating FAEST as a slightly non-standard Fiat-Shamir transformation of a 7-challenge (6-challenge for FAEST-EM) interactive proof system, FAEST-IP⁺. The 8 (7) prover messages of this protocol are

1. $\text{pmsg}_{-3} = \text{iv}^{\text{pre}}$
2. $\text{pmsg}_{-2} = \iota$, the empty string
3. $\text{pmsg}_{-1} = \text{com}_{0,0} \parallel \text{com}_{0,1} \parallel \dots \text{com}_{0,N_0} \parallel \text{com}_{1,0} \parallel \dots \text{com}_{\tau-1,N_{\tau-1}}$ as produced by FAEST.Sign in subroutine VOLECommit in subrouting BAVC.Commit
4. $\text{pmsg}_0 = \iota$, the empty string
5. pmsg_1 is the argument of H_2^1 in FAEST.Sign with μ, com and iv removed
6. pmsg_2 is the argument of H_2^2 in FAEST.Sign with chall_1 removed
7. pmsg_3 is the argument of H_2^3 in FAEST.Sign with chall_2 removed
8. pmsg_4 is the signature, with chall_3 removed.

For FAEST-EM, pmsg_{-2} is omitted. For clarity, we keep the numbering of rounds in this case, meaning that the prover messages are $\text{pmsg}_{-3}, \text{pmsg}_{-1}, \dots$ and similar for the challenges. The challenges are

1. $\text{chall}_{-3} = \text{iv}$
2. $\text{chall}_{-2} = (\text{uhash}_0, \dots, \text{uhash}_{\tau-1})$
3. $\text{chall}_{-1} = h_0 \parallel \dots \parallel h_\tau$
4. $\text{chall}_0 = \text{com}$
5. chall_1
6. chall_2
7. chall_3

For FAEST-EM, chall_{-2} is omitted. The honest prover computes the prover messages as FAEST.Sign would compute the corresponding quantities, except: i) it samples $r, \text{iv}^{\text{pre}}$ at random, ii) there is only one attempt to obtain a chall_3 with a suffix of w_{grind} zeroes (denote the variant of FAEST where Line 4 of FAEST.Sign is replaced by random sampling, and where only one, say random, value of ctr is attempted, by FAEST _{r} (FAEST-EM _{r})), and iii) it uses the challenges in place of hash values where FAEST.Sign calls

H_0, H_1, H_4 or any of the $H_2^i, i = 1, 2, 3$. Note that as the change from FAEST to FAEST_r leaves verification unchanged and thus does not affect soundness.

The FAEST-IP⁺ verifier runs FAEST.Verify, omitting Lines 2,3,10 and 15, and replacing Lines 18 and 19. Instead of Lines 18 and 19, the verifier compares the result $\tilde{a}'_0$ of Line 17 with the value $\tilde{a}_0$ from pmsg₃, and rejects if they differ, accepts otherwise. In case of FAEST-EM, the FAEST-IP⁺ verifier additionally rejects if any of the com_{i,j} have more than one pre-image. This verifier is not efficient, which is not a problem as we are only interested in soundness of the IP.

Note that FAEST-IP⁺ is in the plain model: We replaced the call to H₃ by random sampling and all other hash calls by interaction.

We now describe the Fiat-Shamir transformation we apply to FAEST-IP⁺ to get back to a variant of FAEST (with random iv^{pre}, r and with an equivalent verification). For readability, we omit the output length parameter of the hash functions. The challenges are derived as follows.

1. chall₋₃ = H₄(pmsg₋₃)
2. chall₋₂ = H₀(chall₋₃)
3. chall₋₁ = h₀ || ... || h_τ with h_i = H₁(com_{i,0} || com_{i,1} || ... || com_{i,N_i})
4. chall₀ := H₁(chall₋₁ || pmsg₀)
5. chall₁ := H₂¹(H₂⁰(pk || msg) || com || pmsg₁ || iv^{pre})
6. chall₂ := H₂²(chall₁ || pmsg₂)
7. chall₃ := H₂³(chall₂ || pmsg₃)

There are three main ways in which this transformation deviates from all standard variants of Fiat-Shamir: i) chall₋₁ is derived without hashing the previous challenge. The commitment com_{i,j} is, however, an injective function of uhash_i. ii) The function for deriving chall₋₁ is not a random oracle. The two-step computation of com is, however, only for efficiency (to allow parallel hashing), so it should be plausible that this is not a problem, and will be taken into account in the proof. iii) The instance (the public key) is pre-hashed and only included in the hash argument for deriving chall₁, the 5th challenge.

The standard Fiat-Shamir prover (outputting all pmsg_i) and verifier (recomputing all chall_i) with challenges derived as described above yield a variant FAEST_r^{elc} (“explicit leaf commitment”) of FAEST_r (FAEST-EM_r^{elc} of FAEST-EM_r) that is equivalent from a security point of view, and differs only in efficiency (see Lemma 59)

6.7 RBR Soundness of FAEST-IP⁺.

We prove RBR soundness for the expanded IP.

Lemma 57. FAEST-IP⁺ has RBR soundness with passive prefix up to chall₋₃, where challenge chall₋₂ is isolated, with RBR soundness errors

$$\varepsilon_{-3} = \varepsilon_{-1} = \varepsilon_0 = 0 \tag{11}$$

$$\varepsilon_{-2} = \tau \cdot 2^{-\lambda} \tag{12}$$

$$\varepsilon_1 = \varepsilon_2 = \varepsilon_3 = \varepsilon := 3 \cdot 2^{-\lambda} \tag{13}$$

Proof. Define the state function RBRState to output 1 if one of the values uhash_i in chall₋₂ defines a non-injective $\mathcal{H}_{\text{uhash}_i}$. If that is not the case, let $g_1 = (\mathbf{u}_i, \mathbf{V}_i)_i$ be the VOLEs defined by the unique preimages of the com_{i,j}. If none of the values uhash_i in chall₋₂ defines a non-injective $\mathcal{H}_{\text{uhash}_i}$, RBRState computes its output using the RBR state function from Lemma 49 on input $(\mathbf{c}_1 || \dots || \mathbf{c}_{\tau-1} || \text{iv}^{\text{pre}}) || g_1$. Now the expression for ε_{-2} follows by the analysis in the proof of Lemma 32, and the expression for $\varepsilon_1, \varepsilon_2, \varepsilon_3$ is due to Lemma 49. The passive prefix and isolation statements follow by construction of RBRState.

For FAEST-EM, it suffices to additionally observe that we defined the verifier to reject whenever the extraction of the com_{i,j} yields multiple pre-images. $\square$

6.8 FAEST_r^{elc} Soundness to FAEST-IP⁺ RBR Soundness.

We prove the soundness of FAEST based on the round-by-round soundness of FAEST-IP⁺, using Lemma 25 and the notation introduced in Section 6.2. For signature schemes, we consider a notion of soundness where an adversary presented with an invalid public key cannot produce a valid message-signature pair, except with the soundness advantage AdvSnd.

Lemma 58. For any static QROM adversary $\mathcal{A}^{H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4}$ against $\text{FAEST}_r^{\text{elc}} = \text{FS}[\text{FAEST-IP}^+]$ that makes a total of $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_4$ quantum queries to its oracles $H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4$, respectively,

$$\text{AdvSnd}_{\mathcal{A}}^{\text{FAEST}_r^{\text{elc}}} \leq 10 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot \tau \cdot 2^{-\lambda}.$$

For FAEST-EM, the advantage is instead bounded as

$$\text{AdvSnd}_{\mathcal{A}}^{\text{FAEST-EM}_r^{\text{elc}}} \leq 10(\tau + 1) \cdot Q^3 \cdot 2^{-2\lambda} + 30 \cdot Q^2 \cdot 2^{-\lambda}.$$

Here,

$$Q = Q_0 + Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4 + 2\tau + 12.$$

Proof. We instantiate the random oracles via domain separation of a random oracle H with sufficiently large output length for this proof, by setting $H_i^j = H(i \| j \| \cdot)$ truncated to the correct output length, with appropriate encoding of the number prefixes, where $H_i^0 := H_i$ for $i \in \{0, 1, 3, 4\}$. For ease of notation, we define $\hat{H}_i$ such that $\hat{H}_i$ is used to derive chall_i . In particular, $\hat{H}_{-1}$ is the function used to map $\text{com}_{0,0} \| \text{com}_{0,1} \| \dots \| \text{com}_{0,N_0} \| \text{com}_{1,0} \| \dots \| \text{com}_{\tau-1, N_{\tau-1}}$ to $h_0 \| \dots \| h_{\tau-1}$. Let T^H be the set of transcripts for H and $\mathcal{R}_i^j$ the range of H_i^j , $\hat{\mathcal{R}}_i$ the range of $\hat{H}_i$, $M_i^j = |\mathcal{R}_i^j|$ and $\hat{M}_i = |\hat{\mathcal{R}}_i|$. To use Lemma 25, we define the following predicate on pairs of partial transcripts $\text{trc} = (\text{msg}, \mu, \text{pmsg}_{-3}, \text{chall}_{-3}, \dots, \text{chall}_i)$,

$$\begin{aligned} \text{pred}_{\text{pk}, H}(\text{trc}, \text{msg}) = & \text{RBRState}(\text{pk}, \text{trc}) \\ & \wedge \neg \text{RBRState}(\text{pk}, \text{trc}_{i-1}) \\ & \wedge \left(\forall j \in [-3 : i] \setminus \{-1, 1\} : \text{chall}_j = \hat{H}_j(\text{chall}_{j-1} \| \text{pmsg}_j) \right) \\ & \wedge \left(i < -1 \vee \text{chall}_{-1} = \hat{H}_{-1}(\text{pmsg}_{-1}) \right) \\ & \wedge \left(i < 1 \vee \text{chall}_1 = \hat{H}_1(\mu \| \text{chall}_0 \| \text{pmsg}_1) \right) \\ & \wedge H_2^0(\text{pk}, \text{msg}) = \mu \end{aligned}$$

Here we have defined the truncation $\text{trc}_{i-1} = (\text{msg}, \mu, \text{pmsg}_1, \text{chall}_1, \dots, \text{chall}_{i-1})$. If $\text{pred}_{\text{pk}, H}(\text{trc}, \mu)$, we say (trc, μ) is *bad* for H . We further define a predicate $P : (\mathcal{Y} \cup \{\perp\})^{\mathcal{X}} \rightarrow \{0, 1\}$ by $P(D) : \Leftrightarrow$ there exists $\text{bad}(\text{trc}, \mu)$ for D . P has $(\tau + 4)$ -local witnesses. In particular if (trc, μ) is bad for D and trc ends in chall_{-3} or chall_{-2} , then a 1-local witness exists as chall_{-2} is isolated and chall_{-3} has RBR soundness error 0 so trc cannot actually end in chall_{-2} and fulfil pred . If trc ends in chall_i for $i \in [-1 : 3]$, the list of random oracle inputs in the domains of H_1, H_2^0 and $\hat{H}_i, i = 1, 2, 3$, queried when evaluating $\text{pred}_{\text{pk}, H}(\text{trc}, \text{msg})$ is a witness if $\text{pred}_{\text{pk}, H}(\text{trc}, \mu)$. The at most $\tau + 5$ inputs include the τ inputs to H_1 for deriving chall_{-1} . We call i the length of the witness. In the following we denote by D_i^j and $\hat{D}_i$ the partial functions obtained in the same way as H_i^j and $\hat{H}_i$ are obtained from H . We also define the collision predicate P_C by setting $P_C(D)$ iff there exist inputs x, x' in the domain of the same hash function H_1 or $H_2^i, i = 0, 1, 2, 3$ such that $D(x) = D(x') \neq \perp$. Note that this prevents collisions for chall_{-1} as well as $\hat{H}_0 = H_1$ is used for deriving chall_{-1} . The collision predicate clearly has $(\tau + 5)$ -local witnesses (it has 2-local witnesses). We construct algorithm $\mathcal{B}^H$ as follows. $\mathcal{B}^H$ runs $(\text{msg}, \pi) \leftarrow \mathcal{A}^H$ and recomputes the challenges. Denote the completed transcript by trc . If there exists i such that $\text{RBRState}(\text{pk}, \text{trc}_i)$ and $\neg \text{RBRState}(\text{pk}, \text{trc}_{i-1})$, output the predicate witness z , else, output $\perp$. By RBR soundness (Lemma 57) we have

$$\begin{aligned} \Pr_{\pi \leftarrow \mathcal{A}^H(\text{pk})} [\text{verifier accepts } \pi] & \leq \Pr_{z \leftarrow \mathcal{B}^H} [\langle z, P, H \rangle] \\ & \leq \Pr_{z \leftarrow \mathcal{B}^H} [\langle z, P \vee P_C, H \rangle] \end{aligned} \tag{14}$$

We now aim to bound the quantity

$$\max_{\substack{x, D: \\ |D| \leq k \\ \neg P'(D)}} \Pr_{u \leftarrow \mathcal{Y}} [P'(D[x \mapsto u])]$$

for $P' = P \vee P_C$. Here, D is a partial function with the same domain and range as H . We denote by $|D_i|$ the number of different inputs x in the domain of $\hat{H}_i$ such that $D(x) \neq \perp$. We distinguish different cases depending on which domain x is from.

1. x is in the domain of $\hat{H}_{-3}$. As the RBR soundness error for this round is 0 and RBR soundness holds with passive prefix up to chall_{-2} , we have

$$\Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq 0$$

2. x is in the domain of $\hat{H}_{-2}$. There are two ways to fulfill $\mathbf{P}'(D[x \mapsto u])$, i) if a collision occurs, i.e., $\exists x' : \hat{D}_{-2}(x) = \hat{D}_{-2}(x') \neq \perp$, or ii) if a u is chosen such that $\text{RBRState}(\text{pk}, \tau|x|u) = 1$. By Lemma 57, chall_{-2} is isolated, thus we have

$$\Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq \frac{|\hat{D}_{-2}|}{\hat{M}_{-2}} + \tau \cdot 2^{-\lambda} =: \hat{\delta}_{-2}$$

using the RBR soundness error given by Lemma 57.

3. x is in the domain of H_1 . For D with $\neg \mathbf{P}'(D)$, there are 2 ways to fulfil $\mathbf{P}'(D[x \mapsto u])$. The first one is if a new collision is found, i.e. $\exists x' : D_1^0(x) = D_1^0(x') \neq \perp$. This happens with probability at most $\frac{k}{\hat{M}_1^0}$. The second one is that there exists $x' = h_0 \| \dots \| h_{\tau-1}$ in the domain of H_1 such that $D_1^0(x') \neq \perp$ and there exists $i \in [0, \tau)$ such that $u = h_i$, or $x' = \mu \| \text{com} \| \text{pmsg}_0$ in the domain of $\hat{H}_1$ such that $u = \text{com}$. This happens with probability at most $\frac{\tau k}{\hat{M}_1} + \frac{k}{\hat{M}_2}$. In summary, we get

$$\Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq \frac{(\tau + 1)|D_1|}{\hat{M}_1} + \frac{\hat{D}_1}{\hat{M}_1} =: \delta_1.$$

4. x is in the domain of H_2^0 . The only way to fulfil $\mathbf{P}'(D[x \mapsto u])$ in this case is by forming a collision, so

$$\Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq \frac{|D_2^0|}{\hat{M}_i} =: \delta_{2,0}$$

5. x is in the domain of $\hat{H}_i$ for $i = 1, 2$. For D with $\neg \mathbf{P}'(D)$, there are 3 ways to fulfil $\mathbf{P}'(D[x \mapsto u])$. The first option is that there exists x' in the domain of $\hat{H}_i$, $x' \neq x$ such that $D(x') = u$ (i.e. a new collision has formed). The second option is that a new witness of length $i' > i$ for $\mathbf{P}$ is completed. In this case, there exists pmsg_{i+1} such that $\hat{D}_{i+1}(u \| \text{pmsg}_{i+1}) \neq \perp$. The third option is that a new witness of length i for $\mathbf{P}$ is completed. In this case, it is necessary that there exists a partial witness (which is unique by $\neg \mathbf{P}_C(D)$)

$$z = \left(0 \| 0 \| \text{iv}, 1 \| 1 \| \text{com}_{0,0} \| \dots \| \text{com}_{0,N_0}, \dots, 1 \| 1 \| \text{com}_{\tau-1,0} \| \dots \| \text{com}_{\tau-1,N_{\tau-1}}, \right. \\ \left. 1 \| 1 \| \text{chall}_{-1}, 2 \| 1 \| \mu \| \text{chall}_0 \| \text{pmsg}_1 \right)$$

if $i = 2$, and a partial witness of the same form except without the last entry if $i = 1$, with the following properties. Let $\text{chall}_{i-1} = \hat{D}_{i-1}(z_{\text{last}})$, where z_{last} is the last entry of z , let trc be the transcript obtained from z , and let trc_{i-2} be its truncation to the second-to-last challenge. Then

- (a) z is consistent, i.e., recomputing the chall_i that appear in the entries of z using D succeeds
- (b) $\text{RBRState}(\text{pk}, \text{trc}_{i-2}) = 0$
- (c) there exists pmsg_i such that $x = 2 \| i \| \text{chall}_{i-1} \| \text{pmsg}_i$ and $\text{RBRState}(\text{pk}, (\text{pmsg}_1, \text{chall}_1, \dots, \text{pmsg}_i, u)) = 1$.

In summary, we have

$$\Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq \frac{|\hat{D}_i| + |\hat{D}_{i+1}|}{\hat{M}_i} + \varepsilon =: \hat{\delta}_i,$$

where the term $\frac{|\hat{D}_i| + |\hat{D}_{i+1}|}{\hat{M}_i}$ is from the first and second option, we have used RBR soundness from Lemma 57 to bound the probability

$$\Pr_{u \leftarrow \mathcal{Y}}[\text{RBRState}(\text{pk}, (\text{pmsg}_1, \text{chall}_1, \dots, \text{pmsg}_i, u))] \leq \varepsilon$$

in the third option, and ε is defined in Lemma 57.

6. x is in the domain of $\hat{H}_3$. The analysis here is identical to item 4., except that $\hat{H}_3$ is not included in the collision predicate, and chall_3 is the last challenge, so only option 3 exists for $\hat{H}_3$. We thus get

$$\Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq \varepsilon =: \hat{\delta}_3$$

in this case

In conclusion, we have

$$\max_{\substack{x, D: \\ |D| \leq k \\ \neg \mathbf{P}'(D)}} \Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}'(D[x \mapsto u])] \leq \max(\hat{\delta}_{-2}, \delta_1, \delta_{2,0}, \hat{\delta}_1, \hat{\delta}_2, \hat{\delta}_3) := \delta(k).$$

Using Eq. (14), and applying Lemma 25 to the $(\tilde{Q} + \tau + 7)$ -query algorithm $\mathcal{B}$, where $\tilde{Q} = Q_0 + Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4$, we obtain

$$\begin{aligned} & \Pr_{(\text{pk}, \pi) \leftarrow \mathcal{A}^{\text{H}_1, \text{H}_2^1, \dots, \text{H}_2^\ell}}[\text{pk} \notin \mathcal{L} \wedge \text{verifier accepts}] \\ & \leq \left(\sum_{k=1}^{\tilde{Q}+2\tau+12} \sqrt{10 \max_{\substack{x, D: \\ |D| \leq k \\ \neg \mathbf{P}(D)}} \Pr_{u \leftarrow \mathcal{Y}}[\mathbf{P}(D[x \mapsto u])]} \right)^2 \\ & \leq \left(\sum_{k=1}^{\tilde{Q}+2\tau+12} \sqrt{10\delta(k)} \right)^2 \\ & \leq 10(\tilde{Q} + 2\tau + 12) \sum_{k=1}^{\tilde{Q}+2(\ell+1)} \delta(k) \end{aligned}$$

Here, the third inequality is the Cauchy-Schwarz inequality. Evaluating the maximum for the FAEST parameters and upper-bounding by a sum when it depends on Q which term is larger we get $\delta(k) \leq (\tau + 1)(\tilde{Q} + 2\tau + 12)2^{-2\lambda} + \tau \cdot 2^{-\lambda}$

$$\begin{aligned} & \Pr_{(\text{pk}, \pi) \leftarrow \mathcal{A}^{\text{H}_1, \text{H}_2^1, \dots, \text{H}_2^\ell}}[\text{pk} \notin \mathcal{L} \wedge \text{verifier accepts}] \\ & \leq 10(\tau + 1)(\tilde{Q} + 2\tau + 12)^3 2^{-2\lambda} + 10(\tilde{Q} + 2\tau + 12)^2 \tau \cdot 2^{-\lambda}. \end{aligned}$$

For FAEST-EM, the term $10(\tilde{Q} + 2\tau + 12)^2 \tau \cdot 2^{-\lambda}$ is replaced by the next largest term that scales with Q^2 , $30(\tilde{Q} + 2\tau + 12)^2 \cdot 2^{-\lambda}$. $\square$

6.9 FAEST EUF-KO $\Leftarrow$ FAEST_r^{elc} Soundness

Lemma 59. Let $\mathcal{A}^{\text{H}_0, \text{H}_1, \text{H}_2^0, \text{H}_2^1, \text{H}_2^2, \text{H}_2^3, \text{H}_3, \text{H}_4}$ be a QROM EUF-KO adversary against FAEST that makes a total of $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_3, Q_4$ queries to its oracles. Then there exist a quantum PRG adversary $\mathcal{B}_1$ against OWF and a QROM adversary $\mathcal{B}^{\text{H}_0, \text{H}_1, \text{H}_2^0, \text{H}_2^1, \text{H}_2^2, \text{H}_2^3, \text{H}_4}$ against FAEST_r^{elc} that makes at most a total of $Q_0 + 1, Q_1 + \tau + 1, Q_{2,0} + 1, Q_{2,1} + 1, Q_{2,2} + 1, Q_{2,3} + 1, Q_4 + 1$ queries to its oracles such that

$$\text{AdvEUFKO}_{\mathcal{A}}^{\text{FAEST}} \leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot (\text{AdvSnd}_{\mathcal{B}}^{\text{FAEST}_r^{\text{elc}}} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}})$$

For FAEST-EM, there exists an adversary $\mathcal{C}$ against the almost injectivity (Definition 27) of PRG such that the following similar bound holds.

$$\text{AdvEUFKO}_{\mathcal{A}}^{\text{FAEST-EM}} \leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot (\text{AdvSnd}_{\mathcal{B}}^{\text{FAEST-EM}_r^{\text{elc}}} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}} + \text{AdvInj}_{\mathcal{C}}^{\text{PRG}_{2\lambda}}[L]).$$

Proof. First, define $\mathcal{A}'^{H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4}$ as identical to $\mathcal{A}$ except that it simulates H_3 locally. As FAEST.Verify does not call H_3 , this change is perfectly indistinguishable, so

$$\text{AdvSnd}_{\mathcal{A}}^{\text{FAEST}} = \text{AdvSnd}_{\mathcal{A}}^{\text{FAEST}_r} = \text{AdvSnd}_{\mathcal{A}'}^{\text{FAEST}_r}.$$

Now define a prover $\mathcal{B}$ for $\text{FAEST}_r^{\text{elc}}$ as follows. $\mathcal{B}(\text{pk})$ runs $(\text{msg}, \sigma) \leftarrow \mathcal{A}(\text{pk})$ and then $\text{FAEST.Verify}(\text{msg}, \text{pk}, \sigma)$, storing intermediate results of the verification computation as needed for the following. For each prover message of the 7-challenge IP, we will now describe how the data contained in it (and thus in a $\text{FAEST}_r^{\text{elc}}$ proof) that are not part of a FAEST_r signature are generated. For pmsg_{-1} , use the values $h_{i,j}$ hashed in Line 36 of BAVC.Reconstruct (as called by VOLEReconstruct , as called by FAEST.Verify) in place of $\text{com}_{i,j}$. For pmsg_i , $i = 1, 2, 3$, use the hash input of lines 10, 15, and 18 of FAEST.Verify , respectively. By construction, if FAEST.Verify accepts, then the resulting proof is accepted by $\text{FAEST}_r^{\text{elc}}$ verification. We have thus shown that

$$\text{AdvSnd}_{\mathcal{A}}^{\text{FAEST}} = \text{AdvSnd}_{\mathcal{B}}^{\text{FAEST}_r^{\text{elc}}}.$$

For FAEST-EM , the same argument shows that

$$\text{AdvSnd}_{\mathcal{A}}^{\text{FAEST-EM}} = \text{AdvSnd}_{\mathcal{B}}^{\widetilde{\text{FAEST-EM}_r^{\text{elc}}}},$$

where $\widetilde{\text{FAEST-EM}_r^{\text{elc}}}$ is identical to $\text{FAEST-EM}_r^{\text{elc}}$ except that the verifier does not reject when one of the $\text{com}_{i,j}$ has multiple pre-images. We now construct adversary $\mathcal{C}$ against the almost injectivity game from Definition 27 as follows. $\mathcal{C}$ runs $\mathcal{A}$ while simulating oracles locally. It then outputs the list of $\text{com}_{i,j}$ submitted by $\mathcal{A}$, together with the challenge iv output by H_4 when queried on iv^{pre} . We thus have

$$\text{AdvSnd}_{\mathcal{B}}^{\widetilde{\text{FAEST-EM}_r^{\text{elc}}}} - \text{AdvSnd}_{\mathcal{B}}^{\text{FAEST-EM}_r^{\text{elc}}} \leq \text{AdvInj}_{\mathcal{C}}^{\text{PRG}_{2\lambda}}[L].$$

The claim then follows by applying Eq. (7) proven in the proof of Theorem 39, which holds for any computational model. $\square$

6.10 Quantum-Guess Unpredictability of VOLECommit .

We start by proving a slight quantum generalization of Lemma 36.

Lemma 60 (VOLECommit is quantum-list unpredictable). *Fix some $\hat{\ell} \in \mathbb{N}$ and admissible param , $\text{param}_{\text{VOLE}}$. Also let $\beta = \frac{\lambda}{128} \in \mathbb{N}$. $\text{VOLECommit}(\cdot, \hat{\ell}, \text{param}, \text{param}_{\text{VOLE}})$ is (γ, ε) -quantum list unpredictable for any adversary $\mathcal{A}$ which makes at most Q_1 queries to H_1 , and outputs lists $\mathcal{L}$ of size at most Q_{com} , for*

$$\begin{aligned} \gamma &= e^{(4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{128}} \\ \varepsilon &= \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \\ &\quad + \left(3\sqrt{Q_1} + 2\sqrt{Q_{\text{com}}}\right) \cdot 2^{-\lambda} + Q_{\text{com}} \cdot 2^{-2\lambda} \end{aligned}$$

where by abuse of notation, we write $\text{AdvPRP}^{\text{LeafCommit}}$ for $\text{AdvPRP}^{\text{PRG}_{4\lambda}}$ in FAEST , and $\text{AdvPRP}^{\text{PRG}_{2\lambda}}$ in FAEST-EM .

Proof. The proof proceeds identically to the proof of Lemma 36, until Eq. (5). Now, define G_2'' where h_0 is sampled independently uniformly random. By Proposition 1 in [GHHM21] we have

$$|\Pr[G_2' = 1] - \Pr[G_2'' = 1]| \leq \frac{3}{2} \sqrt{Q_1} 2^{-\lambda}$$

Define G_2''' where additionally, com is sampled uniformly at random. Similarly we get

$$|\Pr[G_2'' = 1] - \Pr[G_2''' = 1]| \leq \frac{3}{2} \sqrt{Q_1} 2^{-\lambda}$$

Finally, we need to add the membership test measurement and analyze the probability that it returns 1. Define G_3' like G_2''' except that the membership test measurement from step 4. is applied (but the output

is not used). Let ε be the probability that the measurement outputs 1. Then by a gentle measurement lemma we get

$$|\Pr[G'_3 = 1] - \Pr[G''_2 = 1]| \leq 2\sqrt{\varepsilon}$$

Finally we observe that the outputs of G'_3 and G'_0 are identical except if the measurement in Line 4 returns 1. We thus get

$$|\Pr[G'_0 = 1] - \Pr[G'_3 = 1]| \leq \varepsilon$$

To bound ε , let $L = L_0 L_1 \dots L_{Q_{\text{com}}-1}$, where each subregister L_i contains a commitment guess. We observe that for fixed com , the projector corresponding to the 1 outcome of the measurement in step 4 is upper-bounded in the Loewner order as²⁵

$$P_{1,\text{com}} \leq \sum_{i=0}^{Q_{\text{com}}-1} |\text{com}\rangle\langle\text{com}|_{L_i} \otimes \mathbb{1}_{L_{i^c}},$$

where $L_{i^c} = L_0 L_1 \dots L_{i-1} L_{i+1} \dots L_{Q_{\text{com}}-1}$. Taking the expectation over a uniformly random com we get

$$\mathbb{E}_{\text{com} \leftarrow \{0,1\}^{2\lambda}} P_{1,\text{com}} \leq \frac{Q_{\text{com}}}{2^{2\lambda}} \mathbb{1}.$$

It follows that $\varepsilon \leq \frac{Q_{\text{com}}}{2^{2\lambda}}$. After collecting all the terms to obtain a bound on $\Pr[G_2 = 1]$, the proof is concluded identically as the proof of Lemma 36 from after Eq. (6). $\square$

6.11 FAEST EUF-CMA $\Leftarrow$ FAEST EUF-KO

Theorem 61 (FAEST is EUF-CMA secure). *Let $H_0, H_1, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as quantum-accessible random oracles and H_3 be a PRF. Then for any QPT adversary $\mathcal{A}$ which makes Q_{sig} queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to H_0, H_1, H_2^1, H_2^2 and H_2^3 respectively, there exists an EUF-KO adversary $\mathcal{B}$ such that*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-KO}_{\mathcal{B}}^{\text{FAEST}} \\ &+ 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \\ &+ 2 \cdot Q_{\text{sig}} \cdot \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRG}^{\text{LeafCommit}}_{\mathcal{B}_4}[N_0] + \text{AdvPRP}^{\text{PRG}_{\ell}}_{\mathcal{B}_5}[1] \right) \\ &+ 2 \cdot Q_{\text{sig}} \cdot \left(\left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}} \right) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right) \\ &+ Q_{\text{sig}} \frac{(Q_2^0 + Q_2^1 + 3)^2}{2^{2\lambda}} \\ &+ \frac{3}{2} \cdot \sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}} + \frac{3}{2} \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}}}, \end{aligned}$$

where again $\text{AdvPRP}^{\text{LeafCommit}}$ for $\text{AdvPRP}^{\text{PRG}_{4\lambda}}$ in FAEST, and $\text{AdvPRP}^{\text{PRG}_{2\lambda}}$ in FAEST-EM.

Proof. We follow the same sequence of game hops as in the proof of Theorem 51, excluding game G_{10} . We only prove bounds for game hops where they differ from the classical ones beyond advantage terms now being defined with respect to quantum algorithms, and show how to finish the reduction without G_{10} .

G_3 : In G_3 , the quantum-accessible random oracle is simulated using the compressed oracle technique [Zha19]. Upon a query to $\mathcal{O}_{\text{sig}}$, $\text{chall}_1 \in \{0,1\}^{5\lambda+64}$ is sampled at random. When $\mathcal{O}_{\text{sig}}$ is about to query H_2^1 on $\mu \parallel \text{com} \parallel c_1 \parallel \dots \parallel c_{\tau-1} \parallel iv$, a two-outcome measurement is applied to the compressed oracle database to test whether it contains an entry with this input. If yes, abort, else program H_2^1 to output chall_1 on this input.

We argue indistinguishability by reduction to unpredictability as stated in Lemma 60. We use a hybrid argument, where $G_{3,i}$ implements measurement and abort only for the first i queries to $\mathcal{O}_{\text{sig}}$. Clearly,

²⁵ The operator on the RHS in fact counts the number of times com appears in the list.

$G_{3,0} = G_2$ and $G_{3,Q_{\text{sig}}} = G_3$. Moreover, $G_{3,i-1}$ and $G_{3,i}$ only differ in the i -th $\mathcal{O}_{\text{sig}}$ query. The i -th reduction $\mathcal{A}_{\text{unpred},i}$ to quantum-list unpredictability plays unpredictability game in Lemma 60 such that it emulates $G_{3,i}$, except that for the i -th $\mathcal{O}_{\text{sig}}$ -query, it outputs its current state and the subregisters of all compressed oracle database input registers corresponding to the commitments. It then receives $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1})$ from the unpredictability experiment (instead of running VOLECommit), and then continues from there. By construction, $G_{3,i-1} = 1 \iff \text{ExpQUnpred}_{0,\mathcal{A}_{\text{unpred}}}^{\text{VOLECommit},\hat{\ell},\text{param},\text{param}_{\text{VOLE}}} = 1$ and $G_{3,i} = 1 \iff \text{ExpQUnpred}_{1,\mathcal{A}_{\text{unpred},i}}^{\text{VOLECommit},\hat{\ell},\text{param},\text{param}_{\text{VOLE}}} = 1$. Thus, from Lemma 60 and a hybrid argument we get

$$\begin{aligned} \Pr[G_3 = 1] &\leq e^{Q_{\text{sig}} \cdot (4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{128}} \cdot \left(\Pr[G_1 = 1] \right. \\ &\quad + Q_{\text{sig}} \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right. \\ &\quad \left. \left. + \left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}} \right) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right) \right) \end{aligned} \quad (15)$$

G_4 & G_5 : For G_4 and G_5 , instead of adding an abort condition, we just reprogram H_2^2 respectively H_2^3 to the randomly sampled values for chall_2 and chall_3 . By the adaptive reprogramming lemma (Proposition 1 in [GHHM21]) we get

$$\Pr[G_3 = 1] \leq \Pr[G_4 = 1] + \frac{3}{2} \cdot Q_{\text{sig}} \cdot \sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}}$$

and

$$\Pr[G_4 = 1] \leq \Pr[G_5 = 1] + \frac{3}{2} Q_{\text{sig}} \cdot \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}}}.$$

Now consider the candidate forgery that the adversary outputs in game G_9 . We now define two adversaries, an EUF-KO adversary $\mathcal{B}$ and a multi-target pre-image finding adversary $\mathcal{C}$, as follows. Both adversaries simulate $\mathcal{A}$ playing game G_9 . $\mathcal{B}$ simulates the reprogramming performed by G_9 internally. $\mathcal{C}$ also simulates the reprogramming performed by G_9 internally, but uses pre-image challenges in place of randomly sampled values for reprogramming. Once $\mathcal{A}$ has finished, $\mathcal{C}$ runs FAEST.Verify on $\mathcal{A}$'s candidate forgery and outputs any preimage it finds among the queries FAEST.Verify makes. Now suppose $\mathcal{B}$ outputs a candidate forgery that is a successful forgery under the reprogrammed oracle simulated by $\mathcal{B}$ but invalid under the un-reprogrammed oracle that $\mathcal{B}$ has access to. Then one of H_2^0 , H_2^1 and H_2^2 is queried by FAEST.Verify on an input that was reprogrammed by the simulated signature oracle. The input for H_2^0 includes the message, i.e., it has to be different from all reprogrammed inputs for a valid forgery. There are now two cases left to consider.

1. Suppose FAEST.Verify queries H_2^1 on a reprogrammed input. Then the forgery message leads to the same value μ as one of the signing oracle queries.
2. Suppose FAEST.Verify queries H_2^1 on a fresh input but H_2^2 on a reprogrammed input. Then the value chall_1 recomputed by FAEST.Verify using the fresh H_2^1 -query coincides with one generated in a signing query.

As the pre-image finding challenges are uniformly random, we can consider $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ in the same probability space. Due to the considerations above we get the equation

$$\Pr[G_9 = 1] \leq \text{AdvEUFKO}_{\mathcal{B}} + \Pr[\mathcal{C} \text{ succeeds}].$$

Using a standard bound [HRS16], we have

$$\Pr[\mathcal{C} \text{ succeeds}] \leq Q_{\text{sig}} \frac{(Q_2^0 + Q_2^1 + 3)^2}{2^{2\lambda}}.$$

Here we have added 1 to the query numbers for H_2^0 and H_2^1 to account for the fact that $\mathcal{C}$ runs FAEST.Verify . Collecting all loss terms, including the ones from the games hops that are identical to Theorem 51, we get the desired bound. $\square$

We can now combine the above to get the EUF-CMA security of FAEST in the QROM. The part of the corollary below that is about FAEST is a restatement of Theorem 56.

Corollary 62 (FAEST is EUF-CMA secure in the QROM). *Let $H_0, H_1, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as quantum-accessible random oracles and H_3 be a PRF. Then for any QPT adversary $\mathcal{A}$ which makes Q_{sig} queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to H_0, H_1, H_2^1, H_2^2 and H_2^3 respectively,*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{FAEST}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-KO}_{\mathcal{A}}^{\text{FAEST}} \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{LeafCommit}}[1] \right) \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}} \right) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right) \\ &\quad + 3 \cdot Q_{\text{sig}} \cdot \left(\sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}} + \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda+64}}} \right) \end{aligned}$$

with

$$\text{AdvEUF-KO}_{\mathcal{A}}^{\text{FAEST}} \leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot (10 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot \tau \cdot 2^{-\lambda} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}}),$$

for FAEST.

For FAEST-EM, there exists a QROM adversary against the almost injectivity of PRG such that

$$\begin{aligned} \text{AdvEUF-KO}_{\mathcal{A}}^{\text{FAEST-EM}} &\leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot (10 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot 2^{-\lambda} \\ &\quad + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}} + \text{AdvInj}_{\mathcal{C}}^{\text{PRG}_{2\lambda}}[L]). \end{aligned}$$

Here,

$$Q = Q_0 + Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4 + 3\tau + 19,$$

and again we write $\text{AdvPRP}^{\text{LeafCommit}}$ for $\text{AdvPRP}^{\text{PRG}_{4\lambda}}$ in FAEST, and $\text{AdvPRP}^{\text{PRG}_{2\lambda}}$ in FAEST-EM.

Remark 63. We have chosen to instantiate all random oracles via domain separation to facilitate the proof of Lemma 58 in its relatively simple form. A tighter bound differentiating between the query number of the different random oracles instead of upper-bounding by the sum can be obtained by proving a straightforward, albeit tedious, generalization of Lemma 25 to multiple distinct quantum-accessible random oracles and applying it to improve Lemma 58.

7 Implementation and Benchmarks

To benchmark our improvements we have implemented the new variants of the FAEST signature scheme in C++ based on the implementation of [FAE23, BBM⁺24]. All experiments were run on a notebook with AMD Ryzen 7 PRO 4750U processor.

LeafCommit. First we provide microbenchmarks of the new PRG-based leaf-commitment functions and compare them with the previous SHAKE-based approach in Table 3. We see that for $\lambda = 128$ the *Full* variant is ca. 7× faster and the *Simple* variant is 15× faster than the SHAKE-based commitments. The improvement is less pronounced for $\lambda \in \{192, 256\}$ with ca. 2.4× for *Full* and 3.9× for *Simple*.

Table 3. Runtimes in ns per commitment for the different LeafCommit options.

LeafCommit	Size	$\lambda = 128$		$\lambda = 192$		$\lambda = 256$	
SHAKE	2λ	164.07		165.56		166.64	
PRG-Full	3λ	22.89	(13.95%)	65.93	(39.82%)	73.21	(43.93%)
PRG-Simple	2λ	10.44	(6.36%)	41.73	(25.21%)	42.75	(25.65%)

Vector Commitments. We benchmarked the batch all-but-one vector commitments with the parameter sets $(\lambda, \tau, w_{\text{grind}}, T_{\text{open}})$ that is used in the normal, non-EM FAEST variants and present the results in Table 4. For $\lambda = 128/192/256$, using the *Full* PRG leaf commitment, the runtime of both committing and verifying is reduced to at most 27/74/83% of the runtime with SHAKE. With the *Simple* PRG leaf commitment, it is reduced to at most 20/53/59%. Note that with the *Full LeafCommit* the output size is 3λ compared to 2λ with the SHAKE and the *Simple* alternatives. Thus, the total size of the commitments that need to get hashed together in BAVC.Commit grows and affects the efficiency.²⁶

Table 4. Runtimes in μs for BAVC.Commit and Reconstruct.

λ	LeafCommit	Fast			Small		
		$(\tau, w_{\text{grind}}, T_{\text{open}})$	Commit	Reconstruct	$(\tau, w_{\text{grind}}, T_{\text{open}})$	Commit	Reconstruct
128	SHAKE		1 085	1 094		8 436	8 524
	PRG-Full	(16, 8, 110)	245	253	(11, 7, 102)	2 274	2 270
	PRG-Simple		172	179		1 533	1 651
192	SHAKE		1 336	1 355		12 726	11 016
	PRG-Full	(24, 8, 163)	977	997	(16, 12, 162)	8 124	8 082
	PRG-Simple		688	698		5 725	5 818
256	SHAKE		2 028	2 065		18 220	18 364
	PRG-Full	(32, 8, 246)	1 614	1 612	(22, 6, 245)	14 900	15 337
	PRG-Simple		1 155	1 158		10 758	10 696

Signatures. Finally, we examine the effect of integrating *both* our new LeafCommit and the optimized zero-knowledge proof into the FAEST signature scheme. We compare against the FAEST version from [BBM⁺24] (“FAESTER”) in Table 5. For the standard variants, we use the *Full LeafCommit* and for the EM variants we use the *Simple LeafCommit*.

The sizes of public and secret keys for the 192/256-bit variants have been reduced by 16 B due to encrypting two related instead of two random blocks. The signature sizes have been reduced by 88 B for 128S up to 2560 B for EM-256F by our optimizations to the zero-knowledge proofs. For signing, we see the highest reductions of 40 % to 56 % for the 128-bit variants. For 192 and 256 bit, we see 2 % to 19 % for the standard variants and 20 % to 35 % for the EM variants. For verification, the improvement is almost always a few percentage points higher.

Acknowledgements and Thanks. This work has been supported by VILLUM FONDEN under the grant VIL53029, the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme under grant agreements number 101115790 (OBELiSC) and 101124977 (DECRYPTSYS), a DFF Sapere Aude grant 10.46540/2064-00034B (IM-3PQC), the Danish Independent Research Council under the grant DFF-0165-00107B (C3PO), the Digital Europe Program under project number 101091642 (“QCI-CAT”), the project “PREPARED” which is funded by Austrian security research programme KIRAS of the Federal Ministry of Finance (BMF).

We would like to thank the anonymous reviewers at IACR CRYPTO and Keita Xagawa for their helpful comments and suggestions.

²⁶ Without counting the final hash of all the leaf commitments, the percentages would be 23/55/60% and 17/40/45% for *Full* and *Simple*, respectively.

Table 5. Key and signature sizes in B, and runtimes in μs for different FAEST variants.

Variant		sk	pk	Fast			Small		
				σ	Sign	Verify	σ	Sign	Verify
128	[BBM ⁺ 24]	32	32	6 052	878	802	4 594	6 485	5 790
	this	32	32	5 924	524	432	4 506	3 878	3 041
192	[BBM ⁺ 24]	56	64	16 100	2 149	1 996	12 028	18 545	14 598
	this	40	48	14 948	2 030	1 759	11 260	15 038	11 475
256	[BBM ⁺ 24]	64	64	28 084	3 321	3 226	21 752	25 169	25 364
	this	48	48	26 548	3 067	2 901	20 696	24 716	24 726
EM-128	[BBM ⁺ 24]	32	32	5 444	852	786	4 170	6 389	6 077
	this	32	32	5 060	455	337	3 906	2 809	2 253
EM-192	[BBM ⁺ 24]	48	48	13 532	1 944	1 899	10 108	17 423	16 800
	this	48	48	12 380	1 467	1 333	9 340	11 401	10 576
EM-256	[BBM ⁺ 24]	64	64	26 036	3 260	3 119	19 744	25 382	24 553
	this	64	64	23 476	2 615	2 431	17 984	17 537	17 004

References

- AHJ⁺23. Carlos Aguilar Melchor, Andreas Hülsing, David Joseph, Christian Majenz, Eyal Ronen, and Dongze Yue. SDitH in the QROM. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VII*, volume 14444 of *LNCS*, pages 317–350. Springer, Singapore, December 2023.
- BBB⁺25. Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. Shorter, tighter, FAESTer: Optimizations and improved (QROM) analysis for VOLE-in-the-head signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 124–156. Springer, Cham, August 2025.
- BBD⁺23. Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 581–615. Springer, Cham, August 2023.
- BBM⁺24. Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 463–493. Springer, Singapore, December 2024.
- BCC⁺24. Dung Bui, Eliana Carozza, Geoffroy Couteau, Dahmun Goudarzi, and Antoine Joux. Faster signatures from MPC-in-the-head. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 396–428. Springer, Singapore, December 2024.
- BCD24. Dung Bui, Kelong Cong, and Cyprien Delpech de Saint Guilhem. Improved all-but-one vector commitment with applications to post-quantum signatures. Cryptology ePrint Archive, Report 2024/097, 2024.
- BCS16. Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part II*, volume 9986 of *LNCS*, pages 31–60. Springer, Berlin, Heidelberg, October / November 2016.
- BDK⁺21. Carsten Baum, Cyprien Delpech de Saint Guilhem, Daniel Kales, Emmanuela Orsini, Peter Scholl, and Greg Zaverucha. Banquet: Short and fast signatures from AES. In Juan Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 266–297. Springer, Cham, May 2021.
- BHK⁺19. Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS⁺ signature framework. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2129–2146. ACM Press, November 2019.
- BJKS94. Jürgen Bierbrauer, Thomas Johansson, Gregory Kabatianskii, and Ben Smeets. On families of hash functions via geometric codes and concatenation. In Douglas R. Stinson, editor, *CRYPTO’93*, volume 773 of *LNCS*, pages 331–342. Springer, Berlin, Heidelberg, August 1994.
- CCH⁺18. Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. Fiat-Shamir from simpler assumptions. Cryptology ePrint Archive, Report 2018/1004, 2018.
- CDG⁺17. Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1825–1842. ACM Press, October / November 2017.
- CFHL21. Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 598–629. Springer, Cham, October 2021.
- CW79. Larry Carter and Mark N. Wegman. Universal classes of hash functions. *J. Comput. Syst. Sci.*, 18(2):143–154, 1979.
- CY24. Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. self-published, 2024.
- DDOS19. Cyprien Delpech de Saint Guilhem, Lauren De Meyer, Emmanuela Orsini, and Nigel P. Smart. BBQ: Using AES in picnic signatures. In Kenneth G. Paterson and Douglas Stebila, editors, *SAC 2019*, volume 11959 of *LNCS*, pages 669–692. Springer, Cham, August 2019.
- DFMS19. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 356–383. Springer, Cham, August 2019.
- DFMS22a. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Efficient NIZKs and signatures from commit-and-open protocols in the QROM. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 729–757. Springer, Cham, August 2022.

- DFMS22b. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 677–706. Springer, Cham, May / June 2022.
- DH23. Changchang Ding and Yan Huang. Dubhe: Succinct zero-knowledge proofs for standard AES and related applications. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 4373–4390. USENIX Association, August 2023.
- Dil17. Dilithium team. Crystals-dilithium: Algorithm specifications and supporting documentation. <https://pq-crystals.org/dilithium/data/dilithium-specification.pdf>, accessed 13.02.2025, 2017.
- DKR⁺22. Christoph Dobraunig, Daniel Kales, Christian Rechberger, Markus Schofnegger, and Greg Zaverucha. Shorter signatures based on tailor-made minimalist symmetric-key crypto. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 843–857. ACM Press, November 2022.
- FAE23. FAEST Team. FAEST: Algorithm specifications (version 1.1). 2023. <https://faest.info/faest-spec-v1.1.pdf>, accessed 13-02-2025.
- GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive re-programming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021.
- HJMN24. Andreas Hülsing, David Joseph, Christian Majenz, and Anand Kumar Narayanan. On round elimination for special-sound multi-round identification and the generality of the hypercube for MPCitH. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 373–408. Springer, Cham, August 2024.
- HRS16. Andreas Hülsing, Joost Rijneveld, and Fang Song. Mitigating multi-target attacks in hash-based signatures. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part I*, volume 9614 of *LNCS*, pages 387–416. Springer, Berlin, Heidelberg, March 2016.
- IKOS07. Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th ACM STOC*, pages 21–30. ACM Press, June 2007.
- KKW18. Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 525–537. ACM Press, October 2018.
- KLS24. Seongkwang Kim, Byeonghak Lee, and Mincheol Son. Relaxed vector commitment for shorter signatures. Cryptology ePrint Archive, Report 2024/1004, 2024.
- KW03. Jonathan Katz and Nan Wang. Efficiency improvements for signature schemes with tight security reductions. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *ACM CCS 2003*, pages 155–164. ACM Press, October 2003.
- KZ20. Daniel Kales and Greg Zaverucha. Improving the performance of the Picnic signature scheme. *IACR TCHES*, 2020(4):154–188, 2020.
- Nao91. Moni Naor. Bit commitment using pseudorandomness. *Journal of Cryptology*, 4(2):151–158, January 1991.
- Roy22. Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.
- RRR16. Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. Constant-round interactive proofs for delegating computation. In Daniel Wichs and Yishay Mansour, editors, *48th ACM STOC*, pages 49–62. ACM Press, June 2016.
- Sti92. Douglas R. Stinson. Universal hashing and authentication codes. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 74–85. Springer, Berlin, Heidelberg, August 1992.
- YSWW21. Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021.
- ZCD⁺20. Greg Zaverucha, Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Jonathan Katz, Xiao Wang, Vladimir Kolesnikov, and Daniel Kales. Picnic. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.